United States Patent Application Publication	20250278821
Kind Code	A1
Publication Date	September 04, 2025
Inventor(s)	Xiao; Zhisheng et al.

DENOISING DIFFUSION GENERATIVE ADVERSARIAL NETWORKS

Abstract

Apparatuses, systems, and techniques are presented to train and utilize one or more neural networks. A denoising diffusion generative adversarial network (denoising diffusion GAN) reduces a number of denoising steps during a reverse process. The denoising diffusion GAN does not assume a Gaussian distribution for large steps of the denoising process and applies a multi-model model to permit denoising with fewer steps. Systems and methods further minimize a divergence between a diffused real data distribution and a diffused generator distribution over several timesteps. Accordingly, various embodiments may enable faster sample generation, in which the samples are generated from noise using the denoising diffusion GAN.

Inventors: **Xiao; Zhisheng (Chicago, IL), Kreis; Karsten (Vancouver, CA), Vahdat; Arash (Mountain View, CA)**

Applicant: **Nvidia Corporation** (Santa Clara, CA)

Family ID: **85706367**

Appl. No.: **19/207065**

Filed: **May 13, 2025**

Related U.S. Application Data

parent US continuation 17957143 20220930 parent-grant-document US 12333688 child US 19207065
us-provisional-application US 63250372 20210930

Publication Classification

Int. Cl.: **G06T5/70** (20240101)

U.S. Cl.:

CPC **G06T5/70** (20240101); G06T2207/20081 (20130101); G06T2207/20084 (20130101); G06T2207/20182 (20130101)

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation application and claims priority to U.S. patent application Ser. No. 17/957,143, filed on Sep. 30, 2022, which is a non-provisional application and claims priority to U.S. Provisional Patent Application No. 63/250,372, filed on Sep. 30, 2021, the full disclosures of which are hereby incorporated by reference in their entirety for all purposes.

FIELD

[0002] At least one embodiment pertains to processing resources used to perform and facilitate artificial intelligence. For example, at least one embodiment pertains to processors or computing systems used to train neural networks according to various novel techniques described herein.

BACKGROUND

[0003] There has been progress in developing new deep generative learning frameworks. Yet, these frameworks often struggle with addressing three key requirements simultaneously and they often trade some of them for others. These requirements, termed as the generative learning trilemma, include: high sample quality, mode coverage, and fast sampling. Particularly, de-noising diffusion models have recently shown amazing sample quality and diversity, but, their expensive sampling does not yet allow them to be applied in many real-world applications.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] Various embodiments in accordance with the present disclosure will be described with reference to the drawings, in which:

[0005] FIG. **1** illustrates an example of an environment for a denoising diffusion generative adversarial network (GAN), according to at least one embodiment;

[0006] FIG. **2** illustrates an example of an environment for a denoising diffusion GAN, according to at least one embodiment;

[0007] FIG. **3**A illustrates an example of a representation of a true denoising distribution compared to marginal data distributions, according to at least one embodiment;

[0008] FIG. **3**B illustrates an example environment for training a denoising diffusion GAN, according to at least one embodiment;

[0009] FIG. **4**A illustrates an example flow chart of a process for generating an output using a denoising diffusion GAN, according to at least one embodiment;

[0010] FIG. **4**B illustrates an example flow chart of a process for determining a loss for a denoising diffusion GAN, according to at least one embodiment;

[0011] FIG. **5** illustrates an example flow chart of a process of training a denoising diffusion GAN, according to at least one embodiment;

[0012] FIG. **6** illustrates a distributed system, in accordance with at least one embodiment;

[0013] FIG. **7** illustrates an exemplary datacenter, in accordance with at least one embodiment;

[0014] FIG. **8** illustrates a client-server network, in accordance with at least one embodiment;

[0015] FIG. **9** illustrates a computer network, in accordance with at least one embodiment;

[0016] FIG. **10**A illustrates a networked computer system, in accordance with at least one embodiment;

[0017] FIG. **10**B illustrates a networked computer system, in accordance with at least one

embodiment;
[0018] FIG. **10**C illustrates a networked computer system, in accordance with at least one embodiment;
[0019] FIG. **11** illustrates one or more components of a system environment in which services may be offered as third party network services, in accordance with at least one embodiment;
[0020] FIG. **12** illustrates a cloud computing environment, in accordance with at least one embodiment;
[0021] FIG. **13** illustrates a set of functional abstraction layers provided by a cloud computing environment, in accordance with at least one embodiment;
[0022] FIG. **14** illustrates a supercomputer at a chip level, in accordance with at least one embodiment;
[0023] FIG. **15** illustrates a supercomputer at a rack module level, in accordance with at least one embodiment;
[0024] FIG. **16** illustrates a supercomputer at a rack level, in accordance with at least one embodiment;
[0025] FIG. **17** illustrates a supercomputer at a whole system level, in accordance with at least one embodiment;
[0026] FIG. **18**A illustrates inference and/or training logic, in accordance with at least one embodiment;
[0027] FIG. **18**B illustrates inference and/or training logic, in accordance with at least one embodiment;
[0028] FIG. **19** illustrates training and deployment of a neural network, in accordance with at least one embodiment;
[0029] FIG. **20** illustrates an architecture of a system of a network, in accordance with at least one embodiment;
[0030] FIG. **21** illustrates an architecture of a system of a network, in accordance with at least one embodiment;
[0031] FIG. **22** illustrates a control plane protocol stack, in accordance with at least one embodiment;
[0032] FIG. **23** illustrates a user plane protocol stack, in accordance with at least one embodiment;
[0033] FIG. **24** illustrates components of a core network, in accordance with at least one embodiment;
[0034] FIG. **25** illustrates components of a system to support network function virtualization (NFV), in accordance with at least one embodiment;
[0035] FIG. **26** illustrates a processing system, in accordance with at least one embodiment;
[0036] FIG. **27** illustrates a computer system, in accordance with at least one embodiment;
[0037] FIG. **28** illustrates a system, in accordance with at least one embodiment;
[0038] FIG. **29** illustrates an exemplary integrated circuit, in accordance with at least one embodiment;
[0039] FIG. **30** illustrates a computing system, according to at least one embodiment;
[0040] FIG. **31** illustrates an APU, in accordance with at least one embodiment;
[0041] FIG. **32** illustrates a CPU, in accordance with at least one embodiment;
[0042] FIG. **33** illustrates an exemplary accelerator integration slice, in accordance with at least one embodiment;
[0043] FIGS. **34**A-**34**B illustrate exemplary graphics processors, in accordance with at least one embodiment;
[0044] FIG. **35**A illustrates a graphics core, in accordance with at least one embodiment;
[0045] FIG. **35**B illustrates a GPGPU, in accordance with at least one embodiment;
[0046] FIG. **36**A illustrates a parallel processor, in accordance with at least one embodiment;
[0047] FIG. **36**B illustrates a processing cluster, in accordance with at least one embodiment;
[0048] FIG. **36**C illustrates a graphics multiprocessor, in accordance with at least one embodiment;

[0049] FIG. **37** illustrates a software stack of a programming platform, in accordance with at least one embodiment;
[0050] FIG. **38** illustrates a CUDA implementation of a software stack of FIG. **37**, in accordance with at least one embodiment;
[0051] FIG. **39** illustrates a ROCm implementation of a software stack of FIG. **37**, in accordance with at least one embodiment;
[0052] FIG. **40** illustrates an OpenCL implementation of a software stack of FIG. **37**, in accordance with at least one embodiment;
[0053] FIG. **41** illustrates software that is supported by a programming platform, in accordance with at least one embodiment; and
[0054] FIG. **42** illustrates compiling code to execute on programming platforms of FIGS. **37-40**, in accordance with at least one embodiment.

DETAILED DESCRIPTION

[0055] Systems and methods of the present disclosure overcome one or more of the aforementioned problems by providing a denoising diffusion generative adversarial network (denoising diffusion GAN, DDGAN, etc.) that directly reduces a number of denoising steps during a reverse process. Such a system may solve traditional convergence issues associated with GANs while also reducing sampling costs to enable wider ranges of real-world applications. Various embodiments eliminate the large number of denoising steps of existing models that assume denoising distributions can be modeled by a Gaussian distribution. Because this assumption is applicable to small denoising steps, very large numbers of denoising steps are required for existing models, thereby making them unsuitable for real-world applications. In at least one embodiment, instead of minimizing a divergence between real and diffused data at an end of the process, systems and methods minimize the divergence between a diffused real data distribution and a diffused generator distribution over several timesteps (t). During training, the generator may be updated by backpropagating the gradient of a forward diffusion chain and then through a discriminator. Moreover, various embodiments may inject a diffusion-based Gaussian mixture distribution into the discriminator, which may enable the denoising process to approximate several steps at a time. Accordingly, various embodiments may enable faster sample generation, in which the samples are generated from noise using the denoising diffusion GAN.
[0056] As noted, slow sampling in certain models can be attributed to, for example, a Gaussian assumption in the denoising step which holds only for small step sizes. To enable denoising for large steps, and hence, to reduce the total number of denoising steps, systems and methods of the present disclosure model the denoising distribution using a complex multimodal distribution. For example, embodiments include denoising diffusion GANs that model each denoising step using a multimodal conditional GAN. Implementation of these denoising diffusion GANs provide sample quality and diversity on-par with original diffusion models with greater speeds. Additionally, compared to traditional GANs, the present disclosure exhibits improved mode coverage and sampled diversity. Accordingly, embodiments of the present disclosure reduce the sampling cost in diffusion models to an extent that allows real-world applications of diffusion models at a low computational cost.
[0057] Various embodiment solve the slow sampling problem of diffusion models by using an expressive and multi-modal denoising distribution. In particular, each denoising step may be modeled with a conditional Generative Adversarial Network (GAN), where the GAN is given a noisy observation and tries to produce a less noisy sample. The GAN is trained by adversarial loss. Conditioned on the same noisy observation, the discriminator of the GAN tries to distinguish whether a denoised sample is generated by the generator or sampled in the previous step of the forward process (in which case it is a true denoising sample), and the generator tries to produce denoised samples conditioned on the same noisy observation. During sampling, the process can start with white noise and repeat the denoising step a few times to denoise the white noise into clean data. Accordingly, embodiments introduce a generative model as a combination of diffusion models and GANs, where each denoising step of the diffusion model is modeled with a conditional GAN.

Previous work either trained a GAN that produces samples in one step without explicit denoising, or trained a diffusion model that does denoising without GAN components. In particular, almost all prior work on diffusion models use Gaussian distribution to model the denoising distribution. In other words, given a noisy observation, these models assume the denoised data is centered around a single point. Such an assumption is valid when the denoising step size is very small, and this is why previous models need a large number of denoising steps to accommodate the small step size. However, such an assumption does not hold with a large denoising step size. Embodiments of the present disclosure use a fundamentally different assumption on denoising distribution by assuming a multi-modal and complex distribution modeled with conditional GANs. To this end, denoising may be accomplished in a smaller number of steps.

[0058] FIG. **1** illustrates a schematic representation **100** of an environment in which embodiments of the present disclosure may be practiced. In this example, an input **102** may be processed by a denoising diffusion GAN **104** to generate an output **106**. The input **102** may be provided by a user or as part of an image or video processing pipeline, among other various options. For example, the denoising diffusion GAN **104** may be used for image generation, where an input **102** includes an input image or frame and the output **106** includes an output image or frame of an item within the image, such as from a different angle. Additionally, other than content generation, various embodiments may also be associated with representation learning or artistic tools. Further non-limiting examples of processes that may incorporate one or more aspects of the denoising diffusion GAN **104** include image super-resolution, text-to-image generation, image synthesis, controllable generation, image editing, image-to-image generation, segmentation, video synthesis, medical imaging, three-dimensional (3D) generation, discrete state modeling, and the like. Accordingly, it should be appreciated that the respective inputs **102** and outputs **106** may vary based, at least in part, on the application associated with the denoising diffusion GAN **104**.

[0059] Many varieties of deep generative models have been applied to a broad range of applications, such as image, audio, point cloud and graph generation in cross-domain alignment, and semi-supervised learning, among others. However, current generative learning frameworks cannot yet simultaneously satisfy three key requirements, often needed for the wide adoption of generative models in real-world problems. These requirements include (i) high-quality sample generation, (ii) mode coverage and sample diversity, and (iii) fast and computationally inexpensive sampling. For example, most current works in image synthesis focus on high-quality generation. However, mode coverage and data diversity are important for representing minorities in the training data and for reducing the negative social impacts of generative models. Additionally, applications such as interactive image editing require fast sampling. Present models often sacrifice one of the requirements for another, and therefore, may not be suitable to a variety of applications.

[0060] Generative adversarial networks (GANs) generate high-quality samples rapidly, but they are known for poor mode coverage. As a result, GANs often fail requirement (ii). Conversely, likelihood-based models, such as variational autoencoders (VAEs) and normalizing flows cover data modes faithfully, but they often suffer from low sample quality, failing requirement (i). Diffusion models have emerged as powerful generative models. Diffusion models define a forward diffusion process that maps data to noise by gradually perturbing the input data. Data generation is achieved using a parametrized reverse process that performs iterative denoising, starting from random noise. Diffusion models demonstrate surprisingly sufficient results in sample quality, beating GANs in image generation. They also demonstrate good mode coverage, indicated by high likelihood. Diffusion models have already been applied to a variety of generation tasks. However, sampling from them often requires 1000s of network evaluations, thereby failing requirement (iii), making their application to real-world problems expensive.

[0061] At least one embodiment tackles this generative learning trilemma by reformulating denoising diffusion models specifically for fast sampling while maintaining strong mode coverage and sample quality. An approach can involve investigating the slow sampling issue of diffusion models and observing that diffusion models commonly assume that the denoising distribution can be approximated by Gaussian distributions. However, the Gaussian assumption holds only in the infinitesimal limit of small denoising steps, which leads to the requirement of a large number of steps in the reverse process. When the reverse process has a larger step size (i.e., it has fewer denoising steps), a non-Gaussian multi-modal distribution can be used for modeling the denoising distribution. Intuitively, in image synthesis, the multi-modal distribution arises from the fact that multiple plausible clean images may correspond to the same noisy image when denoising for a large step.

[0062] To enable denoising for large steps, embodiments of the present disclosure incorporate parametrizing the denoising distribution with an expressive multi-modal distribution. In particular, at least one embodiment introduces a novel generative learning framework termed as denoising diffusion GAN in which the denoising distribution is modeled with conditional GANs. In image generation, by way of non-limiting example, systems and methods can obtain sample quality and mode coverage competitive with diffusion models while taking only as few as two denoising steps, achieving about a 2000× speed-up in sampling. Compared to traditional GANs, systems and methods described herein outperform state-of-the-art GANs in sample diversity while being competitive in sample fidelity.

[0063] Various embodiments of the present disclosure may, therefore, overcome the problems of existing frameworks while also address the requirements of the generative learning trilemma. Systems and methods may employ complex, multi-modal denoising distributions in order to address the overcome the slow sampling of diffusion models using a Gaussian assumption. In at least one embodiment, denoising diffusion GANs employ a reverse process that is parametrized by conditional GANs. This feature, among various others described herein, may achieve several orders of magnitude speed-up compared to current diffusion models for both image generation and editing. Additionally, various embodiments, overcome the deep generative learning trilemma to a large extent, making the diffusion models applicable to real-world applications at a low computational cost.

[0064] FIG. **2** is a schematic representation **200** of the denoising diffusion GAN **104**. As noted, an input **102**, such as an image or textual input, is provided to the denoising diffusion GAN **104**, which may be trained to generate the output **106**. As an example, if the input **102** corresponds to a textual input, such as “a teddy bear wearing a green hat” the denoising diffusion GAN **104** may be trained to generate an image according to those parameters, for example, after evaluation of the input prompt. Various features, such as pre-processing steps, post processing steps, training information, variable latent inputs, and the like have been omitted for clarity, but it should be appreciated that various additional components and modules may be associated with the denoising diffusion GAN **104**.

[0065] A diffusion module **202** may be used to gradually add noise (in a forward process) or to remove noise (in a reverse process) from one or more inputs, such as an image input. Regarding the forward diffusion process, the data x.sub.0˜q(x.sub.0) in T steps with pre-defined variance schedule β.sub.t may be defined as:

[00001] $q(x_{1:T} \mid x_0) = \text{.Math.}_{t \geq 1}\ q(x_t \mid x_{t-1}), q(x_t \mid x_{t-1}) = \ (x_t; \sqrt{1 - \ _t} x_{t-1}, \ _t I)$ (1)

where q(x.sub.0) is a data generating distribution. The reverse denoising process is defined by:

[00002] $p\ (x_{0:T}) = p(x_T)\ \text{.Math.}_{t \geq 1}\ p\ (x_{t-1} \mid x_t),$ (2)

$p\ (x_{t-1} \mid x_t) = \ (x_{t-I};\ \ (x_t, t),\ _t^2 I)$

where μ.sub.θ(x.sub.t,t) and σ.sub.t.sup.2 are the mean and variance for the denoising model and θ denotes its parameters. In training, systems and methods seek to maximize the likelihood p.sub.θ(x.sub.0)=∫p.sub.θ(x.sub.0:T)dx.sub.1:T, by maximizing the evidence lower bound (ELBO) (custom-character≤log p.sub.θ(x.sub.0)). The ELBO can be written as matching the true denoising distribution q(x.sub.t-1|x.sub.t) with the parameterized denoising model p.sub.θ(x.sub.t-1|x.sub.t) using:

[00003] $\mathcal{L} = -\ \text{.Math.}_{t \geq 1}\ \mathbb{E}_{q(x_t)}[D_{KL}(q(x_{t-1} \mid x_t)\ \text{.Math.}\ p\ (x_{t-1} \mid x_t))] + C,$ (3)

where C contains constant terms that are independent of θ and D.sub.KL denotes the Kullback-Leibler (KL) divergence. The objective above is intractable due to the unavailability of q(x.sub.t-1|x.sub.t). However, custom-character can be written in an alternative form with tractable distributions.

[0066] Two key assumptions are commonly made in diffusion models. First, the denoising distribution p.sub.θ(x.sub.t-1|x.sub.t) is modeled with a Gaussian distribution. Second, the number of denoising steps T is often assumed to be in the order of hundreds to thousands of steps. These assumptions may be relevant for both discrete-time diffusion models and in continuous-time diffusion models.

[0067] The denoising distribution q(x.sub.t-1|x.sub.t) can be written as q(x.sub.t-1|x.sub.t)∝q(x.sub.t|x.sub.t-1)q(x.sub.t-1) using the Bayes' rule where q(x.sub.t|x.sub.t-1) is the forward Gaussian diffusion shown in Equation 1 and q(x.sub.t-1) is the marginal data distribution at step t. It can be shown that in two situations the true denoising distribution takes a Gaussian form. First, in the limit of infinitesimal step size β.sub.t, the product in the Bayes' rule is dominated by q(x.sub.t|x.sub.t-1) and the reversal of the diffusion process takes an identical functional form as the forward process. Thus, when β.sub.t is sufficiently small, since q(x.sub.t|x.sub.t-1) is a Gaussian, the denoising distribution q(x.sub.t-1|x.sub.t) is also Gaussian, and the approximation used by current diffusion models can be accurate. In order to satisfy this constraint, diffusion models often have thousands of steps with small β.sub.t. If the data marginal q(x.sub.t) is Gaussian, the denoising distribution q(x.sub.t-1|x.sub.t) is also a Gaussian distribution. While there have been suggestions to bring the data distribution closer to Gaussian using a VAE encoder, such as within LSGM, there remain challenges with transforming the data to Gaussian itself and VAE encoders cannot solve it perfectly or with sufficient precision for the present embodiments. That is why LSGM still requires tens to hundreds of steps on complex datasets. Accordingly, neither of the conditions are met (i.e., when the denoising step is large and the data distribution is non-Gaussian, there are no guarantees that the Gaussian assumption on the denoising distribution holds.)

[0068] FIG. **3**A illustrates a graphical representation **300** including marginal diffused data distribution **302** and true denoising distributions **304**. In this example, the respective distributions **302** are shown through the forward process. In this example, it can be seen that the distributions **304** become more multi-modal as the step size increases. For example, at a first step, the distribution **304**A is substantially Gaussian. However, at a third step **304**C or fourth step **304**D, complexity increases. Accordingly, the assumption with respect to the Gaussian distribution for denoising fails when larger step sizes are implemented.

[0069] In at least one embodiment, a goal can be to reduce the number of denoising diffusion steps T required in the reverse process of diffusion models. Moreover, due to the inaccuracy of the Gaussian assumption for larger step sizes, shown in FIG. **3**A, the denoising distribution can be modeled with an expressive multi-modal distribution. Given that conditional GANs have been shown to model complex conditional distributions in image domain, embodiments of the present disclosure adopt them to approximate the true denoising distribution q(x.sub.t-1|x.sub.t).

[0070] Specifically, forward diffusion in embodiments of the present disclosure is set up similar to the diffusion models in Equation 1, but with an assumption that T is assumed to be small (e.g., smaller than the number of steps used with existing methods). It should be appreciated that the “small” number of steps T may vary based, at least in in part, on the application being processed by the system. In at least one embodiment, T is less than or equal to 8. However, it should be appreciated that T may also be less than or equal to 4. Furthermore, in at least one embodiment, T may be between 4 and 8. Additionally, in other embodiments, T may be greater than 8, greater than 10, greater than 10, greater than 50, or within various changes such as between 5 and 10, between 10 and 15, between 10 and 30, between 50 and 100, or other reasonable ranges. Moreover, each diffusion step has a larger β.sub.t.

[0071] Training may be formulated by matching the conditional GAN generator p.sub.θ(x.sub.t-

1|x.sub.t) and q(x.sub.t-1|x.sub.t) using an adversarial loss that minimizes a divergence D.sub.adv per denoising step using:

[00004] $$\min_{\theta} \sum_{t \geq 1} \mathbb{E}_{q(t)}[D_{\mathrm{adv}}(q(x_{t-1} \mid x_t) \| p_{\theta}(x_{t-1} \mid x_t))], \quad (4)$$

where D.sub.adv can be Wasserstein distance, Jenson-Shannon divergence, or f-divergence depending on the adversarial training setup.

[0072] At least one embodiment can rely on non-saturating GANs, which are widely used in successful GAN frameworks such as StyleGANs. In this case, D.sub.adv takes a special instance of f-divergence called softened reverse KL, which is different from the forward KL divergence used in the original diffusion model training in Equation 3.

[0073] To set up the adversarial training, a time-dependent discrimination is set as D.sub.φ(x.sub.t-1, x.sub.t, t):custom-character.sup.N×custom-character.sup.N×custom-character.fwdarw.[0,1], with parameters φ, taking N-dimensional x.sub.t-1 and x.sub.t as inputs, and deciding whether x.sub.t-1 is a plausible denoised version of x.sub.t. A discrimination may **204** then be trained by:

[00005]

$$\min_{\phi} \sum_{t \geq 1} \mathbb{E}_{q(x_t)}[\mathbb{E}_{q(x_{t-1} \mid x_t)}[-\log(D_{\phi}(x_{t-1}, x_t, t))] + \mathbb{E}_{p_{\theta}(x_{t-1} \mid x_t)}[-\log(1 - D_{\phi}(x_{t-1}, x_t, t))]], \quad (5)$$

[0074] Given the discriminator **204**, a generator **206** may be trained by:

[00006] $$\max_{\theta} \sum_{t \geq 1} \mathbb{E}_{q(x_t)} \mathbb{E}_{p_{\theta}(x_{t-1} \mid x_t)}[\log(D_{\phi}(x_{t-1}, x_t, t))], \quad (6)$$

which updates the generator **206** with a non-saturating GAN objective.

[0075] Returning to FIG. **2**, the illustrated embodiment includes the input **102**, which can be a video, image, textual input, or the like, provided to the denoising diffusion GAN **104** for processing via the diffusion module **202**. As noted, the diffusion module may include one or both of the forward process or the reverse process, where the forward process may gradually add noise to the input **102** to a certain point (e.g., a point where the input **102** is unrecognizable or has reached a threshold level). In the reverse process, the number of denoising steps may be reduced, as noted above, and may incorporate a non-Gaussian, multimodal distribution for modeling the denoising distribution. By removing the Gaussian assumption, the reduction in steps may maintain generation quality while also improving speed.

[0076] In various embodiments, the generator **206** structure largely follows a U-net, which consists of multiple ResNet blocks and Attention blocks. Hyper-parameters for the network design, such as the number of blocks and number of channels, may vary. In at least one embodiment, the number of ResNet blocks is 2, the number of initial input channels is 64 or 128, scale of attention blocks may be 16, and the channel multiplier for each scale may be (1,2,2,2) or (1,1,2,2,4,4). In at least one embodiment, the latent dimension is 256 with 3 latent mapping layers and a latent embedding dimension of 512. It should be appreciated that these parameters are provided by way of one non-limiting example and are not intended to limit the scope of the present disclosure.

[0077] Various embodiments of the generator **206** further incorporate Swish activation function and upsampling and downsampling with anti-aliasing based on Finite Impulse Response. Additionally, re-scaling all skipped connections using residual block design from BigGAN and incorporating progressing growing architectures may also be used with embodiments of the present disclosure. Sinusoidal positional embeddings for conditioning on integer time steps may also be incorporated.

[0078] In at least one embodiment, an extra latent variable z is provided as input to the generator **206**. This latent variable may be defined as z˜custom-character(0,I). The group normalization (GN) layers in the network may be replaced with adaptive group normalization (AdaGN) layers to allow the input of latent variables. The latent variable z is first mapped by a fully connected network (called mapping network), and then the resulting embedding vector, denoted by w, will be sent to every AdaGN layer. Each AdaGN layer contains one fully connected layer that takes w as input, and outputs the per-channel shift and scale parameters for the group normalization. The feature map after groupnorm may go through an affine transformation using these parameters. The mapping network for the FC layer in AdaGN are independent of time steps t, but it should be appreciated that various

embodiments may incorporate time embedding in these layers.
[0079] In at least one embodiment, the discriminator **204** is a time-dependent discrimination designed with a convolution network with ResNet blocks, where the design of ResNet blocks is similar to that of the generator **206**. The discrimination **204** is positioned to discriminate x.sub.t-1 condiaiton on x.sub.t and t. The time conditioning is enforced by the same sinusoidal positional embeddings as in the generator **206**. The x.sub.t conditioning is enforced by concatenating x.sub.t and x.sub.t-1 as the input to the discriminator. LeakReLU activation with a negatives slope of 0.2 for all layers may be used. Additionally, embodiments may include a minibatch standard deviation layer after all the ResNet blocks. By way of non-limiting example, network structures for the discrimination **204** may include a 1×1 conv2d, 128, a ResBlock 128, a ResBlock down 256, a ResBlock 512, a minibatch std layer, Global Sum Pooling, and a FC layer.fwdarw.scalar.
[0080] As shown in FIG. **2**, after diffusion within the diffusion module **202**, an intermediate output is provided to both the discriminator **204** and the generator **206**, where a second intermediate output is provided by the generator **202**. The discriminator **204** may then evaluate at least these intermediate inputs, among others, and may provide feedback to improve the generator **206**, as will be described below. The generator **206** may then provide the output **106**, which as noted above, may vary based on various training parameters and also the input **102**.
[0081] Embodiments of the present disclosure may modify or otherwise adjust denoising parameters compared to existing network structures in order to reduce a number of steps in the reverse process to improve speeds, among other benefits. Instead of directly predicting x.sub.t-1 at the denoising step, diffusion models can be interpreted as parameterizing the denoising model by p.sub.θ(x.sub.t-1|x.sub.t):=q(x.sub.t-1|x.sub.t, x.sub.0)=f.sub.θ(x.sub.t, t)) in which first x.sub.0 is predicted using the denoising model f.sub.θ(x.sub.t, t), and then x.sub.t-1 is sampled using the posterior distribution q(x.sub.t-1|x.sub.t, x.sub.0) given x.sub.t and the predicted x.sub.0. The distribution q(x.sub.t-1|x.sub.t, x.sub.0) is the distribution over x.sub.t-1 when denoising from x.sub.t towards x.sub.0, and it may have a Gaussian form for the diffusion process in Equation 1 independent from the step size and the complexity of the data distribution. Similarly, p.sub.θ(x.sub.t-1|x.sub.t) may be defined by:
[00007]

$$p\ (x_{t-1} \mid x_t) := \int p\ (x_0 \mid x_t) q(x_{t-1} \mid x_t, x_0) dx_0 = \int p(z) q(x_{t-1} \mid x_t, x_0 = G\ (x_t, z, t))\mathrm{dz}, \quad (7)$$

where p.sub.θ(x.sub.0|x.sub.t) is the implicit distribution imposed by GAN generator G.sub.θ(x.sub.t, z, t): custom-character.sup.N×custom-character.sup.L×custom-character.fwdarw. custom-character.sup.N that outputs x.sub.0 given x.sub.t and L-dimensional latent variable z˜p(z):=custom-character(z; 0,I).
[0082] Such a parameterization provides multiple advantages over existing methods. For example, because p.sub.θ(x.sub.t-1|x.sub.t) is formulated similarly to a DDPM, inductive bias, such as the network structure design, may be incorporated. However, in various embodiments, x.sub.0 is produced by the generator with random latent variable z, and not predicted by a deterministic mapping of x.sub.t. Accordingly, the denoising distribution may become multimodal and complex in contrast to the unimodal denoising model in DDPM. Furthermore, in various embodiments, for different t's, x.sub.t has different levels of perturbation, and hence using a single network to predict x.sub.t-1 directly at different t may be difficult. However, various embodiments overcome this challenge because the generator **206** only needs to predict unperturbed x.sub.0 and then add back perturbation using q(x.sub.t-1|x.sub.t, x.sub.0).
[0083] FIG. **3**B is a schematic diagram of a training environment **350** that may be used with embodiments of the present disclosure. Denoising Diffusion GANs are trained using an adversarial training setup. Given a training image, a forward Gaussian diffusion process is used to sample from a pair of diffused samples at two successive steps. In this example, a training input **352** is provided at x.sub.0, which may be similar to the input **102** shown in FIG. **2**. Forward diffusion, for example via the diffusion module **202**, may generate an intermediate image **354** at x.sub.t-1. The intermediate image **354** is shown with more noise when compared to the training input **352**, which is a

substantially clear and/or higher quality image. As noted above, an image input is provided as an example only and other inputs may also be utilized during operation or training within the scope of the present disclosure. Forward diffusion may continue throughout the denoising distribution, for example, until x.sub.t, which is associated with a noisy image **356**.
[0084] In the illustrated configuration, both the intermediate image **354** and the noisy image **356** are provided as inputs to the discrimination **204**. Additionally, the noisy image **356** is further provided to the generator **206**, which is used to output a generated image **358** at x′.sub.0. For example, the GAN may be a conditional denoising GAN that stochastically generates the generated image **358**. Thereafter, a posterior distribution is used to generate an intermediate generated image **360** at x′.sub.t-1. As shown, noise is added back to the generated **358** to form the intermediate generated image **360**. This intermediate generated image **360** is then provided to the discrimination **204** along with both the intermediate image **354** and the noisy image **356**. The conditional GAN may then be trained to denoise inputs using an adversarial loss for different steps in the diffusion process.
[0085] In one example training process, a number of diffusion steps may be set, for example, to 4. Time steps may be equally spaced between 0 and 1. Forward diffusion may be completed using Equation 1, which may corresponding to a discretization of the Variance Preserving (VP) SDE. Accordingly, embodiments may choose β.sub.t by the variance function of VPSDE, which is given by:

[00008] $$^{2}(t) = 1 - e^{-\ {}_{min} t - 0.5(\ {}_{max} - \ {}_{min})t^{2}}, \quad (8)$$

where β.sub.max=20 and β.sub.min=0.1 as constants. In other words, β.sub.t may be determined by plugging in a discrete time t into σ.sup.2(t).
[0086] During training, a randomly sampled integer time step for each datapoint may be used within a batch. Additionally, a regularization term may be added to the objective for the discriminator. The model may then be trained using Adam optimizer, and may use cosine learning rate decay for training both the generator and discriminator.
[0087] Embodiments of the present disclosure provide several advantages over traditional GANs. GANs are known to suffer from training instability and mode collapse, and some possible reasons include the difficulty of directly generating samples from a complex distribution in one-shot, and the overfitting issue when the discriminator only looks at clean samples. In contrast, the present model breaks the generation process to several conditional denoising diffusion steps in which each step is relatively simple to model, due to the strong conditioning on x.sub.t. Moreover, the diffusion process smoothens the data distribution, making the discriminator less likely to overfit. Thus, such a model can be expected to exhibit better training stability and mode coverage.
[0088] Diffusion-based models learn the finite time reversal of a diffusion process, sharing the idea of learning transition operators of Markov chains. Since then, there have been a number of improvements and alternatives to diffusion models. One approach generalizes diffusion process to continuous time, and provide an unified view for diffusion models and denoising score matching. Another process can add an auxiliary adversarial loss to the main objective. Note that this is fundamentally different from embodiments of the present disclosure, as those auxiliary adversarial losses only act as an image enhancer, and do not use latent variables, therefore the de-noising distribution is still a unimodal Gaussian. Other explorations include introducing alternative noise distribution in the forward process, jointly optimizing the model and noise schedule, and applying the model to latent space.
[0089] One major drawback of diffusion or score-based models is the slow sampling speed due to a large number of iterative sampling steps. To alleviate this issue, multiple methods have been proposed, including knowledge distillation, learning an adaptive noise schedule, introducing non-Markovian diffusion processes, and using better SDE solvers for continuous-time models. In particular, one approach uses x.sub.0 sampling as a crucial ingredient to a method, but the denoising distribution is still a Gaussian. These methods either suffer from significant degrade in sample quality, or still require many sampling steps.

[0090] Among variants of diffusion models, one approach proposes to model the single-step denoising distribution by a conditional energy-based model (EBM), sharing the high-level idea of using expressive denoising distribution with us. However, this approach is from the perspective of facilitating the training of EBMs. More importantly, although only a few denoising steps are needed, expensive MCMC has to be used to sample from each denoising step, making the sampling process slow with ˜180 network evaluations. One approach explores modeling the denoising distribution of a diffusion process on discrete latent space with an auto-regressive model per step in a few denoising steps, however, the auto-regressive structure of their denoising distribution still makes sampling slow.
[0091] Since a model in accordance with at least one embodiment model is trained with adversarial loss, embodiments are related to recent advances in improving the sample quality and diversity of GANs, including data augmentation, consistency regularization, and entropy regularization. In addition, the idea of training generative models with smoothed distribution is possible for auto-regressive models.
[0092] An example denoising diffusion GAN can be evaluated for, as an example, an image synthesis problem. This can with briefly introducing the network architecture design, while additional implementation details are presented later herein. For a GAN generator, an NCSN++ architecture can be used which has a U-net structure. The conditioning x.sub.t is the input of the network, and time embedding is used to ensure conditioning on t. An approach can let the latent variable z control the normalization layers. In particular, all group normalization layers in NCSN++ can be replaced with adaptive group normalization layers in the generator, where the shifting and scaling parameters in group norm are predicted from z using a simple multi-layer fully-connected network.
[0093] One major highlight of such a model is that it excels at all three criteria in the generative learning trilemma. Here, an approach can carefully evaluate a model's performances on sample fidelity, sample diversity and sampling time, and benchmark against a comprehensive list of models on the CIFAR-10 dataset. As evaluation criteria, an approach can adopt the commonly used Fréchet inception distance (FID) and Inception Score (IS) for evaluating sample fidelity. For sample diversity, an approach can use an improved recall score, which is an improved version of an original precision and recall metric. It is shown that the improved recall score can reflect how the variation in the generated samples matches that in the training set. For sampling time, an approach can use the number of function evaluations (NFE) and the clock time when generating a batch of 100 images on V100 GPUs.
[0094] Embodiments provide sample quality that is competitive among the best diffusion models and GANs. Although some variants of diffusion models obtain better IS and FID, they require a large number of function evaluations to generate samples (while we this approach uses only 4 denoising steps). For example, a sampling time is ˜2000× faster and ˜20× faster than other approaches. Note that diffusion models can produce samples in fewer steps while trading off the sample quality. To better benchmark this method against existing diffusion models, the FID score may be plotted or compared versus sampling time of diffusion models by varying the number of denoising steps (or the error tolerance for continuous-time models). Embodiments of the present disclosure illustrate advantages over diffusion models. For example, when comparing such a model to GANs, it can be observed that only StyleGAN2 with adaptive data augmentation has slightly better sample quality than present embodiments. However, GANs have limited sample diversity, as their recall scores are below 0.5. In contrast, an example model obtains a significantly better recall score, even higher than several advanced likelihood-based models, and competitive among diffusion models. Accordingly, such a model simultaneously excels at sample quality, sample diversity, and sampling speed.
[0095] With respect to a number of denoising steps, the effect of using a different number of denoising steps (T) may be varied. Note that T=1 corresponds to training an unconditional GAN, as the conditioning x.sub.t contains almost no information about x.sub.0. Accordingly, T=1 may lead to significantly worse results with low sample diversity, indicated by a low recall score. This confirms

the benefits of breaking generation into several denoising steps, especially on improving the sample diversity. When varying T>1, it can be observed empirically that T=4 gives the best results, whereas there is a slight degrade in performance for larger T. However, embodiments are not limited to using 4 steps. An example approach may need a significantly higher capacity to accommodate for large T, as it may need a conditional GAN for each denoising step.

[0096] An example model shares some similarities with recent work on applying data augmentation to GANs, where the training data and generated samples are perturbed before applying the discriminator. To study the effect of perturbing inputs, a one-shot GAN can be trained with a network structure with Gaussian augmentation. The result, however, is significantly worse than a present T=4 model.

[0097] Two alternative ways to parametrize the denoising distribution can be examined for the same T=4 setting. Instead of letting the generator produce estimated samples of x.sub.0, an approach can set the generator to directly output denoised samples x.sub.t-1 without posterior sampling (direct denoising), or output the noise et that perturbs a clean image to produce x.sub.t (noise generation). Note that the latter case is closely related to most diffusion models where the network deterministically predicts the perturbation noise. It can be demonstrated that although these alternative parametrizations work reasonably well, an example parametrization outperforms them by a large margin.

[0098] Removing latent variables z converts an example denoising model to a unimodal distribution. Performance of such a model may be studied without any latent variables z. It can be determined that the sample quality is significantly worse, suggesting the importance of multi-modal denoising distribution, as used in embodiments of the present disclosure. The effect of latent variables is visualized by showing samples of p.sub.θ(x.sub.0|x.sub.1), where x.sub.1 is a fixed noisy observation. It can be demonstrated that while the majority of information in the conditioning x.sub.1 is preserved, the samples are diverse due to the latent variables.

[0099] Besides the recall score, an approach can also evaluate the mode coverage of embodiments on the popular 25-Gaussians and StackedMNIST. The 25-Gaussians dataset is a 2-D toy dataset, generated by a mixture of 25 two-dimensional Gaussian distributions, arranged in a grid. An approach can train a denoising diffusion GAN with 4 denoising steps and compare it to other models. It can be observed that the vanilla GAN suffers severely from mode collapse, and while techniques like WGAN-GP improve mode coverage, the sample quality is still limited. In contrast, a model covers all the modes while maintaining high sample quality. Such an approach can also train a diffusion model and plot the samples generated by 100 and 500 denoising steps. It can be seen that diffusion models require a large number of steps to maintain high sample quality.

[0100] StackMNIST contains images generated by randomly choosing 3 MNIST images and stacking them along the RGB channels. Hence, the data distribution has 1000 modes. A number of covered modes and the KL divergence are reported from the categorical distribution over 1000 categories of generated samples to true data. It can be observed that such a model covers all modes faithfully and achieves the lowest KL compared to GANs that are specifically designed for better mode coverage or StyleGAN2 that is known to have the best sample quality.

[0101] For high resolution images, a model can be trained on datasets with larger images, including CelebAHQ and LSUN Church Outdoor at 256×256px resolution. An approach can report FID on these two datasets. Similar to CIFAR-10, a model obtains competitive sample quality among the best diffusion models and GANs. In particular, in LSUN Church Outdoor, such a model outperforms both DDPM and ImageBART.

[0102] An interesting application of diffusion models to stroke-based generation is to perturb a stroke painting by the forward diffusion process, and denoise it with a diffusion model. The method is particularly promising because it only requires training an unconditional generative model on the target dataset and does not require training images paired with stroke paintings unlike GANs-based methods. An approach can apply a model as presented herein to stroke-based image synthesis. The generated samples are realistic and diverse, while the conditioning in the stroke painting is faithfully preserved. Compared to other approaches, such a model enjoys a 1100× speedup in generation, as it takes only 0.16 s to generate one image at 256 resolution vs. 181 s for this other approach. This experiment confirms that a model as presented herein enables the application of diffusion models to interactive applications such as image editing.

[0103] Deep generative learning frameworks still struggle with addressing the generative learning trilemma. The diffusion models particularly obtain high-quality and diverse sampling. However, their slow sampling and high computational cost do not yet allow them to be applied in real-world applications widely. In at least some embodiments, one of the main sources of slow sampling in diffusion models is the Gaussian assumption in the denoising distribution which holds only for very small denoising steps. To remedy this, denoising diffusion GANs can be used that model each denoising step using a complex multimodal distribution, enabling us to take large denoising steps. In extensive experiments, it has been shown that denoising diffusion GANs achieve high sample quality and diversity competitive to the original diffusion models while being orders of magnitude faster at sampling. Compared to traditional GANs, such a model enjoys better mode coverage and sample diversity. Such a denoising diffusion GAN overcomes the generative learning trilemma to a large extent, allowing the diffusion models to be applied to real-world problems with low computational cost.

[0104] FIG. **4**A illustrates an example process **400** for generating an output using a denoising diffusion GAN. It should be understood that for this and other processes presented herein that there can be additional, fewer, or alternative steps performed in similar or alternative order, or at least partially in parallel, within the scope of various embodiments unless otherwise specifically stated. In this example, a trained machine learning system receives an input **402**. The input may correspond to an input image, input video, input text, or the like. The input may be preprocessed in one or more steps, for example, to format the input to a particular resolution, to remove portions of the input not necessary for processing, to adjust a size of the input, or the like. In at least one embodiment, the trained machine learning system forms at least a portion of a processing pipeline which may be used for various applications such as image synthesis, text-to-image generation, controllable generation, image editing, image-to-image generation, super-resolution, segmentation, video synthesis, medical imaging, 3D generation, discrete state modeling, and the like.

[0105] The input may be processed and then provided to a trained denoising diffusion GAN **404**. As noted above, the denoising diffusion GAN may be trained so that the reverse process uses fewer steps than a traditional diffusion process. The denoising diffusion GAN may be trained on an input image that undergoes forward diffusion to generate one or more intermediate images that may be compared, by a discriminator, against a generated image, from a generator, that undergoes posterior sampling to add noise. As a result, the discriminator may be comparing noisy images to determine whether an image is real or fake (e.g., whether the noisy image is based on noise added to a real image or the noisy image is based on noise added to a generated image). Adversarial training provides a loss that is then fed back to the generator to improve the network to generate additional images. Such a generator may then be used to process the input to generate an output **406**. For example, a user may provide an input image to a trained denoising diffusion GAN that is trained to generate images having a different perspective or point of view compared to the input image. The output may correspond to one or more objects within the input image that appear from a different point of view. In another example, the input may include a text sequence corresponding to a desired output, such as “a dog wearing a cowboy hat,” and the trained denoising diffusion GAN may process this input and then generate one or more images corresponding to the input.

[0106] FIG. **4**B illustrates an example process **420** for determining loss for a denoising diffusion GAN. In this example, a noisy sample is provided to a conditional GAN **422**, which may form a portion of the denoising diffusion GAN. The noisy sample may be based, at least in part, on a clean input. For example, one or more steps associated with forward diffusion may add noise to the clean input to generate the noisy sample. The noisy sample may be at a particular time or step and may be noisier than a previous time or step.

[0107] In at least one embodiment, the conditional GAN is used to generate a sample that is denoised compared to the input noisy sample **422**. For example, the conditional GAN may generate a clean sample or a sample with less noise (e.g., a threshold amount of noise less than the noisy sample). In at least one embodiment, the generator may generate a clean sample that is then processed to add a threshold amount of noise, for example, to correspond to the noisy image to one or more steps associated with the noisy image. The generated sample is then compared to a true denoising sample **424**, for example within a trained discriminator. The determinator may then be used to determine a loss **426**, which can be used to update and improve the generator.
[0108] FIG. **5** illustrates an example process **500** for training a denoising diffusion GAN. In this example, a sample input is received **502**. The sample input may correspond to an image, but it should be appreciated that embodiments of the present disclosure are not restricted or limited to image analysis and may include various other applications. Noise may be added to the sample image over a denoising distribution **504**. For example, the distribution may correspond to a number of steps, where a different quantity or amount of noise is added to the sample image at subsequent steps. Therefore, a noise level at a first step would be less than a noise level at a second step, and so on. At the end of the distribution, the sample image may be entirely or substantially white noise such that the sample input is substantially unrecognizable, but it should be appreciated that the denoising distribution may not extend to such an extent.
[0109] Adding noise to the sample image may generate a series of noisy inputs, where each noisy input in the series has more noise than the one preceding it. For example, along the distribution, an image at a first step may have less noise than the next image at the second step, and so forth. As noted above, the first step may be considered an image at a time x.sub.t-1 and the second step may be considered an image at a time x.sub.t. A first noisy input may be selected **506**, where the first noisy input corresponds to the input at time x.sub.t-1. In at least one embodiment, this first noisy input may be an input, along the denoising distribution, that has had noise added from a clean sample image. The amount of noise and/or position along the denoising distribution may be recorded or otherwise used in subsequent steps. A second noisy input may also be selected, where the second noisy input corresponds to the input at time x.sub.t, which is later than the time x.sub.t-1, and therefore has more noise (e.g., is further along the distribution).
[0110] In at least one embodiment, the second noisy input is provided to a conditional generator **508**. Selection of the second noisy input may be based, at least in part, on the first noisy input, where the second noisy input may be the next step along the denoising distribution. It should be appreciated that in other embodiments the second noisy step may not be the direct next subsequent step and there may be intervening steps. The generator may then generate a first generated output from the second noisy input **510**. In various embodiments, the first generated output is a clean out (e.g., an output without noise or with a level of noise below a threshold). However, in certain embodiments, the generator may generate a noisy output. When the first generated output is clean or substantially clean, additional noise is added to the first generated output **512**. The noise may be added corresponding to the denoising distribution, such that the amount of noise added corresponds to the time x.sub.t-1, to form a second generated output. As described herein, a reduced number of steps may be utilized for the reverse diffusion process, which may increase a speed in which the system can generate an output.
[0111] A discriminator may receive, as an input, the first noisy input, the second noisy input, and the second generated output **514**. The discriminator may determine which image is “real” or “fake” and generate a loss **516** which may be used to update the generator **518**. This adversarial training process may be completed any number of times until a threshold level of accuracy is obtained.
Servers and Data Centers
[0112] The following figures set forth, without limitation, exemplary network server and datacenter based systems that can be used to implement at least one embodiment.
[0113] FIG. **6** illustrates a distributed system **600**, in accordance with at least one embodiment. In at least one embodiment, distributed system **600** includes one or more client computing devices **602**,

604, **606**, and **608**, which are configured to execute and operate a client application such as a web browser, proprietary client, and/or variations thereof over one or more network(s) **610**. In at least one embodiment, server **612** may be communicatively coupled with remote client computing devices **602**, **604**, **606**, and **608** via network **610**.

[0114] In at least one embodiment, server **612** may be adapted to run one or more services or software applications such as services and applications that may manage session activity of single sign-on (SSO) access across multiple datacenters. In at least one embodiment, server **612** may also provide other services or software applications can include non-virtual and virtual environments. In at least one embodiment, these services may be offered as web-based or cloud services or under a Software as a Service (SaaS) model to users of client computing devices **602**, **604**, **606**, and/or **608**. In at least one embodiment, users operating client computing devices **602**, **604**, **606**, and/or **608** may in turn utilize one or more client applications to interact with server **612** to utilize services provided by these components.

[0115] In at least one embodiment, software components **618**, **620** and **622** of system **600** are implemented on server **612**. In at least one embodiment, one or more components of system **600** and/or services provided by these components may also be implemented by one or more of client computing devices **602**, **604**, **606**, and/or **608**. In at least one embodiment, users operating client computing devices may then utilize one or more client applications to use services provided by these components. In at least one embodiment, these components may be implemented in hardware, firmware, software, or combinations thereof. It should be appreciated that various different system configurations are possible, which may be different from distributed system **600**. The embodiment shown in FIG. **6** is thus at least one embodiment of a distributed system for implementing an embodiment system and is not intended to be limiting.

[0116] In at least one embodiment, client computing devices **602**, **604**, **606**, and/or **608** may include various types of computing systems. In at least one embodiment, a client computing device may include portable handheld devices (e.g., an iPhone®, cellular telephone, an iPad®, computing tablet, a personal digital assistant (PDA)) or wearable devices (e.g., a Google Glass® head mounted display), running software such as Microsoft Windows Mobile®, and/or a variety of mobile operating systems such as iOS, Windows Phone, Android, BlackBerry 10, Palm OS, and/or variations thereof. In at least one embodiment, devices may support various applications such as various Internet-related apps, e-mail, short message service (SMS) applications, and may use various other communication protocols. In at least one embodiment, client computing devices may also include general purpose personal computers including, by way of at least one embodiment, personal computers and/or laptop computers running various versions of Microsoft Windows®, Apple Macintosh®, and/or Linux operating systems.

[0117] In at least one embodiment, client computing devices can be workstation computers running any of a variety of commercially-available UNIX® or UNIX-like operating systems, including without limitation a variety of GNU/Linux operating systems, such as Google Chrome OS. In at least one embodiment, client computing devices may also include electronic devices such as a thin-client computer, an Internet-enabled gaming system (e.g., a Microsoft Xbox gaming console with or without a Kinect® gesture input device), and/or a personal messaging device, capable of communicating over network(s) **610**. Although distributed system **600** in FIG. **6** is shown with four client computing devices, any number of client computing devices may be supported. Other devices, such as devices with sensors, etc., may interact with server **612**.

[0118] In at least one embodiment, network(s) **610** in distributed system **600** may be any type of network that can support data communications using any of a variety of available protocols, including without limitation TCP/IP (transmission control protocol/Internet protocol), SNA (systems network architecture), IPX (Internet packet exchange), AppleTalk, and/or variations thereof. In at least one embodiment, network(s) **610** can be a local area network (LAN), networks based on Ethernet, Token-Ring, a wide-area network, Internet, a virtual network, a virtual private network (VPN), an intranet, an extranet, a public switched telephone network (PSTN), an infra-red network, a wireless network (e.g., a network operating under any of the Institute of Electrical and Electronics (IEEE) 802.11 suite of protocols, Bluetooth®, and/or any other wireless protocol), and/or any combination of these and/or other networks.

[0119] In at least one embodiment, server **612** may be composed of one or more general purpose computers, specialized server computers (including, by way of at least one embodiment, PC (personal computer) servers, UNIX® servers, mid-range servers, mainframe computers, rack-mounted servers, etc.), server farms, server clusters, or any other appropriate arrangement and/or combination. In at least one embodiment, server **612** can include one or more virtual machines running virtual operating systems, or other computing architectures involving virtualization. In at least one embodiment, one or more flexible pools of logical storage devices can be virtualized to maintain virtual storage devices for a server. In at least one embodiment, virtual networks can be controlled by server **612** using software defined networking. In at least one embodiment, server **612** may be adapted to run one or more services or software applications.

[0120] In at least one embodiment, server **612** may run any operating system, as well as any commercially available server operating system. In at least one embodiment, server **612** may also run any of a variety of additional server applications and/or mid-tier applications, including HTTP (hypertext transport protocol) servers, FTP (file transfer protocol) servers, CGI (common gateway interface) servers, JAVA® servers, database servers, and/or variations thereof. In at least one embodiment, exemplary database servers include without limitation those commercially available from Oracle, Microsoft, Sybase, IBM (International Business Machines), and/or variations thereof.

[0121] In at least one embodiment, server **612** may include one or more applications to analyze and consolidate data feeds and/or event updates received from users of client computing devices **602**, **604**, **606**, and **608**. In at least one embodiment, data feeds and/or event updates may include, but are not limited to, Twitter® feeds, Facebook® updates or real-time updates received from one or more third party information sources and continuous data streams, which may include real-time events related to sensor data applications, financial tickers, network performance measuring tools (e.g., network monitoring and traffic management applications), clickstream analysis tools, automobile traffic monitoring, and/or variations thereof. In at least one embodiment, server **612** may also include one or more applications to display data feeds and/or real-time events via one or more display devices of client computing devices **602**, **604**, **606**, and **608**.

[0122] In at least one embodiment, distributed system **600** may also include one or more databases **614** and **616**. In at least one embodiment, databases may provide a mechanism for storing information such as user interactions information, usage patterns information, adaptation rules information, and other information. In at least one embodiment, databases **614** and **616** may reside in a variety of locations. In at least one embodiment, one or more of databases **614** and **616** may reside on a non-transitory storage medium local to (and/or resident in) server **612**. In at least one embodiment, databases **614** and **616** may be remote from server **612** and in communication with server **612** via a network-based or dedicated connection. In at least one embodiment, databases **614** and **616** may reside in a storage-area network (SAN). In at least one embodiment, any necessary files for performing functions attributed to server **612** may be stored locally on server **612** and/or remotely, as appropriate. In at least one embodiment, databases **614** and **616** may include relational databases, such as databases that are adapted to store, update, and retrieve data in response to SQL-formatted commands.

[0123] FIG. **7** illustrates an example data center **700**, in which at least one embodiment may be used. In at least one embodiment, data center **700** includes a data center infrastructure layer **710**, a framework layer **720**, a software layer **730** and an application layer **740**.

[0124] In at least one embodiment, as shown in FIG. **7**, data center infrastructure layer **710** may include a resource orchestrator **712**, grouped computing resources **714**, and node computing resources (“node C.R.s”) **716(1)-716**(N), where “N” represents any whole, positive integer. In at least one embodiment, node C.R.s **716(1)-716**(N) may include, but are not limited to, any number of central processing units (“CPUs”) or other processors (including accelerators, field programmable gate arrays (FPGAs), graphics processors, etc.), memory storage devices **718**(**1**)-**718**(N) (e.g., dynamic read-only memory, solid state storage or disk drives), network input/output ("NW I/O") devices, network switches, virtual machines ("VMs"), power modules, and cooling modules, etc. In at least one embodiment, one or more node C.R.s from among node C.R.s **716**(**1**)-**716**(N) may be a server having one or more of above-mentioned computing resources.

[0125] In at least one embodiment, grouped computing resources **714** may include separate groupings of node C.R.s housed within one or more racks (not shown), or many racks housed in datacenters at various geographical locations (also not shown). In at least one embodiment, separate groupings of node C.R.s within grouped computing resources **714** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s including CPUs or processors may grouped within one or more racks to provide compute resources to support one or more workloads. In at least one embodiment, one or more racks may also include any number of power modules, cooling modules, and network switches, in any combination.

[0126] In at least one embodiment, resource orchestrator **712** may configure or otherwise control one or more node C.R.s **716**(**1**)-**716**(N) and/or grouped computing resources **714**. In at least one embodiment, resource orchestrator **712** may include a software design infrastructure ("SDI") management entity for datacenter **700**. In at least one embodiment, resource orchestrator **712** may include hardware, software or some combination thereof.

[0127] In at least one embodiment, as shown in FIG. **7**, framework layer **720** includes, a job scheduler **732**, a configuration manager **734**, a resource manager **736** and a distributed file system **738**. In at least one embodiment, framework layer **720** may include a framework to support software **752** of software layer **730** and/or one or more application(s) **742** of application layer **740**. In at least one embodiment, software **752** or application(s) **742** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. In at least one embodiment, framework layer **720** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter "Spark") that may utilize distributed file system **738** for large-scale data processing (e.g., "big data"). In at least one embodiment, job scheduler **732** may include a Spark driver to facilitate scheduling of workloads supported by various layers of datacenter **700**. In at least one embodiment, configuration manager **734** may be capable of configuring different layers such as software layer **730** and framework layer **720**, including Spark and distributed file system **738** for supporting large-scale data processing. In at least one embodiment, resource manager **736** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **738** and job scheduler **732**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **714** at datacenter infrastructure layer **710**. In at least one embodiment, resource manager **736** may coordinate with resource orchestrator **712** to manage these mapped or allocated computing resources.

[0128] In at least one embodiment, software **752** included in software layer **730** may include software used by at least portions of node C.R.s **716**(**1**)-**716**(N), grouped computing resources **714**, and/or distributed file system **738** of framework layer **720**. In at least one embodiment, one or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0129] In at least one embodiment, application(s) **742** included in application layer **740** may include one or more types of applications used by at least portions of node C.R.s **716**(**1**)-**716**(N), grouped computing resources **714**, and/or distributed file system **738** of framework layer **720**. In at least one embodiment, one or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, application and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.) or other machine learning applications used in conjunction with one or more embodiments.

[0130] In at least one embodiment, any of configuration manager **734**, resource manager **736**, and resource orchestrator **712** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. In at least one embodiment, self-modifying actions may relieve a data center operator of data center **700** from making possibly bad configuration decisions and possibly avoiding underutilized and/or poor performing portions of a data center.

[0131] In at least one embodiment, data center **700** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, in at least one embodiment, a machine learning model may be trained by calculating weight parameters according to a neural network architecture using software and computing resources described above with respect to data center **700**. In at least one embodiment, trained machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to data center **700** by using weight parameters calculated through one or more training techniques described herein.

[0132] In at least one embodiment, data center may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, or other hardware to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

[0133] Inference and/or training logic **1814** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in system FIG. **7** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0134] FIG. **8** illustrates a client-server network **804** formed by a plurality of network server computers **802** which are interlinked, in accordance with at least one embodiment. In at least one embodiment, each network server computer **802** stores data accessible to other network server computers **802** and to client computers **806** and networks **808** which link into a wide area network **804**. In at least one embodiment, configuration of a client-server network **804** may change over time as client computers **806** and one or more networks **808** connect and disconnect from a network **804**, and as one or more trunk line server computers **802** are added or removed from a network **804**. In at least one embodiment, when a client computer **806** and a network **808** are connected with network server computers **802**, client-server network includes such client computer **806** and network **808**. In at least one embodiment, the term computer includes any device or machine capable of accepting data, applying prescribed processes to data, and supplying results of processes.

[0135] In at least one embodiment, client-server network **804** stores information which is accessible to network server computers **802**, remote networks **808** and client computers **806**. In at least one embodiment, network server computers **802** are formed by main frame computers minicomputers, and/or microcomputers having one or more processors each. In at least one embodiment, server computers **802** are linked together by wired and/or wireless transfer media, such as conductive wire, fiber optic cable, and/or microwave transmission media, satellite transmission media or other conductive, optic or electromagnetic wave transmission media. In at least one embodiment, client computers **806** access a network server computer **802** by a similar wired or a wireless transfer medium. In at least one embodiment, a client computer **806** may link into a client-server network **804** using a modem and a standard telephone communication network. In at least one embodiment, alternative carrier systems such as cable and satellite communication systems also may be used to link into client-server network **804**. In at least one embodiment, other private or time-shared carrier systems may be used. In at least one embodiment, network **804** is a global information network, such as the Internet. In at least one embodiment, network is a private intranet using similar protocols as the Internet, but with added security measures and restricted access controls. In at least one embodiment, network **804** is a private, or semi-private network using proprietary communication protocols.

[0136] In at least one embodiment, client computer **806** is any end user computer, and may also be a mainframe computer, mini-computer or microcomputer having one or more microprocessors. In at least one embodiment, server computer **802** may at times function as a client computer accessing another server computer **802**. In at least one embodiment, remote network **808** may be a local area network, a network added into a wide area network through an independent service provider (ISP) for the Internet, or another group of computers interconnected by wired or wireless transfer media having a configuration which is either fixed or changing over time. In at least one embodiment, client computers **806** may link into and access a network **804** independently or through a remote network **808**.

[0137] FIG. **9** illustrates a computer network **908** connecting one or more computing machines, in accordance with at least one embodiment. In at least one embodiment, network **908** may be any type of electronically connected group of computers including, for instance, the following networks: Internet, Intranet, Local Area Networks (LAN), Wide Area Networks (WAN) or an interconnected combination of these network types. In at least one embodiment, connectivity within a network **908** may be a remote modem, Ethernet (IEEE 802.3), Token Ring (IEEE 802.5), Fiber Distributed Datalink Interface (FDDI), Asynchronous Transfer Mode (ATM), or any other communication protocol. In at least one embodiment, computing devices linked to a network may be desktop, server, portable, handheld, set-top box, personal digital assistant (PDA), a terminal, or any other desired type or configuration. In at least one embodiment, depending on their functionality, network connected devices may vary widely in processing power, internal memory, and other performance aspects.

[0138] In at least one embodiment, communications within a network and to or from computing devices connected to a network may be either wired or wireless. In at least one embodiment, network **908** may include, at least in part, the world-wide public Internet which generally connects a plurality of users in accordance with a client-server model in accordance with a transmission control protocol/internet protocol (TCP/IP) specification. In at least one embodiment, client-server network is a dominant model for communicating between two computers. In at least one embodiment, a client computer (“client”) issues one or more commands to a server computer (“server”). In at least one embodiment, server fulfills client commands by accessing available network resources and returning information to a client pursuant to client commands. In at least one embodiment, client computer systems and network resources resident on network servers are assigned a network address for identification during communications between elements of a network. In at least one embodiment, communications from other network connected systems to servers will include a network address of a relevant server/network resource as part of communication so that an appropriate destination of a data/request is identified as a recipient. In at least one embodiment, when a network **908** comprises the global Internet, a network address is an IP address in a TCP/IP format which may, at least in part, route data to an e-mail account, a website, or other Internet tool resident on a server. In at least one embodiment, information and services which are resident on network servers may be available to a web browser of a client computer through a domain name (e.g. www.site.com) which maps to an IP address of a network server.

[0139] In at least one embodiment, a plurality of clients **902**, **904**, and **906** are connected to a network **908** via respective communication links. In at least one embodiment, each of these clients may access a network **908** via any desired form of communication, such as via a dial-up modem connection, cable link, a digital subscriber line (DSL), wireless or satellite link, or any other form of communication. In at least one embodiment, each client may communicate using any machine that is compatible with a network **908**, such as a personal computer (PC), work station, dedicated terminal, personal data assistant (PDA), or other similar equipment. In at least one embodiment, clients **902**,

904, and **906** may or may not be located in a same geographical area.

[0140] In at least one embodiment, a plurality of servers **910**, **912**, and **914** are connected to a network **918** to serve clients that are in communication with a network **918**. In at least one embodiment, each server is typically a powerful computer or device that manages network resources and responds to client commands. In at least one embodiment, servers include computer readable data storage media such as hard disk drives and RAM memory that store program instructions and data. In at least one embodiment, servers **910**, **912**, **914** run application programs that respond to client commands. In at least one embodiment, server **910** may run a web server application for responding to client requests for HTML pages and may also run a mail server application for receiving and routing electronic mail. In at least one embodiment, other application programs, such as an FTP server or a media server for streaming audio/video data to clients may also be running on a server **910**. In at least one embodiment, different servers may be dedicated to performing different tasks. In at least one embodiment, server **910** may be a dedicated web server that manages resources relating to web sites for various users, whereas a server **912** may be dedicated to provide electronic mail (email) management. In at least one embodiment, other servers may be dedicated for media (audio, video, etc.), file transfer protocol (FTP), or a combination of any two or more services that are typically available or provided over a network. In at least one embodiment, each server may be in a location that is the same as or different from that of other servers. In at least one embodiment, there may be multiple servers that perform mirrored tasks for users, thereby relieving congestion or minimizing traffic directed to and from a single server. In at least one embodiment, servers **910**, **912**, **914** are under control of a web hosting provider in a business of maintaining and delivering third party content over a network **918**.

[0141] In at least one embodiment, web hosting providers deliver services to two different types of clients. In at least one embodiment, one type, which may be referred to as a browser, requests content from servers **910**, **912**, **914** such as web pages, email messages, video clips, etc. In at least one embodiment, a second type, which may be referred to as a user, hires a web hosting provider to maintain a network resource such as a web site, and to make it available to browsers. In at least one embodiment, users contract with a web hosting provider to make memory space, processor capacity, and communication bandwidth available for their desired network resource in accordance with an amount of server resources a user desires to utilize.

[0142] In at least one embodiment, in order for a web hosting provider to provide services for both of these clients, application programs which manage a network resources hosted by servers must be properly configured. In at least one embodiment, program configuration process involves defining a set of parameters which control, at least in part, an application program's response to browser requests and which also define, at least in part, a server resources available to a particular user.

[0143] In one embodiment, an intranet server **916** is in communication with a network **908** via a communication link. In at least one embodiment, intranet server **916** is in communication with a server manager **918**. In at least one embodiment, server manager **918** comprises a database of an application program configuration parameters which are being utilized in servers **910**, **912**, **914**. In at least one embodiment, users modify a database **920** via an intranet **916**, and a server manager **918** interacts with servers **910**, **912**, **914** to modify application program parameters so that they match a content of a database. In at least one embodiment, a user logs onto an intranet server **916** by connecting to an intranet **916** via computer **902** and entering authentication information, such as a username and password.

[0144] In at least one embodiment, when a user wishes to sign up for new service or modify an existing service, an intranet server **916** authenticates a user and provides a user with an interactive screen display/control panel that allows a user to access configuration parameters for a particular application program. In at least one embodiment, a user is presented with a number of modifiable text boxes that describe aspects of a configuration of a user's web site or other network resource. In at least one embodiment, if a user desires to increase memory space reserved on a server for its web site, a user is provided with a field in which a user specifies a desired memory space. In at least one embodiment, in response to receiving this information, an intranet server **916** updates a database **920**. In at least one embodiment, server manager **918** forwards this information to an appropriate server, and a new parameter is used during application program operation. In at least one embodiment, an intranet server **916** is configured to provide users with access to configuration parameters of hosted network resources (e.g., web pages, email, FTP sites, media sites, etc.), for which a user has contracted with a web hosting service provider.

[0145] FIG. **10**A illustrates a networked computer system **1000**A, in accordance with at least one embodiment. In at least one embodiment, networked computer system **1000**A comprises a plurality of nodes or personal computers ("PCs") **1002**, **1018**, **1020**. In at least one embodiment, personal computer or node **1002** comprises a processor **1014**, memory **1016**, video camera **1004**, microphone **1006**, mouse **1008**, speakers **1010**, and monitor **1012**. In at least one embodiment, PCs **1002**, **1018**, **1020** may each run one or more desktop servers of an internal network within a given company, for instance, or may be servers of a general network not limited to a specific environment. In at least one embodiment, there is one server per PC node of a network, so that each PC node of a network represents a particular network server, having a particular network URL address. In at least one embodiment, each server defaults to a default web page for that server's user, which may itself contain embedded URLs pointing to further subpages of that user on that server, or to other servers or pages on other servers on a network.

[0146] In at least one embodiment, nodes **1002**, **1018**, **1020** and other nodes of a network are interconnected via medium **1022**. In at least one embodiment, medium **1022** may be, a communication channel such as an Integrated Services Digital Network ("ISDN"). In at least one embodiment, various nodes of a networked computer system may be connected through a variety of communication media, including local area networks ("LANs"), plain-old telephone lines ("POTS"), sometimes referred to as public switched telephone networks ("PSTN"), and/or variations thereof. In at least one embodiment, various nodes of a network may also constitute computer system users inter-connected via a network such as the Internet. In at least one embodiment, each server on a network (running from a particular node of a network at a given instance) has a unique address or identification within a network, which may be specifiable in terms of an URL.

[0147] In at least one embodiment, a plurality of multi-point conferencing units ("MCUs") may thus be utilized to transmit data to and from various nodes or "endpoints" of a conferencing system. In at least one embodiment, nodes and/or MCUs may be interconnected via an ISDN link or through a local area network ("LAN"), in addition to various other communications media such as nodes connected through the Internet. In at least one embodiment, nodes of a conferencing system may, in general, be connected directly to a communications medium such as a LAN or through an MCU, and that a conferencing system may comprise other nodes or elements such as routers, servers, and/or variations thereof.

[0148] In at least one embodiment, processor **1014** is a general-purpose programmable processor. In at least one embodiment, processors of nodes of networked computer system **1000**A may also be special-purpose video processors. In at least one embodiment, various peripherals and components of a node such as those of node **1002** may vary from those of other nodes. In at least one embodiment, node **1018** and node **1020** may be configured identically to or differently than node **1002**. In at least one embodiment, a node may be implemented on any suitable computer system in addition to PC systems.

[0149] FIG. **10**B illustrates a networked computer system **1000**B, in accordance with at least one embodiment. In at least one embodiment, system **1000**B illustrates a network such as LAN **1024**, which may be used to interconnect a variety of nodes that may communicate with each other. In at least one embodiment, attached to LAN **1024** are a plurality of nodes such as PC nodes **1026**, **1028**, **1030**. In at least one embodiment, a node may also be connected to the LAN via a network server or other means. In at least one embodiment, system **1000**B comprises other types of nodes or elements, for at least one embodiment including routers, servers, and nodes.

[0150] FIG. **10**C illustrates a networked computer system **1000**C, in accordance with at least one embodiment. In at least one embodiment, system **1000**C illustrates a WWW system having communications across a backbone communications network such as Internet **1032**, which may be used to interconnect a variety of nodes of a network. In at least one embodiment, WWW is a set of protocols operating on top of the Internet, and allows a graphical interface system to operate thereon for accessing information through the Internet. In at least one embodiment, attached to Internet **1032** in WWW are a plurality of nodes such as PCs **1040**, **1042**, **1044**. In at least one embodiment, a node is interfaced to other nodes of WWW through a WWW HTTP server such as servers **1034**, **1036**. In at least one embodiment, PC **1044** may be a PC forming a node of network **1032** and itself running its server **1036**, although PC **1044** and server **1036** are illustrated separately in FIG. **10**C for illustrative purposes.

[0151] In at least one embodiment, WWW is a distributed type of application, characterized by WWW HTTP, WWW's protocol, which runs on top of the Internet's transmission control protocol/Internet protocol ("TCP/IP"). In at least one embodiment, WWW may thus be characterized by a set of protocols (i.e., HTTP) running on the Internet as its "backbone."

[0152] In at least one embodiment, a web browser is an application running on a node of a network that, in WWW-compatible type network systems, allows users of a particular server or node to view such information and thus allows a user to search graphical and text-based files that are linked together using hypertext links that are embedded in documents or files available from servers on a network that understand HTTP. In at least one embodiment, when a given web page of a first server associated with a first node is retrieved by a user using another server on a network such as the Internet, a document retrieved may have various hypertext links embedded therein and a local copy of a page is created local to a retrieving user. In at least one embodiment, when a user clicks on a hypertext link, locally-stored information related to a selected hypertext link is typically sufficient to allow a user's machine to open a connection across the Internet to a server indicated by a hypertext link.

[0153] In at least one embodiment, more than one user may be coupled to each HTTP server, through a LAN such as LAN **1038** as illustrated with respect to WWW HTTP server **1034**. In at least one embodiment, system **1000**C may also comprise other types of nodes or elements. In at least one embodiment, a WWW HTTP server is an application running on a machine, such as a PC. In at least one embodiment, each user may be considered to have a unique "server," as illustrated with respect to PC **1044**. In at least one embodiment, a server may be considered to be a server such as WWW HTTP server **1034**, which provides access to a network for a LAN or plurality of nodes or plurality of LANs. In at least one embodiment, there are a plurality of users, each having a desktop PC or node of a network, each desktop PC potentially establishing a server for a user thereof. In at least one embodiment, each server is associated with a particular network address or URL, which, when accessed, provides a default web page for that user. In at least one embodiment, a web page may contain further links (embedded URLs) pointing to further subpages of that user on that server, or to other servers on a network or to pages on other servers on a network.

Cloud Computing and Services

[0154] The following figures set forth, without limitation, exemplary cloud-based systems that can be used to implement at least one embodiment.

[0155] In at least one embodiment, cloud computing is a style of computing in which dynamically scalable and often virtualized resources are provided as a service over the Internet. In at least one embodiment, users need not have knowledge of, expertise in, or control over technology infrastructure, which can be referred to as "in the cloud," that supports them. In at least one embodiment, cloud computing incorporates infrastructure as a service, platform as a service, software as a service, and other variations that have a common theme of reliance on the Internet for satisfying computing needs of users. In at least one embodiment, a typical cloud deployment, such as in a private cloud (e.g., enterprise network), or a datacenter (DC) in a public cloud (e.g., Internet) can consist of thousands of servers (or alternatively, VMs), hundreds of Ethernet, Fiber Channel or Fiber Channel over Ethernet (FCoE) ports, switching and storage infrastructure, etc. In at least one embodiment, cloud can also consist of network services infrastructure like IPsec VPN hubs, firewalls, load balancers, wide area network (WAN) optimizers etc. In at least one embodiment, remote subscribers can access cloud applications and services securely by connecting via a VPN tunnel, such as an IPsec VPN tunnel.

[0156] In at least one embodiment, cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

[0157] In at least one embodiment, cloud computing is characterized by on-demand self-service, in which a consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human inter-action with each service's provider. In at least one embodiment, cloud computing is characterized by broad network access, in which capabilities are available over a network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and PDAs). In at least one embodiment, cloud computing is characterized by resource pooling, in which a provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically as-signed and reassigned according to consumer demand. In at least one embodiment, there is a sense of location independence in that a customer generally has no control or knowledge over an exact location of provided resources, but may be able to specify location at a higher level of abstraction (e.g., country, state, or datacenter).

[0158] In at least one embodiment, resources include storage, processing, memory, network bandwidth, and virtual machines. In at least one embodiment, cloud computing is characterized by rapid elasticity, in which capabilities can be rapidly and elastically provisioned, in some cases automatically, to quickly scale out and rapidly released to quickly scale in. In at least one embodiment, to a consumer, capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time. In at least one embodiment, cloud computing is characterized by measured service, in which cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to a type of service (e.g., storage, processing, bandwidth, and active user accounts). In at least one embodiment, resource usage can be monitored, controlled, and reported providing transparency for both a provider and consumer of a utilized service.

[0159] In at least one embodiment, cloud computing may be associated with various services. In at least one embodiment, cloud Software as a Service (SaaS) may refer to as service in which a capability provided to a consumer is to use a provider's applications running on a cloud infrastructure. In at least one embodiment, applications are accessible from various client devices through a thin client interface such as a web browser (e.g., web-based email). In at least one embodiment, consumer does not manage or control underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with a possible exception of limited user-specific application configuration settings.

[0160] In at least one embodiment, cloud Platform as a Service (PaaS) may refer to a service in which a capability provided to a consumer is to deploy onto cloud infrastructure consumer-created or acquired applications created using programming languages and tools supported by a provider. In at least one embodiment, consumer does not manage or control underlying cloud infrastructure including networks, servers, operating systems, or storage, but has control over deployed applications and possibly application hosting environment configurations.

[0161] In at least one embodiment, cloud Infrastructure as a Service (IaaS) may refer to a service in which a capability provided to a consumer is to provision processing, storage, networks, and other fundamental computing resources where a consumer is able to deploy and run arbitrary software, which can include operating systems and applications. In at least one embodiment, consumer does not manage or control underlying cloud infrastructure, but has control over operating systems, storage, deployed applications, and possibly limited control of select networking components (e.g., host firewalls).

[0162] In at least one embodiment, cloud computing may be deployed in various ways. In at least one embodiment, a private cloud may refer to a cloud infrastructure that is operated solely for an organization. In at least one embodiment, a private cloud may be managed by an organization or a third party and may exist on-premises or off-premises. In at least one embodiment, a community cloud may refer to a cloud infrastructure that is shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). In at least one embodiment, a community cloud may be managed by organizations or a third party and may exist on-premises or off-premises. In at least one embodiment, a public cloud may refer to a cloud infrastructure that is made available to a general public or a large industry group and is owned by an organization providing cloud services. In at least one embodiment, a hybrid cloud may refer to a cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load-balancing between clouds). In at least one embodiment, a cloud computing environment is service oriented with a focus on statelessness, low coupling, modularity, and semantic interoperability.

[0163] FIG. **11** illustrates one or more components of a system environment **1100** in which services may be offered as third party network services, in accordance with at least one embodiment. In at least one embodiment, a third party network may be referred to as a cloud, cloud network, cloud computing network, and/or variations thereof. In at least one embodiment, system environment **1100** includes one or more client computing devices **1104**, **1106**, and **1108** that may be used by users to interact with a third party network infrastructure system **1102** that provides third party network services, which may be referred to as cloud computing services. In at least one embodiment, third party network infrastructure system **1102** may comprise one or more computers and/or servers.

[0164] It should be appreciated that third party network infrastructure system **1102** depicted in FIG. **11** may have other components than those depicted. Further, FIG. **11** depicts an embodiment of a third party network infrastructure system. In at least one embodiment, third party network infrastructure system **1102** may have more or fewer components than depicted in FIG. **11**, may combine two or more components, or may have a different configuration or arrangement of components.

[0165] In at least one embodiment, client computing devices **1104**, **1106**, and **1108** may be configured to operate a client application such as a web browser, a proprietary client application, or some other application, which may be used by a user of a client computing device to interact with third party network infrastructure system **1102** to use services provided by third party network infrastructure system **1102**. Although exemplary system environment **1100** is shown with three client computing devices, any number of client computing devices may be supported. In at least one embodiment, other devices such as devices with sensors, etc. may interact with third party network infrastructure system **1102**. In at least one embodiment, network(s) **1110** may facilitate communications and exchange of data between client computing devices **1104**, **1106**, and **1108** and third party network infrastructure system **1102**.

[0166] In at least one embodiment, services provided by third party network infrastructure system **1102** may include a host of services that are made available to users of a third party network infrastructure system on demand. In at least one embodiment, various services may also be offered including without limitation online data storage and backup solutions, Web-based e-mail services, hosted office suites and document collaboration services, database management and processing, managed technical support services, and/or variations thereof. In at least one embodiment, services provided by a third party network infrastructure system can dynamically scale to meet needs of its users.

[0167] In at least one embodiment, a specific instantiation of a service provided by third party network infrastructure system **1102** may be referred to as a “service instance.” In at least one embodiment, in general, any service made available to a user via a communication network, such as the Internet, from a third party network service provider's system is referred to as a “third party network service.” In at least one embodiment, in a public third party network environment, servers and systems that make up a third party network service provider's system are different from a customer's own on-premises servers and systems. In at least one embodiment, a third party network service provider's system may host an application, and a user may, via a communication network such as the Internet, on demand, order and use an application.

[0168] In at least one embodiment, a service in a computer network third party network infrastructure may include protected computer network access to storage, a hosted database, a hosted web server, a software application, or other service provided by a third party network vendor to a user. In at least one embodiment, a service can include password-protected access to remote storage on a third party network through the Internet. In at least one embodiment, a service can include a web service-based hosted relational database and a script-language middleware engine for private use by a networked developer. In at least one embodiment, a service can include access to an email software application hosted on a third party network vendor's web site.

[0169] In at least one embodiment, third party network infrastructure system **1102** may include a suite of applications, middleware, and database service offerings that are delivered to a customer in a self-service, subscription-based, elastically scalable, reliable, highly available, and secure manner. In at least one embodiment, third party network infrastructure system **1102** may also provide “big data” related computation and analysis services. In at least one embodiment, term “big data” is generally used to refer to extremely large data sets that can be stored and manipulated by analysts and researchers to visualize large amounts of data, detect trends, and/or otherwise interact with data. In at least one embodiment, big data and related applications can be hosted and/or manipulated by an infrastructure system on many levels and at different scales. In at least one embodiment, tens, hundreds, or thousands of processors linked in parallel can act upon such data in order to present it or simulate external forces on data or what it represents. In at least one embodiment, these data sets can involve structured data, such as that organized in a database or otherwise according to a structured model, and/or unstructured data (e.g., emails, images, data blobs (binary large objects), web pages, complex event processing). In at least one embodiment, by leveraging an ability of an embodiment to relatively quickly focus more (or fewer) computing resources upon an objective, a third party network infrastructure system may be better available to carry out tasks on large data sets based on demand from a business, government agency, research organization, private individual, group of like-minded individuals or organizations, or other entity.

[0170] In at least one embodiment, third party network infrastructure system **1102** may be adapted to automatically provision, manage and track a customer's subscription to services offered by third party network infrastructure system **1102**. In at least one embodiment, third party network infrastructure system **1102** may provide third party network services via different deployment models. In at least one embodiment, services may be provided under a public third party network model in which third party network infrastructure system **1102** is owned by an organization selling third party network services and services are made available to a general public or different industry enterprises. In at least one embodiment, services may be provided under a private third party network model in which third party network infrastructure system **1102** is operated solely for a single organization and may provide services for one or more entities within an organization. In at least one embodiment, third party network services may also be provided under a community third party network model in which third party network infrastructure system **1102** and services provided by third party network infrastructure system **1102** are shared by several organizations in a related community. In at least one embodiment, third party network services may also be provided under a hybrid third party network model, which is a combination of two or more different models.

[0171] In at least one embodiment, services provided by third party network infrastructure system **1102** may include one or more services provided under Software as a Service (SaaS) category, Platform as a Service (PaaS) category, Infrastructure as a Service (IaaS) category, or other categories of services including hybrid services. In at least one embodiment, a customer, via a subscription order, may order one or more services provided by third party network infrastructure system **1102**. In at least one embodiment, third party network infrastructure system **1102** then performs processing to provide services in a customer's subscription order.

[0172] In at least one embodiment, services provided by third party network infrastructure system **1102** may include, without limitation, application services, platform services and infrastructure services. In at least one embodiment, application services may be provided by a third party network infrastructure system via a SaaS platform. In at least one embodiment, SaaS platform may be configured to provide third party network services that fall under a SaaS category. In at least one embodiment, SaaS platform may provide capabilities to build and deliver a suite of on-demand applications on an integrated development and deployment platform. In at least one embodiment, SaaS platform may manage and control underlying software and infrastructure for providing SaaS services. In at least one embodiment, by utilizing services provided by a SaaS platform, customers can utilize applications executing on a third party network infrastructure system. In at least one embodiment, customers can acquire an application services without a need for customers to purchase separate licenses and support. In at least one embodiment, various different SaaS services may be provided. In at least one embodiment, this may include, without limitation, services that provide solutions for sales performance management, enterprise integration, and business flexibility for large organizations.

[0173] In at least one embodiment, platform services may be provided by third party network infrastructure system **1102** via a PaaS platform. In at least one embodiment, PaaS platform may be configured to provide third party network services that fall under a PaaS category. In at least one embodiment, platform services may include without limitation services that enable organizations to consolidate existing applications on a shared, common architecture, as well as an ability to build new applications that leverage shared services provided by a platform. In at least one embodiment, PaaS platform may manage and control underlying software and infrastructure for providing PaaS services. In at least one embodiment, customers can acquire PaaS services provided by third party network infrastructure system **1102** without a need for customers to purchase separate licenses and support.

[0174] In at least one embodiment, by utilizing services provided by a PaaS platform, customers can employ programming languages and tools supported by a third party network infrastructure system and also control deployed services. In at least one embodiment, platform services provided by a third party network infrastructure system may include database third party network services, middleware third party network services and third party network services. In at least one embodiment, database third party network services may support shared service deployment models that enable organizations to pool database resources and offer customers a Database as a Service in a form of a database third party network. In at least one embodiment, middleware third party network services may provide a platform for customers to develop and deploy various business applications, and third party network services may provide a platform for customers to deploy applications, in a third party network infrastructure system.

[0175] In at least one embodiment, various different infrastructure services may be provided by an IaaS platform in a third party network infrastructure system. In at least one embodiment, infrastructure services facilitate management and control of underlying computing resources, such as storage, networks, and other fundamental computing resources for customers utilizing services provided by a SaaS platform and a PaaS platform.

[0176] In at least one embodiment, third party network infrastructure system **1102** may also include infrastructure resources **1130** for providing resources used to provide various services to customers of a third party network infrastructure system. In at least one embodiment, infrastructure resources **1130** may include pre-integrated and optimized combinations of hardware, such as servers, storage, and networking resources to execute services provided by a Paas platform and a Saas platform, and other resources.

[0177] In at least one embodiment, resources in third party network infrastructure system **1102** may be shared by multiple users and dynamically re-allocated per demand. In at least one embodiment, resources may be allocated to users in different time zones. In at least one embodiment, third party network infrastructure system **1102** may enable a first set of users in a first time zone to utilize resources of a third party network infrastructure system for a specified number of hours and then enable a re-allocation of same resources to another set of users located in a different time zone, thereby maximizing utilization of resources.
[0178] In at least one embodiment, a number of internal shared services **1132** may be provided that are shared by different components or modules of third party network infrastructure system **1102** to enable provision of services by third party network infrastructure system **1102**. In at least one embodiment, these internal shared services may include, without limitation, a security and identity service, an integration service, an enterprise repository service, an enterprise manager service, a virus scanning and white list service, a high availability, backup and recovery service, service for enabling third party network support, an email service, a notification service, a file transfer service, and/or variations thereof.
[0179] In at least one embodiment, third party network infrastructure system **1102** may provide comprehensive management of third party network services (e.g., SaaS, PaaS, and IaaS services) in a third party network infrastructure system. In at least one embodiment, third party network management functionality may include capabilities for provisioning, managing and tracking a customer's subscription received by third party network infrastructure system **1102**, and/or variations thereof.
[0180] In at least one embodiment, as depicted in FIG. **11**, third party network management functionality may be provided by one or more modules, such as an order management module **1120**, an order orchestration module **1122**, an order provisioning module **1124**, an order management and monitoring module **1126**, and an identity management module **1128**. In at least one embodiment, these modules may include or be provided using one or more computers and/or servers, which may be general purpose computers, specialized server computers, server farms, server clusters, or any other appropriate arrangement and/or combination.
[0181] In at least one embodiment, at step **1134**, a customer using a client device, such as client computing devices **1104**, **1106** or **1108**, may interact with third party network infrastructure system **1102** by requesting one or more services provided by third party network infrastructure system **1102** and placing an order for a subscription for one or more services offered by third party network infrastructure system **1102**. In at least one embodiment, a customer may access a third party network User Interface (UI) such as third party network UI **1112**, third party network UI **1114** and/or third party network UI **1116** and place a subscription order via these UIs. In at least one embodiment, order information received by third party network infrastructure system **1102** in response to a customer placing an order may include information identifying a customer and one or more services offered by a third party network infrastructure system **1102** that a customer intends to subscribe to.
[0182] In at least one embodiment, at step **1136**, an order information received from a customer may be stored in an order database **1118**. In at least one embodiment, if this is a new order, a new record may be created for an order. In at least one embodiment, order database **1118** can be one of several databases operated by third party network infrastructure system **1118** and operated in conjunction with other system elements.
[0183] In at least one embodiment, at step **1138**, an order information may be forwarded to an order management module **1120** that may be configured to perform billing and accounting functions related to an order, such as verifying an order, and upon verification, booking an order.
[0184] In at least one embodiment, at step **1140**, information regarding an order may be communicated to an order orchestration module **1122** that is configured to orchestrate provisioning of services and resources for an order placed by a customer. In at least one embodiment, order orchestration module **1122** may use services of order provisioning module **1124** for provisioning. In at least one embodiment, order orchestration module **1122** enables management of business

processes associated with each order and applies business logic to determine whether an order should proceed to provisioning.

[0185] In at least one embodiment, at step **1142**, upon receiving an order for a new subscription, order orchestration module **1122** sends a request to order provisioning module **1124** to allocate resources and configure resources needed to fulfill a subscription order. In at least one embodiment, order provisioning module **1124** enables an allocation of resources for services ordered by a customer. In at least one embodiment, order provisioning module **1124** provides a level of abstraction between third party network services provided by third party network infrastructure system **1100** and a physical implementation layer that is used to provision resources for providing requested services. In at least one embodiment, this enables order orchestration module **1122** to be isolated from implementation details, such as whether or not services and resources are actually provisioned in real-time or pre-provisioned and only allocated/assigned upon request.

[0186] In at least one embodiment, at step **1144**, once services and resources are provisioned, a notification may be sent to subscribing customers indicating that a requested service is now ready for use. In at least one embodiment, information (e.g. a link) may be sent to a customer that enables a customer to start using requested services.

[0187] In at least one embodiment, at step **1146**, a customer's subscription order may be managed and tracked by an order management and monitoring module **1126**. In at least one embodiment, order management and monitoring module **1126** may be configured to collect usage statistics regarding a customer use of subscribed services. In at least one embodiment, statistics may be collected for an amount of storage used, an amount data transferred, a number of users, and an amount of system up time and system down time, and/or variations thereof.

[0188] In at least one embodiment, third party network infrastructure system **1100** may include an identity management module **1128** that is configured to provide identity services, such as access management and authorization services in third party network infrastructure system **1100**. In at least one embodiment, identity management module **1128** may control information about customers who wish to utilize services provided by third party network infrastructure system **1102**. In at least one embodiment, such information can include information that authenticates identities of such customers and information that describes which actions those customers are authorized to perform relative to various system resources (e.g., files, directories, applications, communication ports, memory segments, etc.). In at least one embodiment, identity management module **1128** may also include management of descriptive information about each customer and about how and by whom that descriptive information can be accessed and modified.

[0189] FIG. **12** illustrates a cloud computing environment **1202**, in accordance with at least one embodiment. In at least one embodiment, cloud computing environment **1202** comprises one or more computer system/servers **1204** with which computing devices such as, personal digital assistant (PDA) or cellular telephone **1206**A, desktop computer **1206**B, laptop computer **1206**C, and/or automobile computer system **1206**N communicate. In at least one embodiment, this allows for infrastructure, platforms and/or software to be offered as services from cloud computing environment **1202**, so as to not require each client to separately maintain such resources. It is understood that types of computing devices **1206**A-N shown in FIG. **12** are intended to be illustrative only and that cloud computing environment **1202** can communicate with any type of computerized device over any type of network and/or network/addressable connection (e.g., using a web browser).

[0190] In at least one embodiment, a computer system/server **1204**, which can be denoted as a cloud computing node, is operational with numerous other general purpose or special purpose computing system environments or configurations. In at least one embodiment, computing systems, environments, and/or configurations that may be suitable for use with computer system/server **1204** include, but are not limited to, personal computer systems, server computer systems, thin clients, thick clients, hand-held or laptop devices, multiprocessor systems, microprocessor-based systems, set top boxes, programmable consumer electronics, network PCs, minicomputer systems, mainframe computer systems, and distributed cloud computing environments that include any of the above systems or devices, and/or variations thereof.

[0191] In at least one embodiment, computer system/server **1204** may be described in a general context of computer system-executable instructions, such as program modules, being executed by a computer system. In at least one embodiment, program modules include routines, programs, objects, components, logic, data structures, and so on, that perform particular tasks or implement particular abstract data types. In at least one embodiment, exemplary computer system/server **1204** may be practiced in distributed loud computing environments where tasks are performed by remote processing devices that are linked through a communications network. In at least one embodiment, in a distributed cloud computing environment, program modules may be located in both local and remote computer system storage media including memory storage devices.

[0192] FIG. **13** illustrates a set of functional abstraction layers provided by cloud computing environment **1202** (FIG. **12**), in accordance with at least one embodiment. It should be understood in advance that components, layers, and functions shown in FIG. **13** are intended to be illustrative only, and components, layers, and functions may vary.

[0193] In at least one embodiment, hardware and software layer **1302** includes hardware and software components. In at least one embodiment, hardware components include mainframes, various RISC (Reduced Instruction Set Computer) architecture based servers, various computing systems, supercomputing systems, storage devices, networks, networking components, and/or variations thereof. In at least one embodiment, software components include network application server software, various application server software, various database software, and/or variations thereof.

[0194] In at least one embodiment, virtualization layer **1302** provides an abstraction layer from which following exemplary virtual entities may be provided: virtual servers, virtual storage, virtual networks, including virtual private networks, virtual applications, virtual clients, and/or variations thereof.

[0195] In at least one embodiment, management layer **1306** provides various functions. In at least one embodiment, resource provisioning provides dynamic procurement of computing resources and other resources that are utilized to perform tasks within a cloud computing environment. In at least one embodiment, metering provides usage tracking as resources are utilized within a cloud computing environment, and billing or invoicing for consumption of these resources. In at least one embodiment, resources may comprise application software licenses. In at least one embodiment, security provides identity verification for users and tasks, as well as protection for data and other resources. In at least one embodiment, user interface provides access to a cloud computing environment for both users and system administrators. In at least one embodiment, service level management provides cloud computing resource allocation and management such that required service levels are met. In at least one embodiment, Service Level Agreement (SLA) management provides pre-arrangement for, and procurement of, cloud computing resources for which a future requirement is anticipated in accordance with an SLA.

[0196] In at least one embodiment, workloads layer **1308** provides functionality for which a cloud computing environment is utilized. In at least one embodiment, workloads and functions which may be provided from this layer include: mapping and navigation, software development and management, educational services, data analytics and processing, transaction processing, and service delivery.

Supercomputing

[0197] The following figures set forth, without limitation, exemplary supercomputer-based systems that can be used to implement at least one embodiment.

[0198] In at least one embodiment, a supercomputer may refer to a hardware system exhibiting substantial parallelism and comprising at least one chip, where chips in a system are interconnected by a network and are placed in hierarchically organized enclosures. In at least one embodiment, a large hardware system filling a machine room, with several racks, each containing several boards/rack modules, each containing several chips, all interconnected by a scalable network, is at least one embodiment of a supercomputer. In at least one embodiment, a single rack of such a large hardware system is at least one other embodiment of a supercomputer. In at least one embodiment, a single chip exhibiting substantial parallelism and containing several hardware components can equally be considered to be a supercomputer, since as feature sizes may decrease, an amount of hardware that can be incorporated in a single chip may also increase.

[0199] FIG. **14** illustrates a supercomputer at a chip level, in accordance with at least one embodiment. In at least one embodiment, inside an FPGA or ASIC chip, main computation is performed within finite state machines (**1404**) called thread units. In at least one embodiment, task and synchronization networks (**1402**) connect finite state machines and are used to dispatch threads and execute operations in correct order. In at least one embodiment, a multi-level partitioned on-chip cache hierarchy (**1408**, **1412**) is accessed using memory networks (**1406**, **1410**). In at least one embodiment, off-chip memory is accessed using memory controllers (**1416**) and an off-chip memory network (**1414**). In at least one embodiment, I/O controller (**1418**) is used for cross-chip communication when a design does not fit in a single logic chip.

[0200] FIG. **15** illustrates a supercomputer at a rock module level, in accordance with at least one embodiment. In at least one embodiment, within a rack module, there are multiple FPGA or ASIC chips (**1502**) that are connected to one or more DRAM units (**1504**) which constitute main accelerator memory. In at least one embodiment, each FPGA/ASIC chip is connected to its neighbor FPGA/ASIC chip using wide busses on a board, with differential high speed signaling (**1506**). In at least one embodiment, each FPGA/ASIC chip is also connected to at least one high-speed serial communication cable.

[0201] FIG. **16** illustrates a supercomputer at a rack level, in accordance with at least one embodiment. FIG. **17** illustrates a supercomputer at a whole system level, in accordance with at least one embodiment. In at least one embodiment, referring to FIG. **16** and FIG. **17**, between rack modules in a rack and across racks throughout an entire system, high-speed serial optical or copper cables (**1602**, **1702**) are used to realize a scalable, possibly incomplete hypercube network. In at least one embodiment, one of FPGA/ASIC chips of an accelerator is connected to a host system through a PCI-Express connection (**1704**). In at least one embodiment, host system comprises a host microprocessor (**1708**) that a software part of an application runs on and a memory consisting of one or more host memory DRAM units (**1706**) that is kept coherent with memory on an accelerator. In at least one embodiment, host system can be a separate module on one of racks, or can be integrated with one of a supercomputer's modules. In at least one embodiment, cube-connected cycles topology provide communication links to create a hypercube network for a large supercomputer. In at least one embodiment, a small group of FPGA/ASIC chips on a rack module can act as a single hypercube node, such that a total number of external links of each group is increased, compared to a single chip. In at least one embodiment, a group contains chips A, B, C and D on a rack module with internal wide differential busses connecting A, B, C and D in a torus organization. In at least one embodiment, there are 12 serial communication cables connecting a rack module to an outside world. In at least one embodiment, chip A on a rack module connects to serial communication cables 0, 1, 2. In at least one embodiment, chip B connects to cables 3, 4, 5. In at least one embodiment, chip C connects to 6, 7, 8. In at least one embodiment, chip D connects to 9, 10, 11. In at least one embodiment, an entire group {A, B, C, D} constituting a rack module can form a hypercube node within a supercomputer system, with up to 212=4096 rack modules (16384 FPGA/ASIC chips). In at least one embodiment, for chip A to send a message out on link 4 of group {A, B, C, D}, a message has to be routed first to chip B with an on-board differential wide bus connection. In at least one embodiment, a message arriving into a group {A, B, C, D} on link 4 (i.e., arriving at B) destined to chip A, also has to be routed first to a correct destination chip (A) internally within a group {A, B, C, D}. In at least one embodiment, parallel supercomputer systems of other sizes may also be implemented.

Artificial Intelligence

[0202] The following figures set forth, without limitation, exemplary artificial intelligence-based systems that can be used to implement at least one embodiment.

[0203] FIG. **18**A illustrates inference and/or training logic **1815** used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided below in conjunction with FIGS. **18**A and/or **18**B.

[0204] In at least one embodiment, inference and/or training logic **1815** may include, without limitation, code and/or data storage **1801** to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic **1815** may include, or be coupled to code and/or data storage **1801** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment code and/or data storage **1801** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage **1801** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0205] In at least one embodiment, any portion of code and/or data storage **1801** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or code and/or data storage **1801** may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or code and/or data storage **1801** is internal or external to a processor, for example, or comprising DRAM, SRAM, flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0206] In at least one embodiment, inference and/or training logic **1815** may include, without limitation, a code and/or data storage **1805** to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage **1805** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic **1815** may include, or be coupled to code and/or data storage **1805** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs).

[0207] In at least one embodiment, code, such as graph code, causes loading of weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment, any portion of code and/or data storage **1805** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. In at least one embodiment, any portion of code and/or data storage **1805** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **1805** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or data storage **1805** is internal or external to a processor, in at least one embodiment, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0208] In at least one embodiment, code and/or data storage **1801** and code and/or data storage **1805** may be separate storage structures. In at least one embodiment, code and/or data storage **1801** and code and/or data storage **1805** may be a combined storage structure. In at least one embodiment, code and/or data storage **1801** and code and/or data storage **1805** may be partially combined and partially separate. In at least one embodiment, any portion of code and/or data storage **1801** and code and/or data storage **1805** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0209] In at least one embodiment, inference and/or training logic **1815** may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") **1810**, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage **1820** that are functions of input/output and/or weight parameter data stored in code and/or data storage **1801** and/or code and/or data storage **1805**. In at least one embodiment, activations stored in activation storage **1820** are generated according to linear algebraic and or matrix-based mathematics performed by ALU(s) **1810** in response to performing instructions or other code, wherein weight values stored in code and/or data storage **1805** and/or data storage **1801** are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage **1805** or code and/or data storage **1801** or another storage on or off-chip.

[0210] In at least one embodiment, ALU(s) **1810** are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) **1810** may be external to a processor or other hardware logic device or circuit that uses them (e.g., a coprocessor). In at least one embodiment, ALUs **1810** may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage **1801**, code and/or data storage **1805**, and activation storage **1820** may share a processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage **1820** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

[0211] In at least one embodiment, activation storage **1820** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, activation storage **1820** may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, a choice of whether activation storage **1820** is internal or external to a processor, in at least one embodiment, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0212] In at least one embodiment, inference and/or training logic **1815** illustrated in FIG. **18**A may be used in conjunction with an application-specific integrated circuit ("ASIC"), such as a TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **1815** illustrated in FIG. **18**A may be used in conjunction with central processing unit ("CPU") hardware, graphics processing unit ("GPU") hardware or other hardware, such as field programmable gate arrays ("FPGAs").

[0213] FIG. **18**B illustrates inference and/or training logic **1815**, according to at least one embodiment. In at least one embodiment, inference and/or training logic **1815** may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic **1815** illustrated in FIG. **18**B may be used in conjunction with an application-specific integrated circuit (ASIC), such as TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **1815** illustrated in FIG. **18**B may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic **1815** includes, without limitation, code and/or data storage **1801** and code and/or data storage **1805**, which may be used to store code (e.g., graph code), weight values and/or other information, including bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. **18**B, each of code and/or data storage **1801** and code and/or data storage **1805** is associated with a dedicated computational resource, such as computational hardware **1802** and computational hardware **1806**, respectively. In at least one embodiment, each of computational hardware **1802** and computational hardware **1806** comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage **1801** and code and/or data storage **1805**, respectively, result of which is stored in activation storage **1820**.

[0214] In at least one embodiment, each of code and/or data storage **1801** and **1805** and corresponding computational hardware **1802** and **1806**, respectively, correspond to different layers of a neural network, such that resulting activation from one storage/computational pair **1801/1802** of code and/or data storage **1801** and computational hardware **1802** is provided as an input to a next storage/computational pair **1805/1806** of code and/or data storage **1805** and computational hardware **1806**, in order to mirror a conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs **1801/1802** and **1805/1806** may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not shown) subsequent to or in parallel with storage/computation pairs **1801/1802** and **1805/1806** may be included in inference and/or training logic **1815**.

[0215] FIG. **19** illustrates training and deployment of a deep neural network, according to at least one embodiment. In at least one embodiment, untrained neural network **1906** is trained using a training dataset **1902**. In at least one embodiment, training framework **1904** is a PyTorch framework, whereas in other embodiments, training framework **1904** is a TensorFlow, Boost, Caffe, Microsoft Cognitive Toolkit/CNTK, MXNet, Chainer, Keras, Deeplearning4j, or other training framework. In at least one embodiment, training framework **1904** trains an untrained neural network **1906** and enables it to be trained using processing resources described herein to generate a trained neural network **1908**. In at least one embodiment, weights may be chosen randomly or by pre-training using a deep belief network. In at least one embodiment, training may be performed in either a supervised, partially supervised, or unsupervised manner.

[0216] In at least one embodiment, untrained neural network **1906** is trained using supervised learning, wherein training dataset **1902** includes an input paired with a desired output for an input, or where training dataset **1902** includes input having a known output and an output of neural network **1906** is manually graded. In at least one embodiment, untrained neural network **1906** is trained in a supervised manner and processes inputs from training dataset **1902** and compares resulting outputs against a set of expected or desired outputs. In at least one embodiment, errors are then propagated back through untrained neural network **1906**. In at least one embodiment, training framework **1904** adjusts weights that control untrained neural network **1906**. In at least one embodiment, training framework **1904** includes tools to monitor how well untrained neural network **1906** is converging towards a model, such as trained neural network **1908**, suitable to generating correct answers, such as in result **1914**, based on input data such as a new dataset **1912**. In at least one embodiment, training framework **1904** trains untrained neural network **1906** repeatedly while adjust weights to refine an output of untrained neural network **1906** using a loss function and adjustment algorithm, such as stochastic gradient descent. In at least one embodiment, training framework **1904** trains untrained neural network **1906** until untrained neural network **1906** achieves a desired accuracy. In at least one embodiment, trained neural network **1908** can then be deployed to implement any number of machine learning operations.

[0217] In at least one embodiment, untrained neural network **1906** is trained using unsupervised learning, wherein untrained neural network **1906** attempts to train itself using unlabeled data. In at least one embodiment, unsupervised learning training dataset **1902** will include input data without any associated output data or "ground truth" data. In at least one embodiment, untrained neural network **1906** can learn groupings within training dataset **1902** and can determine how individual inputs are related to untrained dataset **1902**. In at least one embodiment, unsupervised training can be used to generate a self-organizing map in trained neural network **1908** capable of performing operations useful in reducing dimensionality of new dataset **1912**. In at least one embodiment, unsupervised training can also be used to perform anomaly detection, which allows identification of data points in new dataset **1912** that deviate from normal patterns of new dataset **1912**.

[0218] In at least one embodiment, semi-supervised learning may be used, which is a technique in which in training dataset **1902** includes a mix of labeled and unlabeled data. In at least one embodiment, training framework **1904** may be used to perform incremental learning, such as through transferred learning techniques. In at least one embodiment, incremental learning enables trained neural network **1908** to adapt to new dataset **1912** without forgetting knowledge instilled within trained neural network **1408** during initial training.

[0219] In at least one embodiment, training framework **1904** is a framework processed in connection with a software development toolkit such as an OpenVINO (Open Visual Inference and Neural network Optimization) toolkit. In at least one embodiment, an OpenVINO toolkit is a toolkit such as those developed by Intel Corporation of Santa Clara, CA.

[0220] In at least one embodiment, OpenVINO is a toolkit for facilitating development of applications, specifically neural network applications, for various tasks and operations, such as human vision emulation, speech recognition, natural language processing, recommendation systems, and/or variations thereof. In at least one embodiment, OpenVINO supports neural networks such as convolutional neural networks (CNNs), recurrent and/or attention-based neural networks, and/or various other neural network models. In at least one embodiment, OpenVINO supports various software libraries such as OpenCV, OpenCL, and/or variations thereof.

[0221] In at least one embodiment, OpenVINO supports neural network models for various tasks and operations, such as classification, segmentation, object detection, face recognition, speech recognition, pose estimation (e.g., humans and/or objects), monocular depth estimation, image inpainting, style transfer, action recognition, colorization, and/or variations thereof.

[0222] In at least one embodiment, OpenVINO comprises one or more software tools and/or modules for model optimization, also referred to as a model optimizer. In at least one embodiment, a model optimizer is a command line tool that facilitates transitions between training and deployment of neural network models. In at least one embodiment, a model optimizer optimizes neural network models for execution on various devices and/or processing units, such as a GPU, CPU, PPU, GPGPU, and/or variations thereof. In at least one embodiment, a model optimizer generates an internal representation of a model, and optimizes said model to generate an intermediate representation. In at least one embodiment, a model optimizer reduces a number of layers of a model. In at least one embodiment, a model optimizer removes layers of a model that are utilized for training. In at least one embodiment, a model optimizer performs various neural network operations, such as modifying inputs to a model (e.g., resizing inputs to a model), modifying a size of inputs of a model (e.g., modifying a batch size of a model), modifying a model structure (e.g., modifying layers of a model), normalization, standardization, quantization (e.g., converting weights of a model from a first representation, such as floating point, to a second representation, such as integer), and/or variations thereof.

[0223] In at least one embodiment, OpenVINO comprises one or more software libraries for inferencing, also referred to as an inference engine. In at least one embodiment, an inference engine is a C++ library, or any suitable programming language library. In at least one embodiment, an inference engine is utilized to infer input data. In at least one embodiment, an inference engine implements various classes to infer input data and generate one or more results. In at least one embodiment, an inference engine implements one or more API functions to process an intermediate representation, set input and/or output formats, and/or execute a model on one or more devices.

[0224] In at least one embodiment, OpenVINO provides various abilities for heterogeneous execution of one or more neural network models. In at least one embodiment, heterogeneous execution, or heterogeneous computing, refers to one or more computing processes and/or systems that utilize one or more types of processors and/or cores. In at least one embodiment, OpenVINO provides various software functions to execute a program on one or more devices. In at least one embodiment, OpenVINO provides various software functions to execute a program and/or portions of a program on different devices. In at least one embodiment, OpenVINO provides various software functions to, for example, run a first portion of code on a CPU and a second portion of code on a GPU and/or FPGA. In at least one embodiment, OpenVINO provides various software functions to execute one or more layers of a neural network on one or more devices (e.g., a first set of layers on a first device, such as a GPU, and a second set of layers on a second device, such as a CPU).

[0225] In at least one embodiment, OpenVINO includes various functionality similar to functionalities associated with a CUDA programming model, such as various neural network model operations associated with frameworks such as TensorFlow, PyTorch, and/or variations thereof. In at least one embodiment, one or more CUDA programming model operations are performed using OpenVINO. In at least one embodiment, various systems, methods, and/or techniques described herein are implemented using OpenVINO.

5G Networks

[0226] The following figures set forth, without limitation, exemplary 5G network-based systems that can be used to implement at least one embodiment.

[0227] FIG. **20** illustrates architecture of a system **2000** of a network, in accordance with at least one embodiment. In at least one embodiment, system **2000** is shown to include a user equipment (UE) **2002** and a UE **2004**. In at least one embodiment, UEs **2002** and **2004** are illustrated as smartphones (e.g., handheld touchscreen mobile computing devices connectable to one or more cellular networks) but may also comprise any mobile or non-mobile computing device, such as Personal Data Assistants (PDAs), pagers, laptop computers, desktop computers, wireless handsets, or any computing device including a wireless communications interface.

[0228] In at least one embodiment, any of UEs **2002** and **2004** can comprise an Internet of Things (IoT) UE, which can comprise a network access layer designed for low-power IoT applications utilizing short-lived UE connections. In at least one embodiment, an IoT UE can utilize technologies such as machine-to-machine (M2M) or machine-type communications (MTC) for exchanging data with an MTC server or device via a public land mobile network (PLMN), Proximity-Based Service (ProSe) or device-to-device (D2D) communication, sensor networks, or IoT networks. In at least one embodiment, a M2M or MTC exchange of data may be a machine-initiated exchange of data. In at least one embodiment, an IoT network describes interconnecting IoT UEs, which may include uniquely identifiable embedded computing devices (within Internet infrastructure), with short-lived connections. In at least one embodiment, an IoT UEs may execute background applications (e.g., keep alive messages, status updates, etc.) to facilitate connections of an IoT network.

[0229] In at least one embodiment, UEs **2002** and **2004** may be configured to connect, e.g., communicatively couple, with a radio access network (RAN) **2016**. In at least one embodiment, RAN **2016** may be, in at least one embodiment, an Evolved Universal Mobile Telecommunications System (UMTS) Terrestrial Radio Access Network (E-UTRAN), a NextGen RAN (NG RAN), or some other type of RAN. In at least one embodiment, UEs **2002** and **2004** utilize connections **2012** and **2014**, respectively, each of which comprises a physical communications interface or layer. In at least one embodiment, connections **2012** and **2014** are illustrated as an air interface to enable communicative coupling, and can be consistent with cellular communications protocols, such as a Global System for Mobile Communications (GSM) protocol, a code-division multiple access (CDMA) network protocol, a Push-to-Talk (PTT) protocol, a PTT over Cellular (POC) protocol, a Universal Mobile Telecommunications System (UMTS) protocol, a 3GPP Long Term Evolution (LTE) protocol, a fifth generation (5G) protocol, a New Radio (NR) protocol, and variations thereof.

[0230] In at least one embodiment, UEs **2002** and **2004** may further directly exchange communication data via a ProSe interface **2006**. In at least one embodiment, ProSe interface **2006** may alternatively be referred to as a sidelink interface comprising one or more logical channels, including but not limited to a Physical Sidelink Control Channel (PSCCH), a Physical Sidelink Shared Channel (PSSCH), a Physical Sidelink Discovery Channel (PSDCH), and a Physical Sidelink Broadcast Channel (PSBCH).

[0231] In at least one embodiment, UE **2004** is shown to be configured to access an access point (AP) **2010** via connection **2008**. In at least one embodiment, connection **2008** can comprise a local wireless connection, such as a connection consistent with any IEEE 802.11 protocol, wherein AP **2010** would comprise a wireless fidelity (WiFi®) router. In at least one embodiment, AP **2010** is shown to be connected to an Internet without connecting to a core network of a wireless system.

[0232] In at least one embodiment, RAN **2016** can include one or more access nodes that enable connections **2012** and **2014**. In at least one embodiment, these access nodes (ANs) can be referred to as base stations (BSs), NodeBs, evolved NodeBs (eNBs), next Generation NodeBs (gNB), RAN nodes, and so forth, and can comprise ground stations (e.g., terrestrial access points) or satellite stations providing coverage within a geographic area (e.g., a cell). In at least one embodiment, RAN **2016** may include one or more RAN nodes for providing macrocells, e.g., macro RAN node **2018**, and one or more RAN nodes for providing femtocells or picocells (e.g., cells having smaller coverage areas, smaller user capacity, or higher bandwidth compared to macrocells), e.g., low power (LP) RAN node **2020**.

[0233] In at least one embodiment, any of RAN nodes **2018** and **2020** can terminate an air interface protocol and can be a first point of contact for UEs **2002** and **2004**. In at least one embodiment, any of RAN nodes **2018** and **2020** can fulfill various logical functions for RAN **2016** including, but not limited to, radio network controller (RNC) functions such as radio bearer management, uplink and downlink dynamic radio resource management and data packet scheduling, and mobility management.

[0234] In at least one embodiment, UEs **2002** and **2004** can be configured to communicate using Orthogonal Frequency-Division Multiplexing (OFDM) communication signals with each other or with any of RAN nodes **2018** and **2020** over a multi-carrier communication channel in accordance various communication techniques, such as, but not limited to, an Orthogonal Frequency Division Multiple Access (OFDMA) communication technique (e.g., for downlink communications) or a Single Carrier Frequency Division Multiple Access (SC-FDMA) communication technique (e.g., for uplink and ProSe or sidelink communications), and/or variations thereof. In at least one embodiment, OFDM signals can comprise a plurality of orthogonal sub-carriers.

[0235] In at least one embodiment, a downlink resource grid can be used for downlink transmissions from any of RAN nodes **2018** and **2020** to UEs **2002** and **2004**, while uplink transmissions can utilize similar techniques. In at least one embodiment, a grid can be a time frequency grid, called a resource grid or time-frequency resource grid, which is a physical resource in a downlink in each slot. In at least one embodiment, such a time frequency plane representation is a common practice for OFDM systems, which makes it intuitive for radio resource allocation. In at least one

embodiment, each column and each row of a resource grid corresponds to one OFDM symbol and one OFDM subcarrier, respectively. In at least one embodiment, a duration of a resource grid in a time domain corresponds to one slot in a radio frame. In at least one embodiment, a smallest time-frequency unit in a resource grid is denoted as a resource element. In at least one embodiment, each resource grid comprises a number of resource blocks, which describe a mapping of certain physical channels to resource elements. In at least one embodiment, each resource block comprises a collection of resource elements. In at least one embodiment, in a frequency domain, this may represent a smallest quantity of resources that currently can be allocated. In at least one embodiment, there are several different physical downlink channels that are conveyed using such resource blocks.
[0236] In at least one embodiment, a physical downlink shared channel (PDSCH) may carry user data and higher-layer signaling to UEs **2002** and **2004**. In at least one embodiment, a physical downlink control channel (PDCCH) may carry information about a transport format and resource allocations related to PDSCH channel, among other things. In at least one embodiment, it may also inform UEs **2002** and **2004** about a transport format, resource allocation, and HARQ (Hybrid Automatic Repeat Request) information related to an uplink shared channel. In at least one embodiment, typically, downlink scheduling (assigning control and shared channel resource blocks to UE **2002** within a cell) may be performed at any of RAN nodes **2018** and **2020** based on channel quality information fed back from any of UEs **2002** and **2004**. In at least one embodiment, downlink resource assignment information may be sent on a PDCCH used for (e.g., assigned to) each of UEs **2002** and **2004**.
[0237] In at least one embodiment, a PDCCH may use control channel elements (CCEs) to convey control information. In at least one embodiment, before being mapped to resource elements, PDCCH complex valued symbols may first be organized into quadruplets, which may then be permuted using a sub-block interleaver for rate matching. In at least one embodiment, each PDCCH may be transmitted using one or more of these CCEs, where each CCE may correspond to nine sets of four physical resource elements known as resource element groups (REGs). In at least one embodiment, four Quadrature Phase Shift Keying (QPSK) symbols may be mapped to each REG. In at least one embodiment, PDCCH can be transmitted using one or more CCEs, depending on a size of a downlink control information (DCI) and a channel condition. In at least one embodiment, there can be four or more different PDCCH formats defined in LTE with different numbers of CCEs (e.g., aggregation level, L=1, 2, 4, or 8).
[0238] In at least one embodiment, an enhanced physical downlink control channel (EPDCCH) that uses PDSCH resources may be utilized for control information transmission. In at least one embodiment, EPDCCH may be transmitted using one or more enhanced control channel elements (ECCEs). In at least one embodiment, each ECCE may correspond to nine sets of four physical resource elements known as an enhanced resource element groups (EREGs). In at least one embodiment, an ECCE may have other numbers of EREGs in some situations.
[0239] In at least one embodiment, RAN **2016** is shown to be communicatively coupled to a core network (CN) **2038** via an S1 interface **2022**. In at least one embodiment, CN **2038** may be an evolved packet core (EPC) network, a NextGen Packet Core (NPC) network, or some other type of CN. In at least one embodiment, S1 interface **2022** is split into two parts: S1-U interface **2026**, which carries traffic data between RAN nodes **2018** and **2020** and serving gateway (S-GW) **2030**, and a S1-mobility management entity (MME) interface **2024**, which is a signaling interface between RAN nodes **2018** and **2020** and MMEs **2028**.
[0240] In at least one embodiment, CN **2038** comprises MMEs **2028**, S-GW **2030**, Packet Data Network (PDN) Gateway (P-GW) **2034**, and a home subscriber server (HSS) **2032**. In at least one embodiment, MMEs **2028** may be similar in function to a control plane of legacy Serving General Packet Radio Service (GPRS) Support Nodes (SGSN). In at least one embodiment, MMEs **2028** may manage mobility aspects in access such as gateway selection and tracking area list management. In at least one embodiment, HSS **2032** may comprise a database for network users, including subscription related information to support a network entities' handling of communication sessions.

In at least one embodiment, CN **2038** may comprise one or several HSSs **2032**, depending on a number of mobile subscribers, on a capacity of an equipment, on an organization of a network, etc. In at least one embodiment, HSS **2032** can provide support for routing/roaming, authentication, authorization, naming/addressing resolution, location dependencies, etc.

[0241] In at least one embodiment, S-GW **2030** may terminate a S1 interface **2022** towards RAN **2016**, and routes data packets between RAN **2016** and CN **2038**. In at least one embodiment, S-GW **2030** may be a local mobility anchor point for inter-RAN node handovers and also may provide an anchor for inter-3GPP mobility. In at least one embodiment, other responsibilities may include lawful intercept, charging, and some policy enforcement.

[0242] In at least one embodiment, P-GW **2034** may terminate an SGi interface toward a PDN. In at least one embodiment, P-GW **2034** may route data packets between an EPC network **2038** and external networks such as a network including application server **2040** (alternatively referred to as application function (AF)) via an Internet Protocol (IP) interface **2042**. In at least one embodiment, application server **2040** may be an element offering applications that use IP bearer resources with a core network (e.g., UMTS Packet Services (PS) domain, LTE PS data services, etc.). In at least one embodiment, P-GW **2034** is shown to be communicatively coupled to an application server **2040** via an IP communications interface **2042**. In at least one embodiment, application server **2040** can also be configured to support one or more communication services (e.g., Voice-over-Internet Protocol (VoIP) sessions, PTT sessions, group communication sessions, social networking services, etc.) for UEs **2002** and **2004** via CN **2038**.

[0243] In at least one embodiment, P-GW **2034** may further be a node for policy enforcement and charging data collection. In at least one embodiment, policy and Charging Enforcement Function (PCRF) **2036** is a policy and charging control element of CN **2038**. In at least one embodiment, in a non-roaming scenario, there may be a single PCRF in a Home Public Land Mobile Network (HPLMN) associated with a UE's Internet Protocol Connectivity Access Network (IP-CAN) session. In at least one embodiment, in a roaming scenario with local breakout of traffic, there may be two PCRFs associated with a UE's IP-CAN session: a Home PCRF (H-PCRF) within a HPLMN and a Visited PCRF (V-PCRF) within a Visited Public Land Mobile Network (VPLMN). In at least one embodiment, PCRF **2036** may be communicatively coupled to application server **2040** via P-GW **2034**. In at least one embodiment, application server **2040** may signal PCRF **2036** to indicate a new service flow and select an appropriate Quality of Service (QoS) and charging parameters. In at least one embodiment, PCRF **2036** may provision this rule into a Policy and Charging Enforcement Function (PCEF) (not shown) with an appropriate traffic flow template (TFT) and QoS class of identifier (QCI), which commences a QoS and charging as specified by application server **2040**.

[0244] FIG. **21** illustrates an architecture of a system **2100** of a network in accordance with some embodiments. In at least one embodiment, system **2100** is shown to include a UE **2102**, a 5G access node or RAN node (shown as (R)AN node **2108**), a User Plane Function (shown as UPF **2104**), a Data Network (DN **2106**), which may be, in at least one embodiment, operator services, Internet access or 3rd party services, and a 5G Core Network (5GC) (shown as CN **2110**).

[0245] In at least one embodiment, CN **2110** includes an Authentication Server Function (AUSF **2114**); a Core Access and Mobility Management Function (AMF **2112**); a Session Management Function (SMF **2118**); a Network Exposure Function (NEF **2116**); a Policy Control Function (PCF **2122**); a Network Function (NF) Repository Function (NRF **2120**); a Unified Data Management (UDM **2124**); and an Application Function (AF **2126**). In at least one embodiment, CN **2110** may also include other elements that are not shown, such as a Structured Data Storage network function (SDSF), an Unstructured Data Storage network function (UDSF), and variations thereof.

[0246] In at least one embodiment, UPF **2104** may act as an anchor point for intra-RAT and inter-RAT mobility, an external PDU session point of interconnect to DN **2106**, and a branching point to support multi-homed PDU session. In at least one embodiment, UPF **2104** may also perform packet routing and forwarding, packet inspection, enforce user plane part of policy rules, lawfully intercept packets (UP collection); traffic usage reporting, perform QoS handling for user plane (e.g. packet filtering, gating, UL/DL rate enforcement), perform Uplink Traffic verification (e.g., SDF to QoS flow mapping), transport level packet marking in uplink and downlink, and downlink packet buffering and downlink data notification triggering. In at least one embodiment, UPF **2104** may include an uplink classifier to support routing traffic flows to a data network. In at least one embodiment, DN **2106** may represent various network operator services, Internet access, or third party services.

[0247] In at least one embodiment, AUSF **2114** may store data for authentication of UE **2102** and handle authentication related functionality. In at least one embodiment, AUSF **2114** may facilitate a common authentication framework for various access types.

[0248] In at least one embodiment, AMF **2112** may be responsible for registration management (e.g., for registering UE **2102**, etc.), connection management, reachability management, mobility management, and lawful interception of AMF-related events, and access authentication and authorization. In at least one embodiment, AMF **2112** may provide transport for SM messages for SMF **2118**, and act as a transparent proxy for routing SM messages. In at least one embodiment, AMF **2112** may also provide transport for short message service (SMS) messages between UE **2102** and an SMS function (SMSF) (not shown by FIG. **21**). In at least one embodiment, AMF **2112** may act as Security Anchor Function (SEA), which may include interaction with AUSF **2114** and UE **2102** and receipt of an intermediate key that was established as a result of UE **2102** authentication process. In at least one embodiment, where USIM based authentication is used, AMF **2112** may retrieve security material from AUSF **2114**. In at least one embodiment, AMF **2112** may also include a Security Context Management (SCM) function, which receives a key from SEA that it uses to derive access-network specific keys. In at least one embodiment, furthermore, AMF **2112** may be a termination point of RAN CP interface (N2 reference point), a termination point of NAS (NI) signaling, and perform NAS ciphering and integrity protection.

[0249] In at least one embodiment, AMF **2112** may also support NAS signaling with a UE **2102** over an N3 interworking-function (IWF) interface. In at least one embodiment, N3IWF may be used to provide access to untrusted entities. In at least one embodiment, N3IWF may be a termination point for N2 and N3 interfaces for control plane and user plane, respectively, and as such, may handle N2 signaling from SMF and AMF for PDU sessions and QoS, encapsulate/de-encapsulate packets for IPSec and N3 tunneling, mark N3 user-plane packets in uplink, and enforce QoS corresponding to N3 packet marking taking into account QoS requirements associated to such marking received over N2. In at least one embodiment, N3IWF may also relay uplink and downlink control-plane NAS (NI) signaling between UE **2102** and AMF **2112**, and relay uplink and downlink user-plane packets between UE **2102** and UPF **2104**. In at least one embodiment, N3IWF also provides mechanisms for IPsec tunnel establishment with UE **2102**.

[0250] In at least one embodiment, SMF **2118** may be responsible for session management (e.g., session establishment, modify and release, including tunnel maintain between UPF and AN node); UE IP address allocation & management (including optional Authorization); Selection and control of UP function; Configures traffic steering at UPF to route traffic to proper destination; termination of interfaces towards Policy control functions; control part of policy enforcement and QoS; lawful intercept (for SM events and interface to LI System); termination of SM parts of NAS messages; downlink Data Notification; initiator of AN specific SM information, sent via AMF over N2 to AN; determine SSC mode of a session. In at least one embodiment, SMF **2118** may include following roaming functionality: handle local enforcement to apply QoS SLAB (VPLMN); charging data collection and charging interface (VPLMN); lawful intercept (in VPLMN for SM events and interface to LI System); support for interaction with external DN for transport of signaling for PDU session authorization/authentication by external DN.

[0251] In at least one embodiment, NEF **2116** may provide means for securely exposing services and capabilities provided by 3GPP network functions for third party, internal exposure/re-exposure, Application Functions (e.g., AF **2126**), edge computing or fog computing systems, etc. In at least one embodiment, NEF **2116** may authenticate, authorize, and/or throttle AFs. In at least one embodiment, NEF **2116** may also translate information exchanged with AF **2126** and information exchanged with internal network functions. In at least one embodiment, NEF **2116** may translate between an AF-Service-Identifier and an internal 5GC information. In at least one embodiment, NEF **2116** may also receive information from other network functions (NFs) based on exposed capabilities of other network functions. In at least one embodiment, this information may be stored at NEF **2116** as structured data, or at a data storage NF using a standardized interfaces. In at least one embodiment, stored information can then be re-exposed by NEF **2116** to other NFs and AFs, and/or used for other purposes such as analytics.

[0252] In at least one embodiment, NRF **2120** may support service discovery functions, receive NF Discovery Requests from NF instances, and provide information of discovered NF instances to NF instances. In at least one embodiment, NRF **2120** also maintains information of available NF instances and their supported services.

[0253] In at least one embodiment, PCF **2122** may provide policy rules to control plane function(s) to enforce them, and may also support unified policy framework to govern network behavior. In at least one embodiment, PCF **2122** may also implement a front end (FE) to access subscription information relevant for policy decisions in a UDR of UDM **2124**.

[0254] In at least one embodiment, UDM **2124** may handle subscription-related information to support a network entities' handling of communication sessions, and may store subscription data of UE **2102**. In at least one embodiment, UDM **2124** may include two parts, an application FE and a User Data Repository (UDR). In at least one embodiment, UDM may include a UDM FE, which is in charge of processing of credentials, location management, subscription management and so on. In at least one embodiment, several different front ends may serve a same user in different transactions. In at least one embodiment, UDM-FE accesses subscription information stored in an UDR and performs authentication credential processing; user identification handling; access authorization; registration/mobility management; and subscription management. In at least one embodiment, UDR may interact with PCF **2122**. In at least one embodiment, UDM **2124** may also support SMS management, wherein an SMS-FE implements a similar application logic as discussed previously.

[0255] In at least one embodiment, AF **2126** may provide application influence on traffic routing, access to a Network Capability Exposure (NCE), and interact with a policy framework for policy control. In at least one embodiment, NCE may be a mechanism that allows a 5GC and AF **2126** to provide information to each other via NEF **2116**, which may be used for edge computing implementations. In at least one embodiment, network operator and third party services may be hosted close to UE **2102** access point of attachment to achieve an efficient service delivery through a reduced end-to-end latency and load on a transport network. In at least one embodiment, for edge computing implementations, 5GC may select a UPF **2104** close to UE **2102** and execute traffic steering from UPF **2104** to DN **2106** via N6 interface. In at least one embodiment, this may be based on UE subscription data, UE location, and information provided by AF **2126**. In at least one embodiment, AF **2126** may influence UPF (re)selection and traffic routing. In at least one embodiment, based on operator deployment, when AF **2126** is considered to be a trusted entity, a network operator may permit AF **2126** to interact directly with relevant NFs.

[0256] In at least one embodiment, CN **2110** may include an SMSF, which may be responsible for SMS subscription checking and verification, and relaying SM messages to/from UE **2102** to/from other entities, such as an SMS-GMSC/IWMSC/SMS-router. In at least one embodiment, SMS may also interact with AMF **2112** and UDM **2124** for notification procedure that UE **2102** is available for SMS transfer (e.g., set a UE not reachable flag, and notifying UDM **2124** when UE **2102** is available for SMS).

[0257] In at least one embodiment, system **2100** may include following service-based interfaces: Namf: Service-based interface exhibited by AMF; Nsmf: Service-based interface exhibited by SMF; Nnef: Service-based interface exhibited by NEF; Npcf: Service-based interface exhibited by PCF; Nudm: Service-based interface exhibited by UDM; Naf: Service-based interface exhibited by AF; Nnrf: Service-based interface exhibited by NRF; and Nausf: Service-based interface exhibited by

AUSF.

[0258] In at least one embodiment, system **2100** may include following reference points: N1: Reference point between UE and AMF; N2: Reference point between (R)AN and AMF; N3: Reference point between (R)AN and UPF; N4: Reference point between SMF and UPF; and N6: Reference point between UPF and a Data Network. In at least one embodiment, there may be many more reference points and/or service-based interfaces between a NF services in NFs, however, these interfaces and reference points have been omitted for clarity. In at least one embodiment, an NS reference point may be between a PCF and AF; an N7 reference point may be between PCF and SMF; an N11 reference point between AMF and SMF; etc. In at least one embodiment, CN **2110** may include an Nx interface, which is an inter-CN interface between MME and AMF **2112** in order to enable interworking between CN **2110** and CN **7221**.

[0259] In at least one embodiment, system **2100** may include multiple RAN nodes (such as (R)AN node **2108**) wherein an Xn interface is defined between two or more (R)AN node **2108** (e.g., gNBs) that connecting to 5GC **410**, between a (R)AN node **2108** (e.g., gNB) connecting to CN **2110** and an eNB (e.g., a macro RAN node), and/or between two eNBs connecting to CN **2110**.

[0260] In at least one embodiment, Xn interface may include an Xn user plane (Xn-U) interface and an Xn control plane (Xn-C) interface. In at least one embodiment, Xn-U may provide non-guaranteed delivery of user plane PDUs and support/provide data forwarding and flow control functionality. In at least one embodiment, Xn-C may provide management and error handling functionality, functionality to manage a Xn-C interface; mobility support for UE **2102** in a connected mode (e.g., CM-CONNECTED) including functionality to manage UE mobility for connected mode between one or more (R)AN node **2108**. In at least one embodiment, mobility support may include context transfer from an old (source) serving (R)AN node **2108** to new (target) serving (R)AN node **2108**; and control of user plane tunnels between old (source) serving (R)AN node **2108** to new (target) serving (R)AN node **2108**.

[0261] In at least one embodiment, a protocol stack of a Xn-U may include a transport network layer built on Internet Protocol (IP) transport layer, and a GTP-U layer on top of a UDP and/or IP layer(s) to carry user plane PDUs. In at least one embodiment, Xn-C protocol stack may include an application layer signaling protocol (referred to as Xn Application Protocol (Xn-AP)) and a transport network layer that is built on an SCTP layer. In at least one embodiment, SCTP layer may be on top of an IP layer. In at least one embodiment, SCTP layer provides a guaranteed delivery of application layer messages. In at least one embodiment, in a transport IP layer point-to-point transmission is used to deliver signaling PDUs. In at least one embodiment, Xn-U protocol stack and/or a Xn-C protocol stack may be same or similar to an user plane and/or control plane protocol stack(s) shown and described herein.

[0262] FIG. **22** is an illustration of a control plane protocol stack in accordance with some embodiments. In at least one embodiment, a control plane **2200** is shown as a communications protocol stack between UE **2002** (or alternatively, UE **2004**), RAN **2016**, and MME(s) **2028**.

[0263] In at least one embodiment, PHY layer **2202** may transmit or receive information used by MAC layer **2204** over one or more air interfaces. In at least one embodiment, PHY layer **2202** may further perform link adaptation or adaptive modulation and coding (AMC), power control, cell search (e.g., for initial synchronization and handover purposes), and other measurements used by higher layers, such as an RRC layer **2210**. In at least one embodiment, PHY layer **2202** may still further perform error detection on transport channels, forward error correction (FEC) coding/de-coding of transport channels, modulation/demodulation of physical channels, interleaving, rate matching, mapping onto physical channels, and Multiple Input Multiple Output (MIMO) antenna processing.

[0264] In at least one embodiment, MAC layer **2204** may perform mapping between logical channels and transport channels, multiplexing of MAC service data units (SDUs) from one or more logical channels onto transport blocks (TB) to be delivered to PHY via transport channels, de-multiplexing MAC SDUs to one or more logical channels from transport blocks (TB) delivered from PHY via transport channels, multiplexing MAC SDUs onto TBs, scheduling information reporting, error correction through hybrid automatic repeat request (HARD), and logical channel prioritization.

[0265] In at least one embodiment, RLC layer **2206** may operate in a plurality of modes of operation, including: Transparent Mode (TM), Unacknowledged Mode (UM), and Acknowledged Mode (AM). In at least one embodiment, RLC layer **2206** may execute transfer of upper layer protocol data units (PDUs), error correction through automatic repeat request (ARQ) for AM data transfers, and concatenation, segmentation and reassembly of RLC SDUs for UM and AM data transfers. In at least one embodiment, RLC layer **2206** may also execute re-segmentation of RLC data PDUs for AM data transfers, reorder RLC data PDUs for UM and AM data transfers, detect duplicate data for UM and AM data transfers, discard RLC SDUs for UM and AM data transfers, detect protocol errors for AM data transfers, and perform RLC re-establishment.

[0266] In at least one embodiment, PDCP layer **2208** may execute header compression and decompression of IP data, maintain PDCP Sequence Numbers (SNs), perform in-sequence delivery of upper layer PDUs at re-establishment of lower layers, eliminate duplicates of lower layer SDUs at re-establishment of lower layers for radio bearers mapped on RLC AM, cipher and decipher control plane data, perform integrity protection and integrity verification of control plane data, control timer-based discard of data, and perform security operations (e.g., ciphering, deciphering, integrity protection, integrity verification, etc.).

[0267] In at least one embodiment, main services and functions of a RRC layer **2210** may include broadcast of system information (e.g., included in Master Information Blocks (MIBs) or System Information Blocks (SIBs) related to a non-access stratum (NAS)), broadcast of system information related to an access stratum (AS), paging, establishment, maintenance and release of an RRC connection between an UE and E-UTRAN (e.g., RRC connection paging, RRC connection establishment, RRC connection modification, and RRC connection release), establishment, configuration, maintenance and release of point-to-point radio bearers, security functions including key management, inter radio access technology (RAT) mobility, and measurement configuration for UE measurement reporting. In at least one embodiment, said MIBs and SIBs may comprise one or more information elements (IEs), which may each comprise individual data fields or data structures.

[0268] In at least one embodiment, UE **2002** and RAN **2016** may utilize a Uu interface (e.g., an LTE-Uu interface) to exchange control plane data via a protocol stack comprising PHY layer **2202**, MAC layer **2204**, RLC layer **2206**, PDCP layer **2208**, and RRC layer **2210**.

[0269] In at least one embodiment, non-access stratum (NAS) protocols (NAS protocols **2212**) form a highest stratum of a control plane between UE **2002** and MME(s) **2028**. In at least one embodiment, NAS protocols **2212** support mobility of UE **2002** and session management procedures to establish and maintain IP connectivity between UE **2002** and P-GW **2034**.

[0270] In at least one embodiment, Si Application Protocol (S1-AP) layer (Si-AP layer **2222**) may support functions of a Si interface and comprise Elementary Procedures (EPs). In at least one embodiment, an EP is a unit of interaction between RAN **2016** and CN **2028**. In at least one embodiment, S1-AP layer services may comprise two groups: UE-associated services and non UE-associated services. In at least one embodiment, these services perform functions including, but not limited to: E-UTRAN Radio Access Bearer (E-RAB) management, UE capability indication, mobility, NAS signaling transport, RAN Information Management (RIM), and configuration transfer.

[0271] In at least one embodiment, Stream Control Transmission Protocol (SCTP) layer (alternatively referred to as a stream control transmission protocol/internet protocol (SCTP/IP) layer) (SCTP layer **2220**) may ensure reliable delivery of signaling messages between RAN **2016** and MME(s) **2028** based, in part, on an IP protocol, supported by an IP layer **2218**. In at least one embodiment, L2 layer **2216** and an L1 layer **2214** may refer to communication links (e.g., wired or wireless) used by a RAN node and MME to exchange information.

[0272] In at least one embodiment, RAN **2016** and MME(s) **2028** may utilize an S1-MME interface to exchange control plane data via a protocol stack comprising a L1 layer **2214**, L2 layer **2216**, IP layer **2218**, SCTP layer **2220**, and Si-AP layer **2222**.

[0273] FIG. **23** is an illustration of a user plane protocol stack in accordance with at least one embodiment. In at least one embodiment, a user plane **2300** is shown as a communications protocol stack between a UE **2002**, RAN **2016**, S-GW **2030**, and P-GW **2034**. In at least one embodiment, user plane **2300** may utilize a same protocol layers as control plane **2200**. In at least one embodiment, UE **2002** and RAN **2016** may utilize a Uu interface (e.g., an LTE-Uu interface) to exchange user plane data via a protocol stack comprising PHY layer **2202**, MAC layer **2204**, RLC layer **2206**, PDCP layer **2208**.

[0274] In at least one embodiment, General Packet Radio Service (GPRS) Tunneling Protocol for a user plane (GTP-U) layer (GTP-U layer **2302**) may be used for carrying user data within a GPRS core network and between a radio access network and a core network. In at least one embodiment, user data transported can be packets in any of IPv4, IPv6, or PPP formats. In at least one embodiment, UDP and IP security (UDP/IP) layer (UDP/IP layer **2302**) may provide checksums for data integrity, port numbers for addressing different functions at a source and destination, and encryption and authentication on selected data flows. In at least one embodiment, RAN **2016** and S-GW **2030** may utilize an S1-U interface to exchange user plane data via a protocol stack comprising L1 layer **2214**, L2 layer **2216**, UDP/IP layer **2302**, and GTP-U layer **2302**. In at least one embodiment, S-GW **2030** and P-GW **2034** may utilize an S5/S8a interface to exchange user plane data via a protocol stack comprising L1 layer **2214**, L2 layer **2216**, UDP/IP layer **2302**, and GTP-U layer **2302**. In at least one embodiment, as discussed above with respect to FIG. **22**, NAS protocols support a mobility of UE **2002** and session management procedures to establish and maintain IP connectivity between UE **2002** and P-GW **2034**.

[0275] FIG. **24** illustrates components **2400** of a core network in accordance with at least one embodiment. In at least one embodiment, components of CN **2038** may be implemented in one physical node or separate physical nodes including components to read and execute instructions from a machine-readable or computer-readable medium (e.g., a non-transitory machine-readable storage medium). In at least one embodiment, Network Functions Virtualization (NFV) is utilized to virtualize any or all of above described network node functions via executable instructions stored in one or more computer readable storage mediums (described in further detail below). In at least one embodiment, a logical instantiation of CN **2038** may be referred to as a network slice **2402** (e.g., network slice **2402** is shown to include HSS **2032**, MME(s) **2028**, and S-GW **2030**). In at least one embodiment, a logical instantiation of a portion of CN **2038** may be referred to as a network sub-slice **2404** (e.g., network sub-slice **2404** is shown to include P-GW **2034** and PCRF **2036**).

[0276] In at least one embodiment, NFV architectures and infrastructures may be used to virtualize one or more network functions, alternatively performed by proprietary hardware, onto physical resources comprising a combination of industry-standard server hardware, storage hardware, or switches. In at least one embodiment, NFV systems can be used to execute virtual or reconfigurable implementations of one or more EPC components/functions.

[0277] FIG. **25** is a block diagram illustrating components, according to at least one embodiment, of a system **2500** to support network function virtualization (NFV). In at least one embodiment, system **2500** is illustrated as including a virtualized infrastructure manager (shown as VIM **2502**), a network function virtualization infrastructure (shown as NFVI **2504**), a VNF manager (shown as VNFM **2506**), virtualized network functions (shown as VNF **2508**), an element manager (shown as EM **2510**), an NFV Orchestrator (shown as NFVO **2512**), and a network manager (shown as NM **2514**).

[0278] In at least one embodiment, VIM **2502** manages resources of NFVI **2504**. In at least one embodiment, NFVI **2504** can include physical or virtual resources and applications (including hypervisors) used to execute system **2500**. In at least one embodiment, VIM **2502** may manage a life cycle of virtual resources with NFVI **2504** (e.g., creation, maintenance, and tear down of virtual machines (VMs) associated with one or more physical resources), track VM instances, track performance, fault and security of VM instances and associated physical resources, and expose VM instances and associated physical resources to other management systems.

[0279] In at least one embodiment, VNFM **2506** may manage VNF **2508**. In at least one embodiment, VNF **2508** may be used to execute EPC components/functions. In at least one embodiment, VNFM **2506** may manage a life cycle of VNF **2508** and track performance, fault and security of virtual aspects of VNF **2508**. In at least one embodiment, EM **2510** may track performance, fault and security of functional aspects of VNF **2508**. In at least one embodiment, tracking data from VNFM **2506** and EM **2510** may comprise, in at least one embodiment, performance measurement (PM) data used by VIM **2502** or NFVI **2504**. In at least one embodiment, both VNFM **2506** and EM **2510** can scale up/down a quantity of VNFs of system **2500**.
[0280] In at least one embodiment, NFVO **2512** may coordinate, authorize, release and engage resources of NFVI **2504** in order to provide a requested service (e.g., to execute an EPC function, component, or slice). In at least one embodiment, NM **2514** may provide a package of end-user functions with responsibility for a management of a network, which may include network elements with VNFs, non-virtualized network functions, or both (management of VNFs may occur via an EM **2510**).
Computer-Based Systems
[0281] The following figures set forth, without limitation, exemplary computer-based systems that can be used to implement at least one embodiment.
[0282] FIG. **26** is a block diagram of a processing system, according to at least one embodiment. In at least one embodiment, system **2600** includes one or more processors **2602** and one or more graphics processors **2608**, and may be a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processors **2602** or processor cores **2607**. In at least one embodiment, system **2600** is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices. In at least one embodiment, one or more graphics processors **2608** include one or more graphics cores.
[0283] In at least one embodiment, system **2600** can include, or be incorporated within a server-based gaming platform, a game console, including a game and media console, a mobile gaming console, a handheld game console, or an online game console. In at least one embodiment, system **2600** is a mobile phone, a smart phone, a tablet computing device or a mobile Internet device. In at least one embodiment, processing system **2600** can also include, couple with, or be integrated within a wearable device, such as a smart watch wearable device, a smart eyewear device, an augmented reality device, or a virtual reality device. In at least one embodiment, processing system **2600** is a television or set top box device having one or more processors **2602** and a graphical interface generated by one or more graphics processors **2608**.
[0284] In at least one embodiment, one or more processors **2602** each include one or more processor cores **2607** to process instructions which, when executed, perform operations for system and user software. In at least one embodiment, each of one or more processor cores **2607** is configured to process a specific instruction sequence **2609**. In at least one embodiment, instruction sequence **2609** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). In at least one embodiment, processor cores **2607** may each process a different instruction sequence **2609**, which may include instructions to facilitate emulation of other instruction sequences. In at least one embodiment, processor core **2607** may also include other processing devices, such a Digital Signal Processor (DSP).
[0285] In at least one embodiment, processor **2602** includes a cache memory **2604**. In at least one embodiment, processor **2602** can have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory is shared among various components of processor **2602**. In at least one embodiment, processor **2602** also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores **2607** using known cache coherency techniques. In at least one embodiment, a register file **2606** is additionally included in processor **2602**, which may include different types of registers for storing different types of data (e.g., integer registers, floating point registers, status registers, and an instruction pointer

register). In at least one embodiment, register file **2606** may include general-purpose registers or other registers.

[0286] In at least one embodiment, one or more processor(s) **2602** are coupled with one or more interface bus(es) **2610** to transmit communication signals such as address, data, or control signals between processor **2602** and other components in system **2600**. In at least one embodiment, interface bus **2610** can be a processor bus, such as a version of a Direct Media Interface (DMI) bus. In at least one embodiment, interface bus **2610** is not limited to a DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI Express), memory busses, or other types of interface busses. In at least one embodiment processor(s) **2602** include an integrated memory controller **2616** and a platform controller hub **2630**. In at least one embodiment, memory controller **2616** facilitates communication between a memory device and other components of system **2600**, while platform controller hub (PCH) **2630** provides connections to I/O devices via a local I/O bus.

[0287] In at least one embodiment, a memory device **2620** can be a dynamic random access memory (DRAM) device, a static random access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In at least one embodiment, memory device **2620** can operate as system memory for system **2600**, to store data **2622** and instructions **2621** for use when one or more processors **2602** executes an application or process. In at least one embodiment, memory controller **2616** also couples with an optional external graphics processor **2612**, which may communicate with one or more graphics processors **2608** in processors **2602** to perform graphics and media operations. In at least one embodiment, a display device **2611** can connect to processor(s) **2602**. In at least one embodiment, display device **2611** can include one or more of an internal display device, as in a mobile electronic device or a laptop device, or an external display device attached via a display interface (e.g., DisplayPort, etc.). In at least one embodiment, display device **2611** can include a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

[0288] In at least one embodiment, platform controller hub **2630** enables peripherals to connect to memory device **2620** and processor **2602** via a high-speed I/O bus. In at least one embodiment, I/O peripherals include, but are not limited to, an audio controller **2646**, a network controller **2634**, a firmware interface **2628**, a wireless transceiver **2626**, touch sensors **2625**, a data storage device **2624** (e.g., hard disk drive, flash memory, etc.). In at least one embodiment, data storage device **2624** can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI Express). In at least one embodiment, touch sensors **2625** can include touch screen sensors, pressure sensors, or fingerprint sensors. In at least one embodiment, wireless transceiver **2626** can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, or Long Term Evolution (LTE) transceiver. In at least one embodiment, firmware interface **2628** enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). In at least one embodiment, network controller **2634** can enable a network connection to a wired network. In at least one embodiment, a high-performance network controller (not shown) couples with interface bus **2610**. In at least one embodiment, audio controller **2646** is a multi-channel high definition audio controller. In at least one embodiment, system **2600** includes an optional legacy I/O controller **2640** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to system **2600**. In at least one embodiment, platform controller hub **2630** can also connect to one or more Universal Serial Bus (USB) controllers **2642** connect input devices, such as keyboard and mouse **2643** combinations, a camera **2644**, or other USB input devices.

[0289] In at least one embodiment, an instance of memory controller **2616** and platform controller hub **2630** may be integrated into a discreet external graphics processor, such as external graphics processor **2612**. In at least one embodiment, platform controller hub **2630** and/or memory controller **2616** may be external to one or more processor(s) **2602**. For example, in at least one embodiment, system **2600** can include an external memory controller **2616** and platform controller hub **2630**, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with processor(s) **2602**.

[0290] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment portions or all of inference and/or training logic **1815** may be incorporated into graphics processor **2608**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in a 3D pipeline. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIG. **18**A or **18**B. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of graphics processor **2608** to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

[0291] FIG. **27** is a block diagram illustrating an exemplary computer system, which may be a system with interconnected devices and components, a system-on-a-chip (SOC) or some combination thereof formed with a processor that may include execution units to execute an instruction, according to at least one embodiment. In at least one embodiment, a computer system **2700** may include, without limitation, a component, such as a processor **2702** to employ execution units including logic to perform algorithms for process data, in accordance with present disclosure, such as in embodiment described herein. In at least one embodiment, computer system **2700** may include processors, such as PENTIUM® Processor family, Xeon™, Itanium®, XScale™ and/or StrongARM™, Intel® Core™, or Intel® Nervana™ microprocessors available from Intel Corporation of Santa Clara, California, although other systems (including PCs having other microprocessors, engineering workstations, set-top boxes and like) may also be used. In at least one embodiment, computer system **2700** may execute a version of WINDOWS operating system available from Microsoft Corporation of Redmond, Wash., although other operating systems (UNIX and Linux, for example), embedded software, and/or graphical user interfaces, may also be used.

[0292] Embodiments may be used in other devices such as handheld devices and embedded applications. Some examples of handheld devices include cellular phones, Internet Protocol devices, digital cameras, personal digital assistants (“PDAs”), and handheld PCs. In at least one embodiment, embedded applications may include a microcontroller, a digital signal processor (“DSP”), system on a chip, network computers (“NetPCs”), set-top boxes, network hubs, wide area network (“WAN”) switches, or any other system that may perform one or more instructions in accordance with at least one embodiment.

[0293] In at least one embodiment, computer system **2700** may include, without limitation, processor **2702** that may include, without limitation, one or more execution units **2708** to perform machine learning model training and/or inferencing according to techniques described herein. In at least one embodiment, computer system **2700** is a single processor desktop or server system, but in another embodiment, computer system **2700** may be a multiprocessor system. In at least one embodiment, processor **2702** may include, without limitation, a complex instruction set computer (“CISC”) microprocessor, a reduced instruction set computing (“RISC”) microprocessor, a very long instruction word (“VLIW”) microprocessor, a processor implementing a combination of instruction sets, or any other processor device, such as a digital signal processor, for example. In at least one embodiment, processor **2702** may be coupled to a processor bus **2710** that may transmit data signals between processor **2702** and other components in computer system **2700**.

[0294] In at least one embodiment, processor **2702** may include, without limitation, a Level 1 (“L1”) internal cache memory (“cache”) **2704**. In at least one embodiment, processor **2702** may have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory may reside external to processor **2702**. Other embodiments may also include a combination of both internal and external caches depending on particular implementation and needs. In at least one embodiment, a register file **2706** may store different types of data in various registers including, without limitation, integer registers, floating point registers, status registers, and an instruction pointer register.

[0295] In at least one embodiment, execution unit **2708**, including, without limitation, logic to perform integer and floating point operations, also resides in processor **2702**. In at least one embodiment, processor **2702** may also include a microcode (“ucode”) read only memory (“ROM”) that stores microcode for certain macro instructions. In at least one embodiment, execution unit **2708** may include logic to handle a packed instruction set **2709**. In at least one embodiment, by including packed instruction set **2709** in an instruction set of a general-purpose processor, along with associated circuitry to execute instructions, operations used by many multimedia applications may be performed using packed data in processor **2702**. In at least one embodiment, many multimedia applications may be accelerated and executed more efficiently by using a full width of a processor's data bus for performing operations on packed data, which may eliminate a need to transfer smaller units of data across that processor's data bus to perform one or more operations one data element at a time.

[0296] In at least one embodiment, execution unit **2708** may also be used in microcontrollers, embedded processors, graphics devices, DSPs, and other types of logic circuits. In at least one embodiment, computer system **2700** may include, without limitation, a memory **2720**. In at least one embodiment, memory **2720** may be a Dynamic Random Access Memory (“DRAM”) device, a Static Random Access Memory (“SRAM”) device, a flash memory device, or another memory device. In at least one embodiment, memory **2720** may store instruction(s) **2719** and/or data **2721** represented by data signals that may be executed by processor **2702**.

[0297] In at least one embodiment, a system logic chip may be coupled to processor bus **2710** and memory **2720**. In at least one embodiment, a system logic chip may include, without limitation, a memory controller hub (“MCH”) **2716**, and processor **2702** may communicate with MCH **2716** via processor bus **2710**. In at least one embodiment, MCH **2716** may provide a high bandwidth memory path **2718** to memory **2720** for instruction and data storage and for storage of graphics commands, data and textures. In at least one embodiment, MCH **2716** may direct data signals between processor **2702**, memory **2720**, and other components in computer system **2700** and to bridge data signals between processor bus **2710**, memory **2720**, and a system I/O interface **2722**. In at least one embodiment, a system logic chip may provide a graphics port for coupling to a graphics controller. In at least one embodiment, MCH **2716** may be coupled to memory **2720** through high bandwidth memory path **2718** and a graphics/video card **2712** may be coupled to MCH **2716** through an Accelerated Graphics Port (“AGP”) interconnect **2714**.

[0298] In at least one embodiment, computer system **2700** may use system I/O interface **2722** as a proprietary hub interface bus to couple MCH **2716** to an I/O controller hub (“ICH”) **2730**. In at least one embodiment, ICH **2730** may provide direct connections to some I/O devices via a local I/O bus. In at least one embodiment, a local I/O bus may include, without limitation, a high-speed I/O bus for connecting peripherals to memory **2720**, a chipset, and processor **2702**. Examples may include, without limitation, an audio controller **2729**, a firmware hub (“flash BIOS”) **2728**, a wireless transceiver **2726**, a data storage **2724**, a legacy I/O controller **2723** containing user input and keyboard interfaces **2725**, a serial expansion port **2727**, such as a Universal Serial Bus (“USB”) port, and a network controller **2734**. In at least one embodiment, data storage **2724** may comprise a hard disk drive, a floppy disk drive, a CD-ROM device, a flash memory device, or other mass storage device.

[0299] In at least one embodiment, FIG. **27** illustrates a system, which includes interconnected hardware devices or “chips”, whereas in other embodiments, FIG. **27** may illustrate an exemplary SoC. In at least one embodiment, devices illustrated in FIG. **27** may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of computer system **2700** are interconnected using compute express link (CXL) interconnects.

[0300] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in system FIG. **27** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0301] FIG. **28** is a block diagram illustrating an electronic device **2800** for utilizing a processor **2810**, according to at least one embodiment. In at least one embodiment, electronic device **2800** may be, for example and without limitation, a notebook, a tower server, a rack server, a blade server, a laptop, a desktop, a tablet, a mobile device, a phone, an embedded computer, or any other suitable electronic device.

[0302] In at least one embodiment, electronic device **2800** may include, without limitation, processor **2810** communicatively coupled to any suitable number or kind of components, peripherals, modules, or devices. In at least one embodiment, processor **2810** is coupled using a bus or interface, such as a I2C bus, a System Management Bus ("SMBus"), a Low Pin Count (LPC) bus, a Serial Peripheral Interface ("SPI"), a High Definition Audio ("HDA") bus, a Serial Advance Technology Attachment ("SATA") bus, a Universal Serial Bus ("USB") (versions 1, 2, 3, etc.), or a Universal Asynchronous Receiver/Transmitter ("UART") bus. In at least one embodiment, FIG. **28** illustrates a system, which includes interconnected hardware devices or "chips", whereas in other embodiments, FIG. **28** may illustrate an exemplary SoC. In at least one embodiment, devices illustrated in FIG. **28** may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of FIG. **28** are interconnected using compute express link (CXL) interconnects.

[0303] In at least one embodiment, FIG. **28** may include a display **2824**, a touch screen **2825**, a touch pad **2830**, a Near Field Communications unit ("NFC") **2845**, a sensor hub **2840**, a thermal sensor **2846**, an Express Chipset ("EC") **2835**, a Trusted Platform Module ("TPM") **2838**, BIOS/firmware/flash memory ("BIOS, FW Flash") **2822**, a DSP **2860**, a drive **2820** such as a Solid State Disk ("SSD") or a Hard Disk Drive ("HDD"), a wireless local area network unit ("WLAN") **2850**, a Bluetooth unit **2852**, a Wireless Wide Area Network unit ("WWAN") **2856**, a Global Positioning System (GPS) unit **2855**, a camera ("USB 3.0 camera") **2854** such as a USB 3.0 camera, and/or a Low Power Double Data Rate ("LPDDR") memory unit ("LPDDR3") **2815** implemented in, for example, an LPDDR3 standard. These components may each be implemented in any suitable manner.

[0304] In at least one embodiment, other components may be communicatively coupled to processor **2810** through components described herein. In at least one embodiment, an accelerometer **2841**, an ambient light sensor ("ALS") **2842**, a compass **2843**, and a gyroscope **2844** may be communicatively coupled to sensor hub **2840**. In at least one embodiment, a thermal sensor **2839**, a fan **2837**, a keyboard **2836**, and touch pad **2830** may be communicatively coupled to EC **2835**. In at least one embodiment, speakers **2863**, headphones **2864**, and a microphone ("mic") **2865** may be communicatively coupled to an audio unit ("audio codec and class D amp") **2862**, which may in turn be communicatively coupled to DSP **2860**. In at least one embodiment, audio unit **2862** may include, for example and without limitation, an audio coder/decoder ("codec") and a class D amplifier. In at least one embodiment, a SIM card ("SIM") **2857** may be communicatively coupled to WWAN unit **2856**. In at least one embodiment, components such as WLAN unit **2850** and Bluetooth unit **2852**, as well as WWAN unit **2856** may be implemented in a Next Generation Form Factor ("NGFF").

[0305] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in system FIG. **28** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases

described herein.
[0306] FIG. **29** illustrates exemplary integrated circuits and associated graphics processors that may be fabricated using one or more IP cores, according to various embodiments described herein. In addition to what is illustrated, other logic and circuits may be included in at least one embodiment, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores.
[0307] FIG. **29** is a block diagram illustrating an exemplary system on a chip integrated circuit **2900** that may be fabricated using one or more IP cores, according to at least one embodiment. In at least one embodiment, integrated circuit **2900** includes one or more application processor(s) **2905** (e.g., CPUs), at least one graphics processor **2910**, and may additionally include an image processor **2915** and/or a video processor **2920**, any of which may be a modular IP core. In at least one embodiment, integrated circuit **2900** includes peripheral or bus logic including a USB controller **2925**, a UART controller **2930**, an SPI/SDIO controller **2935**, and an I22S/I22C controller **2940**. In at least one embodiment, integrated circuit **2900** can include a display device **2945** coupled to one or more of a high-definition multimedia interface (HDMI) controller **2950** and a mobile industry processor interface (MIPI) display interface **2955**. In at least one embodiment, storage may be provided by a flash memory subsystem **2960** including flash memory and a flash memory controller. In at least one embodiment, a memory interface may be provided via a memory controller **2965** for access to SDRAM or SRAM memory devices. In at least one embodiment, some integrated circuits additionally include an embedded security engine **2970**.
[0308] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in integrated circuit **2900** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.
[0309] FIG. **30** is a block diagram illustrating a computing system **3000** according to at least one embodiment. In at least one embodiment, computing system **3000** includes a processing subsystem **3001** having one or more processor(s) **3002** and a system memory **3004** communicating via an interconnection path that may include a memory hub **3005**. In at least one embodiment, memory hub **3005** may be a separate component within a chipset component or may be integrated within one or more processor(s) **3002**. In at least one embodiment, memory hub **3005** couples with an I/O subsystem **3011** via a communication link **3006**. In at least one embodiment, I/O subsystem **3011** includes an I/O hub **3007** that can enable computing system **3000** to receive input from one or more input device(s) **3008**. In at least one embodiment, I/O hub **3007** can enable a display controller, which may be included in one or more processor(s) **3002**, to provide outputs to one or more display device(s) **3010**A. In at least one embodiment, one or more display device(s) **3010**A coupled with I/O hub **3007** can include a local, internal, or embedded display device.
[0310] In at least one embodiment, processing subsystem **3001** includes one or more parallel processor(s) **3012** coupled to memory hub **3005** via a bus or other communication link **3013**. In at least one embodiment, communication link **3013** may use one of any number of standards based communication link technologies or protocols, such as, but not limited to PCI Express, or may be a vendor-specific communications interface or communications fabric. In at least one embodiment, one or more parallel processor(s) **3012** form a computationally focused parallel or vector processing system that can include a large number of processing cores and/or processing clusters, such as a many-integrated core (MIC) processor. In at least one embodiment, some or all of parallel processor(s) **3012** form a graphics processing subsystem that can output pixels to one of one or more display device(s) **3010**A coupled via I/O Hub **3007**. In at least one embodiment, parallel processor(s) **3012** can also include a display controller and display interface (not shown) to enable a direct connection to one or more display device(s) **3010**B. In at least one embodiment, parallel processor(s)

3012 include one or more cores, such as graphics cores **3500** discussed herein.
[0311] In at least one embodiment, a system storage unit **3014** can connect to I/O hub **3007** to provide a storage mechanism for computing system **3000**. In at least one embodiment, an I/O switch **3016** can be used to provide an interface mechanism to enable connections between I/O hub **3007** and other components, such as a network adapter **3018** and/or a wireless network adapter **3019** that may be integrated into platform, and various other devices that can be added via one or more add-in device(s) **3020**. In at least one embodiment, network adapter **3018** can be an Ethernet adapter or another wired network adapter. In at least one embodiment, wireless network adapter **3019** can include one or more of a Wi-Fi, Bluetooth, near field communication (NFC), or other network device that includes one or more wireless radios.
[0312] In at least one embodiment, computing system **3000** can include other components not explicitly shown, including USB or other port connections, optical storage drives, video capture devices, and like, may also be connected to I/O hub **3007**. In at least one embodiment, communication paths interconnecting various components in FIG. **30** may be implemented using any suitable protocols, such as PCI (Peripheral Component Interconnect) based protocols (e.g., PCI-Express), or other bus or point-to-point communication interfaces and/or protocol(s), such as NV-Link high-speed interconnect, or interconnect protocols.
[0313] In at least one embodiment, parallel processor(s) **3012** incorporate circuitry optimized for graphics and video processing, including, for example, video output circuitry, and constitutes a graphics processing unit (GPU), e.g., parallel processor(s) **3012** includes graphics core **3500**. In at least one embodiment, parallel processor(s) **3012** incorporate circuitry optimized for general purpose processing. In at least embodiment, components of computing system **3000** may be integrated with one or more other system elements on a single integrated circuit. For example, in at least one embodiment, parallel processor(s) **3012**, memory hub **3005**, processor(s) **3002**, and I/O hub **3007** can be integrated into a system on chip (SoC) integrated circuit. In at least one embodiment, components of computing system **3000** can be integrated into a single package to form a system in package (SIP) configuration. In at least one embodiment, at least a portion of components of computing system **3000** can be integrated into a multi-chip module (MCM), which can be interconnected with other multi-chip modules into a modular computing system.
[0314] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in system FIG. **3000** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

Processing Systems

[0315] The following figures set forth, without limitation, exemplary processing systems that can be used to implement at least one embodiment.
[0316] FIG. **31** illustrates an accelerated processing unit ("APU") **3100**, in accordance with at least one embodiment. In at least one embodiment, APU **3100** is developed by AMD Corporation of Santa Clara, CA. In at least one embodiment, APU **3100** can be configured to execute an application program, such as a CUDA program. In at least one embodiment, APU **3100** includes, without limitation, a core complex **3110**, a graphics complex **3140**, fabric **3160**, I/O interfaces **3170**, memory controllers **3180**, a display controller **3192**, and a multimedia engine **3194**. In at least one embodiment, APU **3100** may include, without limitation, any number of core complexes **3110**, any number of graphics complexes **3150**, any number of display controllers **3192**, and any number of multimedia engines **3194** in any combination. For explanatory purposes, multiple instances of like objects are denoted herein with reference numbers identifying an object and parenthetical numbers identifying an instance where needed.
[0317] In at least one embodiment, core complex **3110** is a CPU, graphics complex **3140** is a GPU,

and APU **3100** is a processing unit that integrates, without limitation, **3110** and **3140** onto a single chip. In at least one embodiment, some tasks may be assigned to core complex **3110** and other tasks may be assigned to graphics complex **3140**. In at least one embodiment, core complex **3110** is configured to execute main control software associated with APU **3100**, such as an operating system. In at least one embodiment, core complex **3110** is a master processor of APU **3100**, controlling and coordinating operations of other processors. In at least one embodiment, core complex **3110** issues commands that control an operation of graphics complex **3140**. In at least one embodiment, core complex **3110** can be configured to execute host executable code derived from CUDA source code, and graphics complex **3140** can be configured to execute device executable code derived from CUDA source code.

[0318] In at least one embodiment, core complex **3110** includes, without limitation, cores **3120**(**1**)-**3120**(**4**) and an L3 cache **3130**. In at least one embodiment, core complex **3110** may include, without limitation, any number of cores **3120** and any number and type of caches in any combination. In at least one embodiment, cores **3120** are configured to execute instructions of a particular instruction set architecture ("ISA"). In at least one embodiment, each core **3120** is a CPU core.

[0319] In at least one embodiment, each core **3120** includes, without limitation, a fetch/decode unit **3122**, an integer execution engine **3124**, a floating point execution engine **3126**, and an L2 cache **3128**. In at least one embodiment, fetch/decode unit **3122** fetches instructions, decodes such instructions, generates micro-operations, and dispatches separate micro-instructions to integer execution engine **3124** and floating point execution engine **3126**. In at least one embodiment, fetch/decode unit **3122** can concurrently dispatch one micro-instruction to integer execution engine **3124** and another micro-instruction to floating point execution engine **3126**. In at least one embodiment, integer execution engine **3124** executes, without limitation, integer and memory operations. In at least one embodiment, floating point engine **3126** executes, without limitation, floating point and vector operations. In at least one embodiment, fetch-decode unit **3122** dispatches micro-instructions to a single execution engine that replaces both integer execution engine **3124** and floating point execution engine **3126**.

[0320] In at least one embodiment, each core **3120**(*i*), where i is an integer representing a particular instance of core **3120**, may access L2 cache **3128**(*i*) included in core **3120**(*i*). In at least one embodiment, each core **3120** included in core complex **3110**(*j*), where j is an integer representing a particular instance of core complex **3110**, is connected to other cores **3120** included in core complex **3110**(*j*) via L3 cache **3130**(*j*) included in core complex **3110**(*j*). In at least one embodiment, cores **3120** included in core complex **3110**(*j*), where j is an integer representing a particular instance of core complex **3110**, can access all of L3 cache **3130**(*j*) included in core complex **3110**(*j*). In at least one embodiment, L3 cache **3130** may include, without limitation, any number of slices.

[0321] In at least one embodiment, graphics complex **3140** can be configured to perform compute operations in a highly-parallel fashion. In at least one embodiment, graphics complex **3140** is configured to execute graphics pipeline operations such as draw commands, pixel operations, geometric computations, and other operations associated with rendering an image to a display. In at least one embodiment, graphics complex **3140** is configured to execute operations unrelated to graphics. In at least one embodiment, graphics complex **3140** is configured to execute both operations related to graphics and operations unrelated to graphics.

[0322] In at least one embodiment, graphics complex **3140** includes, without limitation, any number of compute units **3150** and an L2 cache **3142**. In at least one embodiment, compute units **3150** share L2 cache **3142**. In at least one embodiment, L2 cache **3142** is partitioned. In at least one embodiment, graphics complex **3140** includes, without limitation, any number of compute units **3150** and any number (including zero) and type of caches. In at least one embodiment, graphics complex **3140** includes, without limitation, any amount of dedicated graphics hardware.

[0323] In at least one embodiment, each compute unit **3150** includes, without limitation, any number of SIMD units **3152** and a shared memory **3154**. In at least one embodiment, each SIMD unit **3152** implements a SIMD architecture and is configured to perform operations in parallel. In at least one embodiment, each compute unit **3150** may execute any number of thread blocks, but each thread block executes on a single compute unit **3150**. In at least one embodiment, a thread block includes, without limitation, any number of threads of execution. In at least one embodiment, a workgroup is a thread block. In at least one embodiment, each SIMD unit **3152** executes a different warp. In at least one embodiment, a warp is a group of threads (e.g., 16 threads), where each thread in a warp belongs to a single thread block and is configured to process a different set of data based on a single set of instructions. In at least one embodiment, predication can be used to disable one or more threads in a warp. In at least one embodiment, a lane is a thread. In at least one embodiment, a work item is a thread. In at least one embodiment, a wavefront is a warp. In at least one embodiment, different wavefronts in a thread block may synchronize together and communicate via shared memory **3154**.

[0324] In at least one embodiment, fabric **3160** is a system interconnect that facilitates data and control transmissions across core complex **3110**, graphics complex **3140**, I/O interfaces **3170**, memory controllers **3180**, display controller **3192**, and multimedia engine **3194**. In at least one embodiment, APU **3100** may include, without limitation, any amount and type of system interconnect in addition to or instead of fabric **3160** that facilitates data and control transmissions across any number and type of directly or indirectly linked components that may be internal or external to APU **3100**. In at least one embodiment, I/O interfaces **3170** are representative of any number and type of I/O interfaces (e.g., PCI, PCI-Extended ("PCI-X"), PCIe, gigabit Ethernet ("GBE"), USB, etc.). In at least one embodiment, various types of peripheral devices are coupled to I/O interfaces **3170** In at least one embodiment, peripheral devices that are coupled to I/O interfaces **3170** may include, without limitation, keyboards, mice, printers, scanners, joysticks or other types of game controllers, media recording devices, external storage devices, network interface cards, and so forth.

[0325] In at least one embodiment, display controller AMD92 displays images on one or more display device(s), such as a liquid crystal display ("LCD") device. In at least one embodiment, multimedia engine **240** includes, without limitation, any amount and type of circuitry that is related to multimedia, such as a video decoder, a video encoder, an image signal processor, etc. In at least one embodiment, memory controllers **3180** facilitate data transfers between APU **3100** and a unified system memory **3190**. In at least one embodiment, core complex **3110** and graphics complex **3140** share unified system memory **3190**.

[0326] In at least one embodiment, APU **3100** implements a memory subsystem that includes, without limitation, any amount and type of memory controllers **3180** and memory devices (e.g., shared memory **3154**) that may be dedicated to one component or shared among multiple components. In at least one embodiment, APU **3100** implements a cache subsystem that includes, without limitation, one or more cache memories (e.g., L2 caches **2728**, L3 cache **3130**, and L2 cache **3142**) that may each be private to or shared between any number of components (e.g., cores **3120**, core complex **3110**, SIMD units **3152**, compute units **3150**, and graphics complex **3140**).

[0327] FIG. **32** illustrates a CPU **3200**, in accordance with at least one embodiment. In at least one embodiment, CPU **3200** is developed by AMD Corporation of Santa Clara, CA. In at least one embodiment, CPU **3200** can be configured to execute an application program. In at least one embodiment, CPU **3200** is configured to execute main control software, such as an operating system. In at least one embodiment, CPU **3200** issues commands that control an operation of an external GPU (not shown). In at least one embodiment, CPU **3200** can be configured to execute host executable code derived from CUDA source code, and an external GPU can be configured to execute device executable code derived from such CUDA source code. In at least one embodiment, CPU **3200** includes, without limitation, any number of core complexes **3210**, fabric **3260**, I/O interfaces **3270**, and memory controllers **3280**.

[0328] In at least one embodiment, core complex **3210** includes, without limitation, cores **3220(1)-3220(4)** and an L3 cache **3230**. In at least one embodiment, core complex **3210** may include, without limitation, any number of cores **3220** and any number and type of caches in any combination. In at least one embodiment, cores **3220** are configured to execute instructions of a particular ISA. In at least one embodiment, each core **3220** is a CPU core.

[0329] In at least one embodiment, each core **3220** includes, without limitation, a fetch/decode unit **3222**, an integer execution engine **3224**, a floating point execution engine **3226**, and an L2 cache **3228**. In at least one embodiment, fetch/decode unit **3222** fetches instructions, decodes such instructions, generates micro-operations, and dispatches separate micro-instructions to integer execution engine **3224** and floating point execution engine **3226**. In at least one embodiment, fetch/decode unit **3222** can concurrently dispatch one micro-instruction to integer execution engine **3224** and another micro-instruction to floating point execution engine **3226**. In at least one embodiment, integer execution engine **3224** executes, without limitation, integer and memory operations. In at least one embodiment, floating point engine **3226** executes, without limitation, floating point and vector operations. In at least one embodiment, fetch-decode unit **3222** dispatches micro-instructions to a single execution engine that replaces both integer execution engine **3224** and floating point execution engine **3226**.

[0330] In at least one embodiment, each core **3220**(*i*), where i is an integer representing a particular instance of core **3220**, may access L2 cache **3228**(*i*) included in core **3220**(*i*). In at least one embodiment, each core **3220** included in core complex **3210**(*j*), where j is an integer representing a particular instance of core complex **3210**, is connected to other cores **3220** in core complex **3210**(*j*) via L3 cache **3230**(*j*) included in core complex **3210**(*j*). In at least one embodiment, cores **3220** included in core complex **3210**(*j*), where j is an integer representing a particular instance of core complex **3210**, can access all of L3 cache **3230**(*j*) included in core complex **3210**(*j*). In at least one embodiment, L3 cache **3230** may include, without limitation, any number of slices.

[0331] In at least one embodiment, fabric **3260** is a system interconnect that facilitates data and control transmissions across core complexes **3210**(**1**)-**3210**(N) (where N is an integer greater than zero), I/O interfaces **3270**, and memory controllers **3280**. In at least one embodiment, CPU **3200** may include, without limitation, any amount and type of system interconnect in addition to or instead of fabric **3260** that facilitates data and control transmissions across any number and type of directly or indirectly linked components that may be internal or external to CPU **3200**. In at least one embodiment, I/O interfaces **3270** are representative of any number and type of I/O interfaces (e.g., PCI, PCI-X, PCIe, GBE, USB, etc.). In at least one embodiment, various types of peripheral devices are coupled to I/O interfaces **3270** In at least one embodiment, peripheral devices that are coupled to I/O interfaces **3270** may include, without limitation, displays, keyboards, mice, printers, scanners, joysticks or other types of game controllers, media recording devices, external storage devices, network interface cards, and so forth.

[0332] In at least one embodiment, memory controllers **3280** facilitate data transfers between CPU **3200** and a system memory **3290**. In at least one embodiment, core complex **3210** and graphics complex **3240** share system memory **3290**. In at least one embodiment, CPU **3200** implements a memory subsystem that includes, without limitation, any amount and type of memory controllers **3280** and memory devices that may be dedicated to one component or shared among multiple components. In at least one embodiment, CPU **3200** implements a cache subsystem that includes, without limitation, one or more cache memories (e.g., L2 caches **3228** and L3 caches **3230**) that may each be private to or shared between any number of components (e.g., cores **3220** and core complexes **3210**).

[0333] FIG. **33** illustrates an exemplary accelerator integration slice **3390**. In at least one embodiment, a “slice” comprises a specified portion of processing resources of accelerator integration circuit **3336**. In at least one embodiment, an application is effective address space **3382** within system memory **3314** stores process elements **3383**. In at least one embodiment, process elements **3383** are stored in response to GPU invocations **3381** from applications **3380** executed on processor **3307**. In at least one embodiment, a process element **3383** contains process state for corresponding application **3380**. In at least one embodiment, a work descriptor (WD) **3384** contained in process element **3383** can be a single job requested by an application or may contain a pointer to a

queue of jobs. In at least one embodiment, WD **3384** is a pointer to a job request queue in an application's effective address space **3382**.
[0334] In at least one embodiment, graphics acceleration module **3346** and/or individual graphics processing engines **3331**(**1**)-**3331**(N) can be shared by all or a subset of processes in a system. In at least one embodiment, an infrastructure for setting up process states and sending a WD **3384** to a graphics acceleration module **3346** to start a job in a virtualized environment may be included.
[0335] In at least one embodiment, a dedicated-process programming model is implementation-specific. In at least one embodiment, in this model, a single process owns graphics acceleration module **3346** or an individual graphics processing engine **3331**. In at least one embodiment, when graphics acceleration module **3346** is owned by a single process, a hypervisor initializes accelerator integration circuit **3336** for an owning partition and an operating system initializes accelerator integration circuit **3336** for an owning process when graphics acceleration module **3346** is assigned.
[0336] In at least one embodiment, in operation, a WD fetch unit **3391** in accelerator integration slice **3390** fetches next WD **3384**, which includes an indication of work to be done by one or more graphics processing engines of graphics acceleration module **3346**. In at least one embodiment, data from WD **3384** may be stored in registers **3345** and used by MMU **3339**, interrupt management circuit **3347** and/or context management circuit **3348** as illustrated. For example, one embodiment of MMU **3339** includes segment/page walk circuitry for accessing segment/page tables **3386** within an OS virtual address space **3385**. In at least one embodiment, interrupt management circuit **3347** may process interrupt events **3392** received from graphics acceleration module **3346**. In at least one embodiment, when performing graphics operations, an effective address **3393** generated by a graphics processing engine **3331**(**1**)-**3331**(N) is translated to a real address by MMU **3339**.
[0337] In at least one embodiment, registers **3345** are duplicated for each graphics processing engine **3331**(**1**)-**3331**(N) and/or graphics acceleration module **3346** and may be initialized by a hypervisor or an operating system. In at least one embodiment, each of these duplicated registers may be included in an accelerator integration slice **3390**. Exemplary registers that may be initialized by a hypervisor are shown in Table 1.
TABLE-US-00001 TABLE 1 Hypervisor Initialized Registers Register # Description 1 Slice Control Register 2 Real Address (RA) Scheduled Processes Area Pointer 3 Authority Mask Override Register 4 Interrupt Vector Table Entry Offset 5 Interrupt Vector Table Entry Limit 6 State Register 7 Logical Partition ID 8 Real address (RA) Hypervisor Accelerator Utilization Record Pointer 9 Storage Description Register
[0338] Exemplary registers that may be initialized by an operating system are shown in Table 2.
TABLE-US-00002 TABLE 2 Operating System Initialized Registers Register # Description 1 Process and Thread Identification 2 Effective Address (EA) Context Save/Restore Pointer 3 Virtual Address (VA) Accelerator Utilization Record Pointer 4 Virtual Address (VA) Storage Segment Table Pointer 5 Authority Mask 6 Work descriptor
[0339] In at least one embodiment, each WD **3384** is specific to a particular graphics acceleration module **3346** and/or graphics processing engines **3331**(**1**)-**3331**(N). In at least one embodiment, it contains all information required by a graphics processing engine **3331**(**1**)-**3331**(N) to do work, or it can be a pointer to a memory location where an application has set up a command queue of work to be completed.
[0340] FIGS. **34**A-**34**B illustrate exemplary integrated circuits and associated graphics processors that may be fabricated using one or more IP cores, according to various embodiments described herein. In addition to what is illustrated, other logic and circuits may be included in at least one embodiment, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores.
[0341] FIGS. **34**A-**34**B are block diagrams illustrating exemplary graphics processors for use within an SoC, according to embodiments described herein. FIG. **34**A illustrates an exemplary graphics processor **3410** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to at least one embodiment. FIG. **34**B illustrates an additional exemplary graphics

processor **3440** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to at least one embodiment. In at least one embodiment, graphics processor **3410** of FIG. **34**A is a low power graphics processor core. In at least one embodiment, graphics processor **3440** of FIG. **34**B is a higher performance graphics processor core. In at least one embodiment, each of graphics processors **3410**, **3440** can be variants of graphics processor **2910** of FIG. **29**.

[0342] In at least one embodiment, graphics processor **3410** includes a vertex processor **3405** and one or more fragment processor(s) **3415**A-**3415**N (e.g., **3415**A, **3415**B, **3415**C, **3415**D, through **3415**N−1, and **3415**N). In at least one embodiment, graphics processor **3410** can execute different shader programs via separate logic, such that vertex processor **3405** is optimized to execute operations for vertex shader programs, while one or more fragment processor(s) **3415**A-**3415**N execute fragment (e.g., pixel) shading operations for fragment or pixel shader programs. In at least one embodiment, vertex processor **3405** performs a vertex processing stage of a 3D graphics pipeline and generates primitives and vertex data. In at least one embodiment, fragment processor(s) **3415**A-**3415**N use primitive and vertex data generated by vertex processor **3405** to produce a framebuffer that is displayed on a display device. In at least one embodiment, fragment processor(s) **3415**A-**3415**N are optimized to execute fragment shader programs as provided for in an OpenGL API, which may be used to perform similar operations as a pixel shader program as provided for in a Direct 3D API.

[0343] In at least one embodiment, graphics processor **3410** additionally includes one or more memory management units (MMUs) **3420**A-**3420**B, cache(s) **3425**A-**3425**B, and circuit interconnect(s) **3430**A-**3430**B. In at least one embodiment, one or more MMU(s) **3420**A-**3420**B provide for virtual to physical address mapping for graphics processor **3410**, including for vertex processor **3405** and/or fragment processor(s) **3415**A-**3415**N, which may reference vertex or image/texture data stored in memory, in addition to vertex or image/texture data stored in one or more cache(s) **3425**A-**3425**B. In at least one embodiment, one or more MMU(s) **3420**A-**3420**B may be synchronized with other MMUs within a system, including one or more MMUs associated with one or more application processor(s) **2905**, image processors **2915**, and/or video processors **2920** of FIG. **29**, such that each processor **2905-2920** can participate in a shared or unified virtual memory system. In at least one embodiment, one or more circuit interconnect(s) **3430**A-**3430**B enable graphics processor **3410** to interface with other IP cores within SoC, either via an internal bus of SoC or via a direct connection.

[0344] In at least one embodiment, graphics processor **3440** includes one or more shader core(s) **3455**A-**3455**N (e.g., **3455**A, **3455**B, **3455**C, **3455**D, **3455**E, **3455**F, through **3455**N−1, and **3455**N) as shown in FIG. **34**B, which provides for a unified shader core architecture in which a single core or type or core can execute all types of programmable shader code, including shader program code to implement vertex shaders, fragment shaders, and/or compute shaders. In at least one embodiment, a number of shader cores can vary. In at least one embodiment, graphics processor **3440** includes an inter-core task manager **3445**, which acts as a thread dispatcher to dispatch execution threads to one or more shader cores **3455**A-**3455**N and a tiling unit **3458** to accelerate tiling operations for tile-based rendering, in which rendering operations for a scene are subdivided in image space, for example to exploit local spatial coherence within a scene or to optimize use of internal caches.

[0345] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in integrated circuit **34**A and/or **34**B for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0346] FIGS. **35**A-**35**B illustrate additional exemplary graphics processor logic according to embodiments described herein. FIG. **35**A illustrates a graphics core **3500** that may be included within graphics processor **2910** of FIG. **29**, in at least one embodiment, and may be a unified shader core **3055**A-**3055**N as in FIG. **30**B in at least one embodiment. FIG. **35**B illustrates a highly-parallel general-purpose graphics processing unit ("GPGPU") **3530** suitable for deployment on a multi-chip module in at least one embodiment.

[0347] In at least one embodiment, graphics core **3500** includes a shared instruction cache **3502**, a texture unit **3518**, and a cache/shared memory **3520** (e.g., including L1, L2, L3, last level cache, or other caches) that are common to execution resources within graphics core **3500**. In at least one embodiment, graphics core **3500** can include multiple slices **3501**A-**3501**N or a partition for each core, and a graphics processor can include multiple instances of graphics core **3500**. In at least one embodiment, each slice **3501**A-**3501**N refers to graphics core **3500**. In at least one embodiment, slices **3501**A-**3501**N have sub-slices, which are part of a slice **3501**A-**3501**N. In at least one embodiment, slices **3501**A-**3501**N are independent of other slices or dependent on other slices. In at least one embodiment, slices **3501**A-**3501**N can include support logic including a local instruction cache **3504**A-**3504**N, a thread scheduler (sequencer) **3506**A-**3506**N, a thread dispatcher **3508**A-**3508**N, and a set of registers **3510**A-**3510**N. In at least one embodiment, slices **3501**A-**3501**N can include a set of additional function units (AFUs **3512**A-**3512**N), floating-point units (FPUs **3514**A-**3514**N), integer arithmetic logic units (ALUs **3516**A-**3516**N), address computational units (ACUs **3513**A-**3513**N), double-precision floating-point units (DPFPUs **3515**A-**3515**N), and matrix processing units (MPUs **3517**A-**3517**N).

[0348] In at least one embodiment, each slice **3501**A-**3501**N includes one or more engines for floating point and integer vector operations and one or more engines to accelerate convolution and matrix operations in AI, machine learning, or large dataset workloads. In at least one embodiment, one or more slices **3501**A-**3501**N include one or more vector engines to compute a vector (e.g., compute mathematical operations for vectors). In at least one embodiment, a vector engine can compute a vector operation in 16-bit floating point (also referred to as "FP16"), 32-bit floating point (also referred to as "FP32"), or 64-bit floating point (also referred to as "FP64"). In at least one embodiment, one or more slices **3501**A-**3501**N includes 16 vector engines that are paired with 16 matrix math units to compute matrix/tensor operations, where vector engines and math units are exposed via matrix extensions. In at least one embodiment, a slice a specified portion of processing resources of a processing unit, e.g., 16 cores and a ray tracing unit or 8 cores, a thread scheduler, a thread dispatcher, and additional functional units for a processor. In at least one embodiment, graphics core **3500** includes one or more matrix engines to compute matrix operations, e.g., when computing tensor operations.

[0349] In at least one embodiment, one or more slices **3501**A-**3501**N includes one or more ray tracing units to compute ray tracing operations (e.g., 16 ray tracing units per slice slices **3501**A-**3501**N). In at least one embodiment, a ray tracing unit computes ray traversal, triangle intersection, bounding box intersect, or other ray tracing operations.

[0350] In at least one embodiment, one or more slices **3501**A-**3501**N includes a media slice that encodes, decodes, and/or transcodes data; scales and/or format converts data; and/or performs video quality operations on video data.

[0351] In at least one embodiment, one or more slices **3501**A-**3501**N are linked to L2 cache and memory fabric, link connectors, high-bandwidth memory (HBM) (e.g., HBM2e, HDM3) stacks, and a media engine. In at least one embodiment, one or more slices **3501**A-**3501**N include multiple cores (e.g., 16 cores) and multiple ray tracing units (e.g., 16) paired to each core. In at least one embodiment, one or more slices **3501**A-**3501**N has one or more L1 caches. In at least one embodiment, one or more slices **3501**A-**3501**N include one or more vector engines; one or more instruction caches to store instructions; one or more L1 caches to cache data; one or more shared local memories (SLMs) to store data, e.g., corresponding to instructions; one or more samplers to sample data; one or more ray tracing units to perform ray tracing operations; one or more geometries to perform operations in geometry pipelines and/or apply geometric transformations to vertices or polygons; one or more rasterizers to describe an image in vector graphics format (e.g., shape) and convert it into a raster image (e.g., a series of pixels, dots, or lines, which when displayed together, create an image that is represented by shapes); one or more a Hierarchical Depth Buffer (Hiz) to buffer data; and/or one or more pixel backends. In at least one embodiment, a slice **3501**A-**3501**N includes a memory fabric, e.g., an L2 cache.

[0352] In at least one embodiment, FPUs **3514**A-**3514**N can perform single-precision (32-bit) and half-precision (16-bit) floating point operations, while DPFPUs **3515**A-**3515**N perform double precision (64-bit) floating point operations. In at least one embodiment, ALUs **3516**A-**3516**N can perform variable precision integer operations at 8-bit, 16-bit, and 32-bit precision, and can be configured for mixed precision operations. In at least one embodiment, MPUs **3517**A-**3517**N can also be configured for mixed precision matrix operations, including half-precision floating point and 8-bit integer operations. In at least one embodiment, MPUs **3517-3517**N can perform a variety of matrix operations to accelerate machine learning application frameworks, including enabling support for accelerated general matrix to matrix multiplication (GEMM). In at least one embodiment, AFUs **3512**A-**3512**N can perform additional logic operations not supported by floating-point or integer units, including trigonometric operations (e.g., sine, cosiInference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in graphics core **3500** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0353] In at least one embodiment, graphics core **3500** includes an interconnect and a link fabric sublayer that is attached to a switch and a GPU-GPU bridge that enables multiple graphics processors **3500** (e.g., 8) to be interlinked without glue to each other with load/store units (LSUs), data transfer units, and sync semantics across multiple graphics processors **3500**. In at least one embodiment, interconnects include standardized interconnects (e.g., PCIe) or some combination thereof.

[0354] In at least one embodiment, graphics core **3500** includes multiple tiles. In at least one embodiment, a tile is an individual die or one or more dies, where individual dies can be connected with an interconnect (e.g., embedded multi-die interconnect bridge (EMIB)). In at least one embodiment, graphics core **3500** includes a compute tile, a memory tile (e.g., where a memory tile can be exclusively accessed by different tiles or different chipsets such as a Rambo tile), substrate tile, a base tile, a HMB tile, a link tile, and EMIB tile, where all tiles are packaged together in graphics core **3500** as part of a GPU. In at least one embodiment, graphics core **3500** can include multiple tiles in a single package (also referred to as a “multi tile package”). In at least one embodiment, a compute tile can have 8 graphics cores **3500**, an L1 cache; and a base tile can have a host interface with PCIe 5.0, HBM2e, MDFI, and EMIB, a link tile with 8 links, 8 ports with an embedded switch. In at least one embodiment, tiles are connected with face-to-face (F2F) chip-on-chip bonding through fine-pitched, 36-micron, microbumps (e.g., copper pillars). In at least one embodiment, graphics core **3500** includes memory fabric, which includes memory, and is tile that is accessible by multiple tiles. In at least one embodiment, graphics core **3500** stores, accesses, or loads its own hardware contexts in memory, where a hardware context is a set of data loaded from registers before a process resumes, and where a hardware context can indicate a state of hardware (e.g., state of a GPU).

[0355] In at least one embodiment, graphics core **3500** includes serializer/deserializer (SERDES) circuitry that converts a serial data stream to a parallel data stream, or converts a parallel data stream to a serial data stream.

[0356] In at least one embodiment, graphics core **3500** includes a high speed coherent unified fabric (GPU to GPU), load/store units, bulk data transfer and sync semantics, and connected GPUs through an embedded switch, where a GPU-GPU bridge is controlled by a controller.

[0357] In at least one embodiment, graphics core **3500** performs an API, where said API abstracts hardware of graphics core **3500** and access libraries with instructions to perform math operations (e.g., math kernel library), deep neural network operations (e.g., deep neural network library), vector operations, collective communications, thread building blocks, video processing, data analytics library, and/or ray tracing operations.

[0358] FIG. **35**B illustrates a general-purpose processing unit (GPGPU) **3530** that can be configured to enable highly-parallel compute operations to be performed by an array of graphics processing units, in at least one embodiment. In at least one embodiment, GPGPU **3530** can be linked directly to other instances of GPGPU **3530** to create a multi-GPU cluster to improve training speed for deep neural networks. In at least one embodiment, GPGPU **3530** includes a host interface **3532** to enable a connection with a host processor. In at least one embodiment, host interface **3532** is a PCI Express interface. In at least one embodiment, host interface **3532** can be a vendor-specific communications interface or communications fabric. In at least one embodiment, GPGPU **3530** receives commands from a host processor and uses a global scheduler **3534** (which may be referred to as a thread sequencer and/or asynchronous compute engine) to distribute execution threads associated with those commands to a set of compute clusters **3536**A-**3536**H. In at least one embodiment, compute clusters **3536**A-**3536**H share a cache memory **3538**. In at least one embodiment, cache memory **3538** can serve as a higher-level cache for cache memories within compute clusters **3536**A-**3536**H.

[0359] In at least one embodiment, GPGPU **3530** includes memory **3544**A-**3544**B coupled with compute clusters **3536**A-**3536**H via a set of memory controllers **3542**A-**3542**B (e.g., one or more controllers for HBM2e). In at least one embodiment, memory **3544**A-**3544**B can include various types of memory devices including dynamic random access memory (DRAM) or graphics random access memory, such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory.

[0360] In at least one embodiment, compute clusters **3536**A-**3536**H each include a set of graphics cores, such as graphics core **3500** of FIG. **35**A, which can include multiple types of integer and floating point logic units that can perform computational operations at a range of precisions including suited for machine learning computations. For example, in at least one embodiment, at least a subset of floating point units in each of compute clusters **3536**A-**3536**H can be configured to perform 16-bit or 32-bit floating point operations, while a different subset of floating point units can be configured to perform 64-bit floating point operations.

[0361] In at least one embodiment, multiple instances of GPGPU **3530** can be configured to operate as a compute cluster. In at least one embodiment, communication used by compute clusters **3536**A-**3536**H for synchronization and data exchange varies across embodiments. In at least one embodiment, multiple instances of GPGPU **3530** communicate over host interface **3532**. In at least one embodiment, GPGPU **3530** includes an I/O hub **3539** that couples GPGPU **3530** with a GPU link **3540** that enables a direct connection to other instances of GPGPU **3530**. In at least one embodiment, GPU link **3540** is coupled to a dedicated GPU-to-GPU bridge that enables communication and synchronization between multiple instances of GPGPU **3530**. In at least one embodiment, GPU link **3540** couples with a high-speed interconnect to transmit and receive data to other GPGPUs or parallel processors. In at least one embodiment, multiple instances of GPGPU **3530** are located in separate data processing systems and communicate via a network device that is accessible via host interface **3532**. In at least one embodiment GPU link **3540** can be configured to enable a connection to a host processor in addition to or as an alternative to host interface **3532**.

[0362] In at least one embodiment, GPGPU **3530** can be configured to train neural networks. In at least one embodiment, GPGPU **3530** can be used within an inferencing platform. In at least one embodiment, in which GPGPU **3530** is used for inferencing, GPGPU **3530** may include fewer compute clusters **3536**A-**3536**H relative to when GPGPU **3530** is used for training a neural network. In at least one embodiment, memory technology associated with memory **3544**A-**3544**B may differ between inferencing and training configurations, with higher bandwidth memory technologies devoted to training configurations. In at least one embodiment, an inferencing configuration of GPGPU **3530** can support inferencing specific instructions. For example, in at least one embodiment, an inferencing configuration can provide support for one or more 8-bit integer dot

product instructions, which may be used during inferencing operations for deployed neural networks.
[0363] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in GPGPU **3530** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.
[0364] FIG. **36**A illustrates a parallel processor **3600** according to at least one embodiment. In at least one embodiment, various components of parallel processor **3600** may be implemented using one or more integrated circuit devices, such as programmable processors, application specific integrated circuits (ASICs), or field programmable gate arrays (FPGA). In at least one embodiment, illustrated parallel processor **3600** is a variant of one or more parallel processor(s) **3012** shown in FIG. **30** according to an exemplary embodiment. In at least one embodiment, a parallel processor **3600** includes one or more graphics cores **3400**.
[0365] In at least one embodiment, parallel processor **3600** includes a parallel processing unit **3602**. In at least one embodiment, parallel processing unit **3602** includes an I/O unit **3604** that enables communication with other devices, including other instances of parallel processing unit **3602**. In at least one embodiment, I/O unit **3604** may be directly connected to other devices. In at least one embodiment, I/O unit **3604** connects with other devices via use of a hub or switch interface, such as a memory hub **3605**. In at least one embodiment, connections between memory hub **3605** and I/O unit **3604** form a communication link **3613**. In at least one embodiment, I/O unit **3604** connects with a host interface **3606** and a memory crossbar **3616**, where host interface **3606** receives commands directed to performing processing operations and memory crossbar **3616** receives commands directed to performing memory operations.
[0366] In at least one embodiment, when host interface **3606** receives a command buffer via I/O unit **3604**, host interface **3606** can direct work operations to perform those commands to a front end **3608**. In at least one embodiment, front end **3608** couples with a scheduler **3610** (which may be referred to as a sequencer), which is configured to distribute commands or other work items to a processing cluster array **3612**. In at least one embodiment, scheduler **3610** ensures that processing cluster array **3612** is properly configured and in a valid state before tasks are distributed to a cluster of processing cluster array **3612**. In at least one embodiment, scheduler **3610** is implemented via firmware logic executing on a microcontroller. In at least one embodiment, microcontroller implemented scheduler **3610** is configurable to perform complex scheduling and work distribution operations at coarse and fine granularity, enabling rapid preemption and context switching of threads executing on processing array **3612**. In at least one embodiment, host software can prove workloads for scheduling on processing cluster array **3612** via one of multiple graphics processing paths. In at least one embodiment, workloads can then be automatically distributed across processing array cluster **3612** by scheduler **3610** logic within a microcontroller including scheduler **3610**.
[0367] In at least one embodiment, processing cluster array **3612** can include up to "N" processing clusters (e.g., cluster **3614**A, cluster **3614**B, through cluster **3614**N), where "N" represents a positive integer (which may be a different integer "N" than used in other figures). In at least one embodiment, each cluster **3614**A-**3614**N of processing cluster array **3612** can execute a large number of concurrent threads. In at least one embodiment, scheduler **3610** can allocate work to clusters **3614**A-**3614**N of processing cluster array **3612** using various scheduling and/or work distribution algorithms, which may vary depending on workload arising for each type of program or computation. In at least one embodiment, scheduling can be handled dynamically by scheduler **3610**, or can be assisted in part by compiler logic during compilation of program logic configured for execution by processing cluster array **3612**. In at least one embodiment, different clusters **3614**A-**3614**N of processing cluster array **3612** can be allocated for processing different types of programs or for performing different types of computations.

[0368] In at least one embodiment, processing cluster array **3612** can be configured to perform various types of parallel processing operations. In at least one embodiment, processing cluster array **3612** is configured to perform general-purpose parallel compute operations. For example, in at least one embodiment, processing cluster array **3612** can include logic to execute processing tasks including filtering of video and/or audio data, performing modeling operations, including physics operations, and performing data transformations.
[0369] In at least one embodiment, processing cluster array **3612** is configured to perform parallel graphics processing operations. In at least one embodiment, processing cluster array **3612** can include additional logic to support execution of such graphics processing operations, including but not limited to, texture sampling logic to perform texture operations, as well as tessellation logic and other vertex processing logic. In at least one embodiment, processing cluster array **3612** can be configured to execute graphics processing related shader programs such as, but not limited to, vertex shaders, tessellation shaders, geometry shaders, and pixel shaders. In at least one embodiment, parallel processing unit **3602** can transfer data from system memory via I/O unit **3604** for processing. In at least one embodiment, during processing, transferred data can be stored to on-chip memory (e.g., parallel processor memory **3622**) during processing, then written back to system memory.
[0370] In at least one embodiment, when parallel processing unit **3602** is used to perform graphics processing, scheduler **3610** can be configured to divide a processing workload into approximately equal sized tasks, to better enable distribution of graphics processing operations to multiple clusters **3614**A-**3614**N of processing cluster array **3612**. In at least one embodiment, portions of processing cluster array **3612** can be configured to perform different types of processing. For example, in at least one embodiment, a first portion may be configured to perform vertex shading and topology generation, a second portion may be configured to perform tessellation and geometry shading, and a third portion may be configured to perform pixel shading or other screen space operations, to produce a rendered image for display. In at least one embodiment, intermediate data produced by one or more of clusters **3614**A-**3614**N may be stored in buffers to allow intermediate data to be transmitted between clusters **3614**A-**3614**N for further processing.
[0371] In at least one embodiment, processing cluster array **3612** can receive processing tasks to be executed via scheduler **3610**, which receives commands defining processing tasks from front end **3608**. In at least one embodiment, processing tasks can include indices of data to be processed, e.g., surface (patch) data, primitive data, vertex data, and/or pixel data, as well as state parameters and commands defining how data is to be processed (e.g., what program is to be executed). In at least one embodiment, scheduler **3610** may be configured to fetch indices corresponding to tasks or may receive indices from front end **3608**. In at least one embodiment, front end **3608** can be configured to ensure processing cluster array **3612** is configured to a valid state before a workload specified by incoming command buffers (e.g., batch-buffers, push buffers, etc.) is initiated.
[0372] In at least one embodiment, each of one or more instances of parallel processing unit **3602** can couple with a parallel processor memory **3622**. In at least one embodiment, parallel processor memory **3622** can be accessed via memory crossbar **3616**, which can receive memory requests from processing cluster array **3612** as well as I/O unit **3604**. In at least one embodiment, memory crossbar **3616** can access parallel processor memory **3622** via a memory interface **3618**. In at least one embodiment, memory interface **3618** can include multiple partition units (e.g., partition unit **3620**A, partition unit **3620**B, through partition unit **3620**N) that can each couple to a portion (e.g., memory unit) of parallel processor memory **3622**. In at least one embodiment, a number of partition units **3620**A-**3620**N is configured to be equal to a number of memory units, such that a first partition unit **3620**A has a corresponding first memory unit **3624**A, a second partition unit **3620**B has a corresponding memory unit **3624**B, and an N-th partition unit **3620**N has a corresponding N-th memory unit **3624**N. In at least one embodiment, a number of partition units **3620**A-**3620**N may not be equal to a number of memory units.
[0373] In at least one embodiment, memory units **3624**A-**3624**N can include various types of

memory devices, including dynamic random access memory (DRAM) or graphics random access memory, such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory. In at least one embodiment, memory units **3624**A-**3624**N may also include 3D stacked memory, including but not limited to high bandwidth memory (HBM), HBM2e, or HDM3. In at least one embodiment, render targets, such as frame buffers or texture maps may be stored across memory units **3624**A-**3624**N, allowing partition units **3620**A-**3620**N to write portions of each render target in parallel to efficiently use available bandwidth of parallel processor memory **3622**. In at least one embodiment, a local instance of parallel processor memory **3622** may be excluded in favor of a unified memory design that utilizes system memory in conjunction with local cache memory.

[0374] In at least one embodiment, any one of clusters **3614**A-**3614**N of processing cluster array **3612** can process data that will be written to any of memory units **3624**A-**3624**N within parallel processor memory **3622**. In at least one embodiment, memory crossbar **3616** can be configured to transfer an output of each cluster **3614**A-**3614**N to any partition unit **3620**A-**3620**N or to another cluster **3614**A-**3614**N, which can perform additional processing operations on an output. In at least one embodiment, each cluster **3614**A-**3614**N can communicate with memory interface **3618** through memory crossbar **3616** to read from or write to various external memory devices. In at least one embodiment, memory crossbar **3616** has a connection to memory interface **3618** to communicate with I/O unit **3604**, as well as a connection to a local instance of parallel processor memory **3622**, enabling processing units within different processing clusters **3614**A-**3614**N to communicate with system memory or other memory that is not local to parallel processing unit **3602**. In at least one embodiment, memory crossbar **3616** can use virtual channels to separate traffic streams between clusters **3614**A-**3614**N and partition units **3620**A-**3620**N.

[0375] In at least one embodiment, multiple instances of parallel processing unit **3602** can be provided on a single add-in card, or multiple add-in cards can be interconnected. In at least one embodiment, different instances of parallel processing unit **3602** can be configured to interoperate even if different instances have different numbers of processing cores, different amounts of local parallel processor memory, and/or other configuration differences. For example, in at least one embodiment, some instances of parallel processing unit **3602** can include higher precision floating point units relative to other instances. In at least one embodiment, systems incorporating one or more instances of parallel processing unit **3602** or parallel processor **3600** can be implemented in a variety of configurations and form factors, including but not limited to desktop, laptop, or handheld personal computers, servers, workstations, game consoles, and/or embedded systems.

[0376] FIG. **36**B is a block diagram of a processing cluster **3614** within a parallel processing unit according to at least one embodiment. In at least one embodiment, a processing cluster is an instance of one of processing clusters **3614**A-**3614**N of FIG. **36**A. In at least one embodiment, processing cluster **3614** can be configured to execute many threads in parallel, where “thread” refers to an instance of a particular program executing on a particular set of input data. In at least one embodiment, single-instruction, multiple-data (SIMD) instruction issue techniques are used to support parallel execution of a large number of threads without providing multiple independent instruction units. In at least one embodiment, single-instruction, multiple-thread (SIMT) techniques are used to support parallel execution of a large number of generally synchronized threads, using a common instruction unit configured to issue instructions to a set of processing engines within each one of processing clusters.

[0377] In at least one embodiment, operation of processing cluster **3614** can be controlled via a pipeline manager **3632** that distributes processing tasks to SIMT parallel processors. In at least one embodiment, pipeline manager **3632** receives instructions from scheduler **3610** of FIG. **36**A and manages execution of those instructions via a graphics multiprocessor **3634** and/or a texture unit **3636**. In at least one embodiment, graphics multiprocessor **3634** is an exemplary instance of a SIMT parallel processor. However, in at least one embodiment, various types of SIMT parallel processors of differing architectures may be included within processing cluster **3614**. In at least one embodiment, one or more instances of graphics multiprocessor **3634** can be included within a processing cluster **3614**. In at least one embodiment, graphics multiprocessor **3634** can process data and a data crossbar **3640** can be used to distribute processed data to one of multiple possible destinations, including other shader units. In at least one embodiment, pipeline manager **3632** can facilitate distribution of processed data by specifying destinations for processed data to be distributed via data crossbar **3640**.

[0378] In at least one embodiment, each graphics multiprocessor **3634** within processing cluster **3614** can include an identical set of functional execution logic (e.g., arithmetic logic units, load-store units, etc.). In at least one embodiment, functional execution logic can be configured in a pipelined manner in which new instructions can be issued before previous instructions are complete. In at least one embodiment, functional execution logic supports a variety of operations including integer and floating point arithmetic, comparison operations, Boolean operations, bit-shifting, and computation of various algebraic functions. In at least one embodiment, same functional-unit hardware can be leveraged to perform different operations and any combination of functional units may be present.

[0379] In at least one embodiment, instructions transmitted to processing cluster **3614** constitute a thread. In at least one embodiment, a set of threads executing across a set of parallel processing engines is a thread group. In at least one embodiment, a thread group executes a common program on different input data. In at least one embodiment, each thread within a thread group can be assigned to a different processing engine within a graphics multiprocessor **3634**. In at least one embodiment, a thread group may include fewer threads than a number of processing engines within graphics multiprocessor **3634**. In at least one embodiment, when a thread group includes fewer threads than a number of processing engines, one or more of processing engines may be idle during cycles in which that thread group is being processed. In at least one embodiment, a thread group may also include more threads than a number of processing engines within graphics multiprocessor **3634**. In at least one embodiment, when a thread group includes more threads than number of processing engines within graphics multiprocessor **3634**, processing can be performed over consecutive clock cycles. In at least one embodiment, multiple thread groups can be executed concurrently on a graphics multiprocessor **3634**.

[0380] In at least one embodiment, graphics multiprocessor **3634** includes an internal cache memory to perform load and store operations. In at least one embodiment, graphics multiprocessor **3634** can forego an internal cache and use a cache memory (e.g., L1 cache **3648**) within processing cluster **3614**. In at least one embodiment, each graphics multiprocessor **3634** also has access to L2 caches within partition units (e.g., partition units **3620**A-**3620**N of FIG. **36**A) that are shared among all processing clusters **3614** and may be used to transfer data between threads. In at least one embodiment, graphics multiprocessor **3634** may also access off-chip global memory, which can include one or more of local parallel processor memory and/or system memory. In at least one embodiment, any memory external to parallel processing unit **3602** may be used as global memory. In at least one embodiment, processing cluster **3614** includes multiple instances of graphics multiprocessor **3634** and can share common instructions and data, which may be stored in L1 cache **3648**.

[0381] In at least one embodiment, each processing cluster **3614** may include an MMU **3645** (memory management unit) that is configured to map virtual addresses into physical addresses. In at least one embodiment, one or more instances of MMU **3645** may reside within memory interface **3618** of FIG. **36**A. In at least one embodiment, MMU **3645** includes a set of page table entries (PTEs) used to map a virtual address to a physical address of a tile and optionally a cache line index. In at least one embodiment, MMU **3645** may include address translation lookaside buffers (TLB) or caches that may reside within graphics multiprocessor **3634** or L1 **3648** cache or processing cluster **3614**. In at least one embodiment, a physical address is processed to distribute surface data access locally to allow for efficient request interleaving among partition units. In at least one embodiment, a cache line index may be used to determine whether a request for a cache line is a hit or miss.

[0382] In at least one embodiment, a processing cluster **3614** may be configured such that each graphics multiprocessor **3634** is coupled to a texture unit **3636** for performing texture mapping operations, e.g., determining texture sample positions, reading texture data, and filtering texture data. In at least one embodiment, texture data is read from an internal texture L1 cache (not shown) or from an L1 cache within graphics multiprocessor **3634** and is fetched from an L2 cache, local parallel processor memory, or system memory, as needed. In at least one embodiment, each graphics multiprocessor **3634** outputs processed tasks to data crossbar **3640** to provide processed task to another processing cluster **3614** for further processing or to store processed task in an L2 cache, local parallel processor memory, or system memory via memory crossbar **3616**. In at least one embodiment, a preROP **3642** (pre-raster operations unit) is configured to receive data from graphics multiprocessor **3634**, and direct data to ROP units, which may be located with partition units as described herein (e.g., partition units **3620**A-**3620**N of FIG. **36**A). In at least one embodiment, preROP **3642** unit can perform optimizations for color blending, organizing pixel color data, and performing address translations.

[0383] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in graphics processing cluster **3614** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0384] FIG. **36**C shows a graphics multiprocessor **3634** according to at least one embodiment. In at least one embodiment, graphics multiprocessor **3634** couples with pipeline manager **3632** of processing cluster **3614**. In at least one embodiment, graphics multiprocessor **3634** has an execution pipeline including but not limited to an instruction cache **3652**, an instruction unit **3654**, an address mapping unit **3656**, a register file **3658**, one or more general purpose graphics processing unit (GPGPU) cores **3662**, and one or more load/store units **3666**, where one or more load/store units **3666** can perform load/store operations to load/store instructions corresponding to performing an operation. In at least one embodiment, GPGPU cores **3662** and load/store units **3666** are coupled with cache memory **3672** and shared memory **3670** via a memory and cache interconnect **3668**.

[0385] In at least one embodiment, instruction cache **3652** receives a stream of instructions to execute from pipeline manager **3632**. In at least one embodiment, instructions are cached in instruction cache **3652** and dispatched for execution by an instruction unit **3654**. In at least one embodiment, instruction unit **3654** can dispatch instructions as thread groups (e.g., warps, wavefronts, waves), with each thread of thread group assigned to a different execution unit within GPGPU cores **3662**. In at least one embodiment, an instruction can access any of a local, shared, or global address space by specifying an address within a unified address space. In at least one embodiment, address mapping unit **3656** can be used to translate addresses in a unified address space into a distinct memory address that can be accessed by load/store units **3666**.

[0386] In at least one embodiment, register file **3658** provides a set of registers for functional units of graphics multiprocessor **3634**. In at least one embodiment, register file **3658** provides temporary storage for operands connected to data paths of functional units (e.g., GPGPU cores **3662**, load/store units **3666**) of graphics multiprocessor **3634**. In at least one embodiment, register file **3658** is divided between each of functional units such that each functional unit is allocated a dedicated portion of register file **3658**. In at least one embodiment, register file **3658** is divided between different warps (which may be referred to as wavefronts and/or waves) being executed by graphics multiprocessor **3634**.

[0387] In at least one embodiment, GPGPU cores **3662** can each include floating point units (FPUs) and/or integer arithmetic logic units (ALUs) that are used to execute instructions of graphics multiprocessor **3634**. In at least one embodiment, GPGPU cores **3662** can be similar in architecture or can differ in architecture. In at least one embodiment, a first portion of GPGPU cores **3662** include a single precision FPU and an integer ALU while a second portion of GPGPU cores include a double precision FPU. In at least one embodiment, FPUs can implement IEEE 754-2008 standard floating point arithmetic or enable variable precision floating point arithmetic. In at least one embodiment, graphics multiprocessor **3634** can additionally include one or more fixed function or special function units to perform specific functions such as copy rectangle or pixel blending operations. In at least one embodiment, one or more of GPGPU cores **3662** can also include fixed or special function logic.

[0388] In at least one embodiment, GPGPU cores **3662** include SIMD logic capable of performing a single instruction on multiple sets of data. In at least one embodiment, GPGPU cores **3662** can physically execute SIMD4, SIMD8, and SIMD16 instructions and logically execute SIMD1, SIMD2, and SIMD32 instructions. In at least one embodiment, SIMD instructions for GPGPU cores can be generated at compile time by a shader compiler or automatically generated when executing programs written and compiled for single program multiple data (SPMD) or SIMT architectures. In at least one embodiment, multiple threads of a program configured for an SIMT execution model can executed via a single SIMD instruction. For example, in at least one embodiment, eight SIMT threads that perform same or similar operations can be executed in parallel via a single SIMD8 logic unit.

[0389] In at least one embodiment, memory and cache interconnect **3668** is an interconnect network that connects each functional unit of graphics multiprocessor **3634** to register file **3658** and to shared memory **3670**. In at least one embodiment, memory and cache interconnect **3668** is a crossbar interconnect that allows load/store unit **3666** to implement load and store operations between shared memory **3670** and register file **3658**. In at least one embodiment, register file **3658** can operate at a same frequency as GPGPU cores **3662**, thus data transfer between GPGPU cores **3662** and register file **3658** can have very low latency. In at least one embodiment, shared memory **3670** can be used to enable communication between threads that execute on functional units within graphics multiprocessor **3634**. In at least one embodiment, cache memory **3672** can be used as a data cache for example, to cache texture data communicated between functional units and texture unit **3636**. In at least one embodiment, shared memory **3670** can also be used as a program managed cache. In at least one embodiment, threads executing on GPGPU cores **3662** can programmatically store data within shared memory in addition to automatically cached data that is stored within cache memory **3672**.

[0390] In at least one embodiment, a parallel processor or GPGPU as described herein is communicatively coupled to host/processor cores to accelerate graphics operations, machine-learning operations, pattern analysis operations, and various general purpose GPU (GPGPU) functions. In at least one embodiment, a GPU may be communicatively coupled to host processor/cores over a bus or other interconnect (e.g., a high-speed interconnect such as PCIe or NVLink). In at least one embodiment, a GPU may be integrated on a package or chip as cores and communicatively coupled to cores over an internal processor bus/interconnect internal to a package or chip. In at least one embodiment, regardless a manner in which a GPU is connected, processor cores may allocate work to such GPU in a form of sequences of commands/instructions contained in a work descriptor. In at least one embodiment, that GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

[0391] Inference and/or training logic **1815** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **1815** are provided herein in conjunction with FIGS. **18**A and/or **18**B. In at least one embodiment, inference and/or training logic **1815** may be used in graphics multiprocessor **3634** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

General Computing

[0392] The following FIG.s set forth, without limitation, exemplary software constructs within general computing that can be used to implement at least one embodiment.

[0393] FIG. **37** illustrates a software stack of a programming platform, in accordance with at least one embodiment. In at least one embodiment, a programming platform is a platform for leveraging hardware on a computing system to accelerate computational tasks. A programming platform may be accessible to software developers through libraries, compiler directives, and/or extensions to programming languages, in at least one embodiment. In at least one embodiment, a programming platform may be, but is not limited to, CUDA, Radeon Open Compute Platform ("ROCm"), OpenCL (OpenCL™ is developed by Khronos group), SYCL, or Intel One API.

[0394] In at least one embodiment, a software stack **3700** of a programming platform provides an execution environment for an application **3701**. In at least one embodiment, application **3701** may include any computer software capable of being launched on software stack **3700**. In at least one embodiment, application **3701** may include, but is not limited to, an artificial intelligence ("AI")/machine learning ("ML") application, a high performance computing ("HPC") application, a virtual desktop infrastructure ("VDI"), or a datacenter workload.

[0395] In at least one embodiment, application **3701** and software stack **3700** run on hardware **3707**. Hardware **3707** may include one or more GPUs, CPUs, FPGAs, AI engines, and/or other types of compute devices that support a programming platform, in at least one embodiment. In at least one embodiment, such as with CUDA, software stack **3700** may be vendor specific and compatible with only devices from particular vendor(s). In at least one embodiment, such as in with OpenCL, software stack **3700** may be used with devices from different vendors. In at least one embodiment, hardware **3707** includes a host connected to one more devices that can be accessed to perform computational tasks via application programming interface ("API") calls. A device within hardware **3707** may include, but is not limited to, a GPU, FPGA, AI engine, or other compute device (but may also include a CPU) and its memory, as opposed to a host within hardware **3707** that may include, but is not limited to, a CPU (but may also include a compute device) and its memory, in at least one embodiment.

[0396] In at least one embodiment, software stack **3700** of a programming platform includes, without limitation, a number of libraries **3703**, a runtime **3705**, and a device kernel driver **3706**. Each of libraries **3703** may include data and programming code that can be used by computer programs and leveraged during software development, in at least one embodiment. In at least one embodiment, libraries **3703** may include, but are not limited to, pre-written code and subroutines, classes, values, type specifications, configuration data, documentation, help data, and/or message templates. In at least one embodiment, libraries **3703** include functions that are optimized for execution on one or more types of devices. In at least one embodiment, libraries **3703** may include, but are not limited to, functions for performing mathematical, deep learning, and/or other types of operations on devices. In at least one embodiment, libraries **3803** are associated with corresponding APIs **3802**, which may include one or more APIs, that expose functions implemented in libraries **3803**.

[0397] In at least one embodiment, application **3701** is written as source code that is compiled into executable code, as discussed in greater detail below in conjunction with FIG. **42**. Executable code of application **3701** may run, at least in part, on an execution environment provided by software stack **3700**, in at least one embodiment. In at least one embodiment, during execution of application **3701**, code may be reached that needs to run on a device, as opposed to a host. In such a case, runtime **3705** may be called to load and launch requisite code on a device, in at least one embodiment. In at least one embodiment, runtime **3705** may include any technically feasible runtime system that is able to support execution of application S**01**.

[0398] In at least one embodiment, runtime **3705** is implemented as one or more runtime libraries associated with corresponding APIs, which are shown as API(s) **3704**. One or more of such runtime libraries may include, without limitation, functions for memory management, execution control, device management, error handling, and/or synchronization, among other things, in at least one embodiment. In at least one embodiment, memory management functions may include, but are not limited to, functions to allocate, deallocate, and copy device memory, as well as transfer data between host memory and device memory. In at least one embodiment, execution control functions may include, but are not limited to, functions to launch a function (sometimes referred to as a

“kernel” when a function is a global function callable from a host) on a device and set attribute values in a buffer maintained by a runtime library for a given function to be executed on a device.
[0399] Runtime libraries and corresponding API(s) **3704** may be implemented in any technically feasible manner, in at least one embodiment. In at least one embodiment, one (or any number of) API may expose a low-level set of functions for fine-grained control of a device, while another (or any number of) API may expose a higher-level set of such functions. In at least one embodiment, a high-level runtime API may be built on top of a low-level API. In at least one embodiment, one or more of runtime APIs may be language-specific APIs that are layered on top of a language-independent runtime API.
[0400] In at least one embodiment, device kernel driver **3706** is configured to facilitate communication with an underlying device. In at least one embodiment, device kernel driver **3706** may provide low-level functionalities upon which APIs, such as API(s) **3704**, and/or other software relies. In at least one embodiment, device kernel driver **3706** may be configured to compile intermediate representation (“IR”) code into binary code at runtime. For CUDA, device kernel driver **3706** may compile Parallel Thread Execution (“PTX”) IR code that is not hardware specific into binary code for a specific target device at runtime (with caching of compiled binary code), which is also sometimes referred to as “finalizing” code, in at least one embodiment. Doing so may permit finalized code to run on a target device, which may not have existed when source code was originally compiled into PTX code, in at least one embodiment. Alternatively, in at least one embodiment, device source code may be compiled into binary code offline, without requiring device kernel driver **3706** to compile IR code at runtime.
[0401] FIG. **38** illustrates a CUDA implementation of software stack **3700** of FIG. **37**, in accordance with at least one embodiment. In at least one embodiment, a CUDA software stack **3800**, on which an application **3801** may be launched, includes CUDA libraries **3803**, a CUDA runtime **3805**, a CUDA driver **3807**, and a device kernel driver **3808**. In at least one embodiment, CUDA software stack **3800** executes on hardware **3809**, which may include a GPU that supports CUDA and is developed by NVIDIA Corporation of Santa Clara, CA.
[0402] In at least one embodiment, application **3801**, CUDA runtime **3805**, and device kernel driver **3808** may perform similar functionalities as application **3701**, runtime **3705**, and device kernel driver **3706**, respectively, which are described above in conjunction with FIG. **37**. In at least one embodiment, CUDA driver **3807** includes a library (libcuda.so) that implements a CUDA driver API **3806**. Similar to a CUDA runtime API **3804** implemented by a CUDA runtime library (cudart), CUDA driver API **3806** may, without limitation, expose functions for memory management, execution control, device management, error handling, synchronization, and/or graphics interoperability, among other things, in at least one embodiment. In at least one embodiment, CUDA driver API **3806** differs from CUDA runtime API **3804** in that CUDA runtime API **3804** simplifies device code management by providing implicit initialization, context (analogous to a process) management, and module (analogous to dynamically loaded libraries) management. In contrast to high-level CUDA runtime API **3804**, CUDA driver API **3806** is a low-level API providing more fine-grained control of a device, particularly with respect to contexts and module loading, in at least one embodiment. In at least one embodiment, CUDA driver API **3806** may expose functions for context management that are not exposed by CUDA runtime API **3804**. In at least one embodiment, CUDA driver API **3806** is also language-independent and supports, e.g., OpenCL in addition to CUDA runtime API **3804**. Further, in at least one embodiment, development libraries, including CUDA runtime **3805**, may be considered as separate from driver components, including user-mode CUDA driver **3807** and kernel-mode device driver **3808** (also sometimes referred to as a “display” driver).
[0403] In at least one embodiment, CUDA libraries **3803** may include, but are not limited to, mathematical libraries, deep learning libraries, parallel algorithm libraries, and/or signal/image/video processing libraries, which parallel computing applications such as application **3801** may utilize. In at least one embodiment, CUDA libraries **3803** may include mathematical libraries such as a cuBLAS library that is an implementation of Basic Linear Algebra Subprograms (“BLAS”) for

performing linear algebra operations, a cuFFT library for computing fast Fourier transforms ("FFTs"), and a cuRAND library for generating random numbers, among others. In at least one embodiment, CUDA libraries **3803** may include deep learning libraries such as a cuDNN library of primitives for deep neural networks and a TensorRT platform for high-performance deep learning inference, among others.

[0404] FIG. **39** illustrates a ROCm implementation of software stack **3700** of FIG. **37**, in accordance with at least one embodiment. In at least one embodiment, a ROCm software stack **3900**, on which an application **3901** may be launched, includes a language runtime **3903**, a system runtime **3905**, a thunk **3907**, a ROCm kernel driver **3908**, and a device kernel driver **3909**. In at least one embodiment, ROCm software stack **3900** executes on hardware **3910**, which may include a GPU that supports ROCm and is developed by AMD Corporation of Santa Clara, CA.

[0405] In at least one embodiment, application **3901** may perform similar functionalities as application **3701** discussed above in conjunction with FIG. **37**. In addition, language runtime **3903** and system runtime **3905** may perform similar functionalities as runtime **3705** discussed above in conjunction with FIG. **37**, in at least one embodiment. In at least one embodiment, language runtime **3903** and system runtime **3905** differ in that system runtime **3905** is a language-independent runtime that implements a ROCr system runtime API **3904** and makes use of a Heterogeneous System Architecture ("HAS") Runtime API. HAS runtime API is a thin, user-mode API that exposes interfaces to access and interact with an AMD GPU, including functions for memory management, execution control via architected dispatch of kernels, error handling, system and agent information, and runtime initialization and shutdown, among other things, in at least one embodiment. In contrast to system runtime **3905**, language runtime **3903** is an implementation of a language-specific runtime API **3902** layered on top of ROCr system runtime API **3904**, in at least one embodiment. In at least one embodiment, language runtime API may include, but is not limited to, a Heterogeneous compute Interface for Portability ("HIP") language runtime API, a Heterogeneous Compute Compiler ("HCC") language runtime API, or an OpenCL API, among others. HIP language in particular is an extension of C++ programming language with functionally similar versions of CUDA mechanisms, and, in at least one embodiment, a HIP language runtime API includes functions that are similar to those of CUDA runtime API **3804** discussed above in conjunction with FIG. **38**, such as functions for memory management, execution control, device management, error handling, and synchronization, among other things.

[0406] In at least one embodiment, thunk (ROCt) **3907** is an interface that can be used to interact with underlying ROCm driver **3908**. In at least one embodiment, ROCm driver **3908** is a ROCK driver, which is a combination of an AMDGPU driver and a HAS kernel driver (amdkfd). In at least one embodiment, AMDGPU driver is a device kernel driver for GPUs developed by AMD that performs similar functionalities as device kernel driver **3706** discussed above in conjunction with FIG. **37**. In at least one embodiment, HAS kernel driver is a driver permitting different types of processors to share system resources more effectively via hardware features.

[0407] In at least one embodiment, various libraries (not shown) may be included in ROCm software stack **3900** above language runtime **3903** and provide functionality similarity to CUDA libraries **3803**, discussed above in conjunction with FIG. **38**. In at least one embodiment, various libraries may include, but are not limited to, mathematical, deep learning, and/or other libraries such as a hipBLAS library that implements functions similar to those of CUDA cuBLAS, a rocFFT library for computing FFTs that is similar to CUDA cuFFT, among others.

[0408] FIG. **40** illustrates an OpenCL implementation of software stack **3700** of FIG. **37**, in accordance with at least one embodiment. In at least one embodiment, an OpenCL software stack **4000**, on which an application **4001** may be launched, includes an OpenCL framework **4005**, an OpenCL runtime **4006**, and a driver **4007**. In at least one embodiment, OpenCL software stack **4000** executes on hardware **3809** that is not vendor-specific. As OpenCL is supported by devices developed by different vendors, specific OpenCL drivers may be required to interoperate with hardware from such vendors, in at least one embodiment.

[0409] In at least one embodiment, application **4001**, OpenCL runtime **4006**, device kernel driver **4007**, and hardware **4008** may perform similar functionalities as application **3701**, runtime **3705**, device kernel driver **3706**, and hardware **3707**, respectively, that are discussed above in conjunction with FIG. **37**. In at least one embodiment, application **4001** further includes an OpenCL kernel **4002** with code that is to be executed on a device.

[0410] In at least one embodiment, OpenCL defines a "platform" that allows a host to control devices connected to a host. In at least one embodiment, an OpenCL framework provides a platform layer API and a runtime API, shown as platform API **4003** and runtime API **4005**. In at least one embodiment, runtime API **4005** uses contexts to manage execution of kernels on devices. In at least one embodiment, each identified device may be associated with a respective context, which runtime API **4005** may use to manage command queues, program objects, and kernel objects, share memory objects, among other things, for that device. In at least one embodiment, platform API **4003** exposes functions that permit device contexts to be used to select and initialize devices, submit work to devices via command queues, and enable data transfer to and from devices, among other things. In addition, OpenCL framework provides various built-in functions (not shown), including math functions, relational functions, and image processing functions, among others, in at least one embodiment.

[0411] In at least one embodiment, a compiler **4004** is also included in OpenCL frame-work **4005**. Source code may be compiled offline prior to executing an application or online during execution of an application, in at least one embodiment. In contrast to CUDA and ROCm, OpenCL applications in at least one embodiment may be compiled online by compiler **4004**, which is included to be representative of any number of compilers that may be used to compile source code and/or IR code, such as Standard Portable Intermediate Representation ("SPIR-V") code, into binary code. Alternatively, in at least one embodiment, OpenCL applications may be compiled offline, prior to execution of such applications.

[0412] FIG. **41** illustrates software that is supported by a programming platform, in accordance with at least one embodiment. In at least one embodiment, a programming platform **4104** is configured to support various programming models **4103**, middlewares and/or libraries **4102**, and frameworks **4101** that an application **4100** may rely upon. In at least one embodiment, application **4100** may be an AI/ML application implemented using, in at least one embodiment, a deep learning framework such as MXNet, PyTorch, or TensorFlow, which may rely on libraries such as cuDNN, NVIDIA Collective Communications Library ("NCCL"), and/or NVIDA Developer Data Loading Library ("DALI") CUDA libraries to provide accelerated computing on underlying hardware.

[0413] In at least one embodiment, programming platform **4104** may be one of a CUDA, ROCm, or OpenCL platform described above in conjunction with FIG. **33**, FIG. **34**, and FIG. **40**, respectively. In at least one embodiment, programming platform **4104** supports multiple programming models **4103**, which are abstractions of an underlying computing system permitting expressions of algorithms and data structures. Programming models **4103** may expose features of underlying hardware in order to improve performance, in at least one embodiment. In at least one embodiment, programming models **4103** may include, but are not limited to, CUDA, HIP, OpenCL, C++ Accelerated Massive Parallelism ("C++ AMP"), Open Multi-Processing ("OpenMP"), Open Accelerators ("OpenACC"), and/or Vulcan Compute.

[0414] In at least one embodiment, libraries and/or middlewares **4102** provide implementations of abstractions of programming models **4104**. In at least one embodiment, such libraries include data and programming code that may be used by computer programs and leveraged during software development. In at least one embodiment, such middlewares include software that provides services to applications beyond those available from programming platform **4104**. In at least one embodiment, libraries and/or middlewares **4102** may include, but are not limited to, cuBLAS, cuFFT, cuRAND, and other CUDA libraries, or rocBLAS, rocFFT, rocRAND, and other ROCm libraries. In addition, in at least one embodiment, libraries and/or middlewares **4102** may include NCCL and ROCm Communication Collectives Library ("RCCL") libraries providing communication routines for GPUs, a MIOpen library for deep learning acceleration, and/or an Eigen library for linear algebra, matrix and vector operations, geometrical transformations, numerical solvers, and related algorithms.

[0415] In at least one embodiment, application frameworks **4101** depend on libraries and/or middlewares **4102**. In at least one embodiment, each of application frameworks **4101** is a software framework used to implement a standard structure of application software. An AI/ML application may be implemented using a framework such as Caffe, Caffe2, TensorFlow, Keras, PyTorch, or MxNet deep learning frameworks, in at least one embodiment.

[0416] FIG. **42** illustrates compiling code to execute on one of programming platforms of FIGS. **37-40**, in accordance with at least one embodiment. In at least one embodiment, a compiler **4201** receives source code **4200** that includes both host code as well as device code. In at least one embodiment, complier **4201** is configured to convert source code **4200** into host executable code **4202** for execution on a host and device executable code **4203** for execution on a device. In at least one embodiment, source code **4200** may either be compiled offline prior to execution of an application, or online during execution of an application.

[0417] In at least one embodiment, source code **4200** may include code in any programming language supported by compiler **4201**, such as C++, C, Fortran, etc. In at least one embodiment, source code **4200** may be included in a single-source file having a mixture of host code and device code, with locations of device code being indicated therein. In at least one embodiment, a single-source file may be a .cu file that includes CUDA code or a .hip.cpp file that includes HIP code. Alternatively, in at least one embodiment, source code **4200** may include multiple source code files, rather than a single-source file, into which host code and device code are separated.

[0418] In at least one embodiment, compiler **4201** is configured to compile source code **4200** into host executable code **4202** for execution on a host and device executable code **4203** for execution on a device. In at least one embodiment, compiler **4201** performs operations including parsing source code **4200** into an abstract system tree (AST), performing optimizations, and generating executable code. In at least one embodiment in which source code **4200** includes a single-source file, compiler **4201** may separate device code from host code in such a single-source file, compile device code and host code into device executable code **4203** and host executable code **4202**, respectively, and link device executable code **4203** and host executable code **4202** together in a single file, as discussed in greater detail below with respect to FIG. **26**.

[0419] In at least one embodiment, host executable code **4202** and device executable code **4203** may be in any suitable format, such as binary code and/or IR code. In a case of CUDA, host executable code **4202** may include native object code and device executable code **4203** may include code in PTX intermediate representation, in at least one embodiment. In a case of ROCm, both host executable code **4202** and device executable code **4203** may include target binary code, in at least one embodiment.

[0420] At least one embodiment of the disclosure can be viewed in view of the following clauses:
1.

[0421] In at least one embodiment, one or more techniques described herein utilize a oneAPI programming model. In at least one embodiment, a oneAPI programming model refers to a programming model for interacting with various compute accelerator architectures. In at least one embodiment, oneAPI refers to an application programming interface (API) designed to interact with various compute accelerator architectures. In at least one embodiment, a oneAPI programming model utilizes a DPC++ programming language. In at least one embodiment, a DPC++ programming language refers to a high-level language for data parallel programming productivity. In at least one embodiment, a DPC++ programming language is based at least in part on C and/or C++ programming languages. In at least one embodiment, a oneAPI programming model is a programming model such as those developed by Intel Corporation of Santa Clara, CA.

[0422] In at least one embodiment, oneAPI and/or oneAPI programming model is utilized to interact with various accelerator, GPU, processor, and/or variations thereof, architectures. In at least one embodiment, oneAPI includes a set of libraries that implement various functionalities. In at least one embodiment, oneAPI includes at least a oneAPI DPC++ library, a oneAPI math kernel library, a oneAPI data analytics library, a oneAPI deep neural network library, a oneAPI collective communications library, a oneAPI threading building blocks library, a oneAPI video processing library, and/or variations thereof.

[0423] In at least one embodiment, a oneAPI DPC++ library, also referred to as oneDPL, is a library that implements algorithms and functions to accelerate DPC++ kernel programming. In at least one embodiment, oneDPL implements one or more standard template library (STL) functions. In at least one embodiment, oneDPL implements one or more parallel STL functions. In at least one embodiment, oneDPL provides a set of library classes and functions such as parallel algorithms, iterators, function object classes, range-based API, and/or variations thereof. In at least one embodiment, oneDPL implements one or more classes and/or functions of a C++ standard library. In at least one embodiment, oneDPL implements one or more random number generator functions.

[0424] In at least one embodiment, a oneAPI math kernel library, also referred to as oneMKL, is a library that implements various optimized and parallelized routines for various mathematical functions and/or operations. In at least one embodiment, oneMKL implements one or more basic linear algebra subprograms (BLAS) and/or linear algebra package (LAPACK) dense linear algebra routines. In at least one embodiment, oneMKL implements one or more sparse BLAS linear algebra routines. In at least one embodiment, oneMKL implements one or more random number generators (RNGs). In at least one embodiment, oneMKL implements one or more vector mathematics (VM) routines for mathematical operations on vectors. In at least one embodiment, oneMKL implements one or more Fast Fourier Transform (FFT) functions.

[0425] In at least one embodiment, a oneAPI data analytics library, also referred to as oneDAL, is a library that implements various data analysis applications and distributed computations. In at least one embodiment, oneDAL implements various algorithms for preprocessing, transformation, analysis, modeling, validation, and decision making for data analytics, in batch, online, and distributed processing modes of computation. In at least one embodiment, oneDAL implements various C++ and/or Java APIs and various connectors to one or more data sources. In at least one embodiment, oneDAL implements DPC++ API extensions to a traditional C++ interface and enables GPU usage for various algorithms.

[0426] In at least one embodiment, a oneAPI deep neural network library, also referred to as oneDNN, is a library that implements various deep learning functions. In at least one embodiment, oneDNN implements various neural network, machine learning, and deep learning functions, algorithms, and/or variations thereof.

[0427] In at least one embodiment, a oneAPI collective communications library, also referred to as oneCCL, is a library that implements various applications for deep learning and machine learning workloads. In at least one embodiment, oneCCL is built upon lower-level communication middleware, such as message passing interface (MPI) and libfabrics. In at least one embodiment, oneCCL enables a set of deep learning specific optimizations, such as prioritization, persistent operations, out of order executions, and/or variations thereof. In at least one embodiment, oneCCL implements various CPU and GPU functions.

[0428] In at least one embodiment, a oneAPI threading building blocks library, also referred to as oneTBB, is a library that implements various parallelized processes for various applications. In at least one embodiment, oneTBB is utilized for task-based, shared parallel programming on a host. In at least one embodiment, oneTBB implements generic parallel algorithms. In at least one embodiment, oneTBB implements concurrent containers. In at least one embodiment, oneTBB implements a scalable memory allocator. In at least one embodiment, oneTBB implements a work-stealing task scheduler. In at least one embodiment, oneTBB implements low-level synchronization primitives. In at least one embodiment, oneTBB is compiler-independent and usable on various processors, such as GPUs, PPUs, CPUs, and/or variations thereof.

[0429] In at least one embodiment, a oneAPI video processing library, also referred to as oneVPL, is a library that is utilized for accelerating video processing in one or more applications. In at least one embodiment, oneVPL implements various video decoding, encoding, and processing functions. In at least one embodiment, oneVPL implements various functions for media pipelines on CPUs, GPUs, and other accelerators. In at least one embodiment, one VPL implements device discovery and selection in media centric and video analytics workloads. In at least one embodiment, oneVPL implements API primitives for zero-copy buffer sharing.

[0430] In at least one embodiment, a oneAPI programming model utilizes a DPC++ programming language. In at least one embodiment, a DPC++ programming language is a programming language that includes, without limitation, functionally similar versions of CUDA mechanisms to define device code and distinguish between device code and host code. In at least one embodiment, a DPC++ programming language may include a subset of functionality of a CUDA programming language. In at least one embodiment, one or more CUDA programming model operations are performed using a oneAPI programming model using a DPC++ programming language.

[0431] In at least one embodiment, any application programming interface (API) described herein is compiled into one or more instructions, operations, or any other signal by a compiler, interpreter, or other software tool. In at least one embodiment, compilation comprises generating one or more machine-executable instructions, operations, or other signals from source code. In at least one embodiment, an API compiled into one or more instructions, operations, or other signals, when performed, causes one or more processors such as graphics processors, graphics cores, parallel processor, processor, processor core, or any other logic circuit further described herein to perform one or more computing operations.

[0432] It should be noted that, while example embodiments described herein may relate to a CUDA programming model, techniques described herein can be utilized with any suitable programming model, such HIP, oneAPI, and/or variations thereof

[0433] Other variations are within spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.

[0434] Use of terms “a” and “an” and “the” and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (meaning “including, but not limited to,”) unless otherwise noted. “Connected,” when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. In at least one embodiment, use of term “set” (e.g., “a set of items”) or “subset” unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, term “subset” of a corresponding set does not necessarily denote a proper subset of corresponding set, but subset and corresponding set may be equal.

[0435] Conjunctive language, such as phrases of form “at least one of A, B, and C,” or “at least one of A, B and C,” unless specifically stated otherwise or otherwise clearly contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases “at least one of A, B, and C” and “at least one of A, B and C” refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, term “plurality” indicates a state of being plural (e.g., “a plurality of items” indicates multiple items). In at least one embodiment, number of items in a plurality is at least two, but can be more when so indicated either explicitly or by context. Further, unless stated otherwise or otherwise clear from context, phrase “based on” means “based at least in part on” and not “based solely on.”

[0436] Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being executed) by one or more processors of a computer system, cause computer system to perform operations described herein. In at least one embodiment, set of non-transitory computer-readable storage media comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors—for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit (“CPU”) executes some of instructions while a graphics processing unit (“GPU”) executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

[0437] In at least one embodiment, an arithmetic logic unit is a set of combinational logic circuitry that takes one or more inputs to produce a result. In at least one embodiment, an arithmetic logic unit is used by a processor to implement mathematical operation such as addition, subtraction, or multiplication. In at least one embodiment, an arithmetic logic unit is used to implement logical operations such as logical AND/OR or XOR. In at least one embodiment, an arithmetic logic unit is stateless, and made from physical switching components such as semiconductor transistors arranged to form logical gates. In at least one embodiment, an arithmetic logic unit may operate internally as a stateful logic circuit with an associated clock. In at least one embodiment, an arithmetic logic unit may be constructed as an asynchronous logic circuit with an internal state not maintained in an associated register set. In at least one embodiment, an arithmetic logic unit is used by a processor to combine operands stored in one or more registers of the processor and produce an output that can be stored by the processor in another register or a memory location.

[0438] In at least one embodiment, as a result of processing an instruction retrieved by the processor, the processor presents one or more inputs or operands to an arithmetic logic unit, causing the arithmetic logic unit to produce a result based at least in part on an instruction code provided to inputs of the arithmetic logic unit. In at least one embodiment, the instruction codes provided by the processor to the ALU are based at least in part on the instruction executed by the processor. In at least one embodiment combinational logic in the ALU processes the inputs and produces an output

which is placed on a bus within the processor. In at least one embodiment, the processor selects a destination register, memory location, output device, or output storage location on the output bus so that clocking the processor causes the results produced by the ALU to be sent to the desired location.
[0439] In the scope of this application, the term arithmetic logic unit, or ALU, is used to refer to any computational logic circuit that processes operands to produce a result. For example, in the present document, the term ALU can refer to a floating point unit, a DSP, a tensor core, a shader core, a coprocessor, or a CPU.
[0440] Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.
[0441] Use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments of disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.
[0442] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.
[0443] In description and claims, terms “coupled” and “connected,” along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, “connected” or “coupled” may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. “Coupled” may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.
[0444] Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as “processing,” “computing,” “calculating,” “determining,” or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system's registers and/or memories into other data similarly represented as physical quantities within computing system's memories, registers or other such information storage, transmission or display devices.
[0445] In a similar manner, term “processor” may refer to any device or portion of a device that processes electronic data from registers and/or memory and transform that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, “processor” may be a CPU or a GPU. A “computing platform” may comprise one or more processors. As used herein, “software” processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. In at least one embodiment, terms “system” and “method” are used herein interchangeably insofar as system may embody one or more methods and methods may be considered a system.
[0446] In present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. In at least one embodiment, process of obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In at least one embodiment, processes of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In at least one embodiment, processes of obtaining,

acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. In at least one embodiment, references may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, processes of providing, outputting, transmitting, sending, or presenting analog or digital data can be accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

[0447] Although descriptions herein set forth example implementations of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities may be defined above for purposes of description, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

[0448] Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

Claims

1. A system, comprising: one or more processors to: generate a first image, using a conditional generator, based on a noisy input; generate a first noisy image from the first image; and determine a loss based, at least in part, on a comparison between the first noisy image, the noisy input, and an intermediate noisy input associated with the noisy input.

2. The system of claim 1, wherein the one or more processors are further to: generate, from a sample input, a series of noisy inputs corresponding to a denoising distribution having a plurality of steps.

3. The system of claim 2, wherein the noisy input is later in time in the series of noisy inputs than the intermediate noisy input.

4. The system of claim 3, wherein the noisy input is directly after the intermediate noisy input in the series of noisy inputs.

5. The system of claim 1, wherein the first noisy image is generated using a reverse process to add noise to the first image.

6. The system of claim 5, wherein the reverse process including posterior sampling.

7. The system of claim 1, wherein the one or more processors are further to: add a latent variable for generating the first image.

8. The system of claim 1, wherein the one or more processors are further to: retrain a generator based, at least in part, on the loss.

9. A computer-implemented method, comprising: receiving, to a generator of a conditional generative adversarial network (GAN), a noisy sample based, at least in part, on a clean input; generating, using the generator and the noisy sample, a generated sample that is less noisy than the noisy sample; comparing, with a discriminator of the conditional GAN, the generated sample to a true denoising sample associated with the noisy sample; and determining a loss based, at least in part, on the comparison.

10. The computer-implemented method of claim 9, further comprising: generating, from the clean, a series of noisy inputs corresponding to a denoising distribution having a plurality of steps.

11. The computer-implemented method of claim 10, further comprising: selecting, from the series of noisy inputs, the true denoising sample at a step prior to the noisy sample.

12. The computer-implemented method of claim 9, further comprising: generating a generated output image, using the generator and the noisy sample; and adding noise to the generated output image to form the generated sample.

13. The computer-implemented method of claim 12, wherein the noise is added using a posterior sampling process.

14. The computer-implemented method of claim 12, further comprising: providing a latent variable, to the generator, to produce the generated output image.

15. The computer-implemented method of claim 9, further comprising: retraining the generator based, at least in part, on the loss.

16. A system, comprising: one or more processors to generate, using a conditional generative adversarial network (GAN), an output image from a noisy input, provided to a conditional generator of the conditional GAN, the conditional generator being trained on a loss computed by a discriminator of the conditional GAN between the noisy input, a noised output image, and an intermediate noisy input associated with the noisy input.

17. The system of claim 16, wherein the one or more processors further generate the noised output image using posterior sampling.

18. The system of claim 16, wherein the noisy input and the intermediate noisy input are generated as a series of steps during a forward discussion process.

19. The system of claim 18, wherein the intermediate noisy input is prior in time than the noisy input.

20. The system of claim 16, wherein the system comprises at least one of: a system for performing simulation operations; a system for performing simulation operations to test or validate autonomous machine applications; an infotainment system of a machine; an entertainment system of a machine; a system for generating synthetic data; a system for collaborative content creation of multi-dimensional assets; a system for performing digital twin simulation; a system for presenting at least one of virtual reality content, augmented reality content, or mixed reality content; a system for rendering graphical output; a system for performing deep learning operations; a system implemented using an edge device; a system incorporating one or more Virtual Machines (VMs); a system implemented at least partially in a data center; or a system implemented at least partially using cloud computing resources.